

COMP H2704 – Web Application Security

TU757 / TU863

CA3 Project (50%)

Technical Report

Group: Group 8

Members:

Pavlo Panchenko

Stepan Tretiakov

Miguel Caracuel Millan

Roman Oleinikov

Lecturer: Dr Vanessa Ayala-Rivera

Total Word Count: 10220

Pavlo Panchenko B00176622 6
Solved Challenges 6
□□□□□ Change Bender's Password: 6
□□□□□ Extra Language: 8
□□□□□ Email Leak: 10
□□□□□ Kill Chatbot: 12
□□□□□ Frontend Typosquatting : 13
Vulnerability Analysis: 15
Broken Authentication: 15
Rationale or investigative process 15
Stepan Tretiakov B00177464 18
Solved Challenges 18
□□□□□ Cross-Site Imaging: 18
□□□□□□ Login Support Team: 21
□□□□ Reset Uvogin's Password: 26
□□□□ Leaked Unsafe Product: 29
□□□□□ Retrieve Blueprint: 33
Vulnerability Analysis 35
Sensitive Data Exposure 35
Security Misconfiguration 36
Miguel Caracuel Millan B00176896 37
Solved Challenges 37
□□ Empty user registration: 37
□□□ Admin Registration: 39
□□□ Payback Time: 42
□□□□ Access Log: 45
□□□ Upload Size: 49
Vulnerability Analysis 52
Observability Failures 52
Improper input validation 54
Roman Oleinikov B00177480 55

Solved Challenges 55
□ □ Reflected XSS 55
□ □ □ Client-side XSS Protection 56
□ □ □ API-only XSS 57
□ □ □ Database Schema 59
□ □ □ □ Ephemeral Accountant 61
Vulnerability Analysis 62
XSS 62
SQL injections 62
SAST 63
Tool and methodology 63
SQL injection in Search and login. 63
Cross-Site Scripting vulnerabilities 64
Path traversal 64
Juice Shop solved Challenges 64
DAST 66
Introduction 66
Testing Methodology 66
Scan Process 66
Manual Validation 66
Findings Overview 67
Detailed Findings 67
SQL Injection 67
Missing Content Security Policy 67
CORS Misconfiguration 67
No HTTPS 68
Missing Anti-Clickjacking Protection 68
Session ID in URL 68
Manual SQL Injection Validation 68
Conclusion 69
References 69

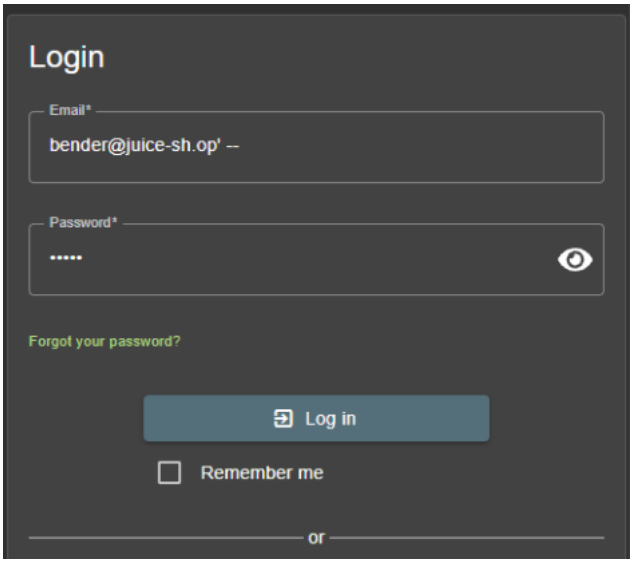
Pavlo Panchenko B00176622

Solved Challenges

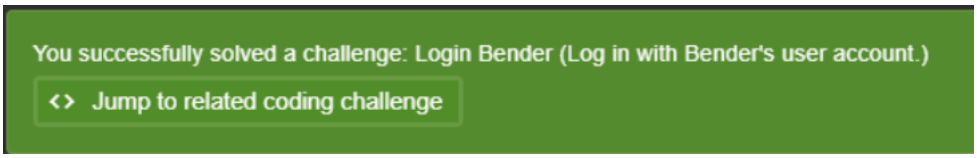
□ □ □ □ □ Change Bender's Password:

The task requires us to change the Bender user password to “slurmCl4ssic” without SQL injection or "Forgot Password"

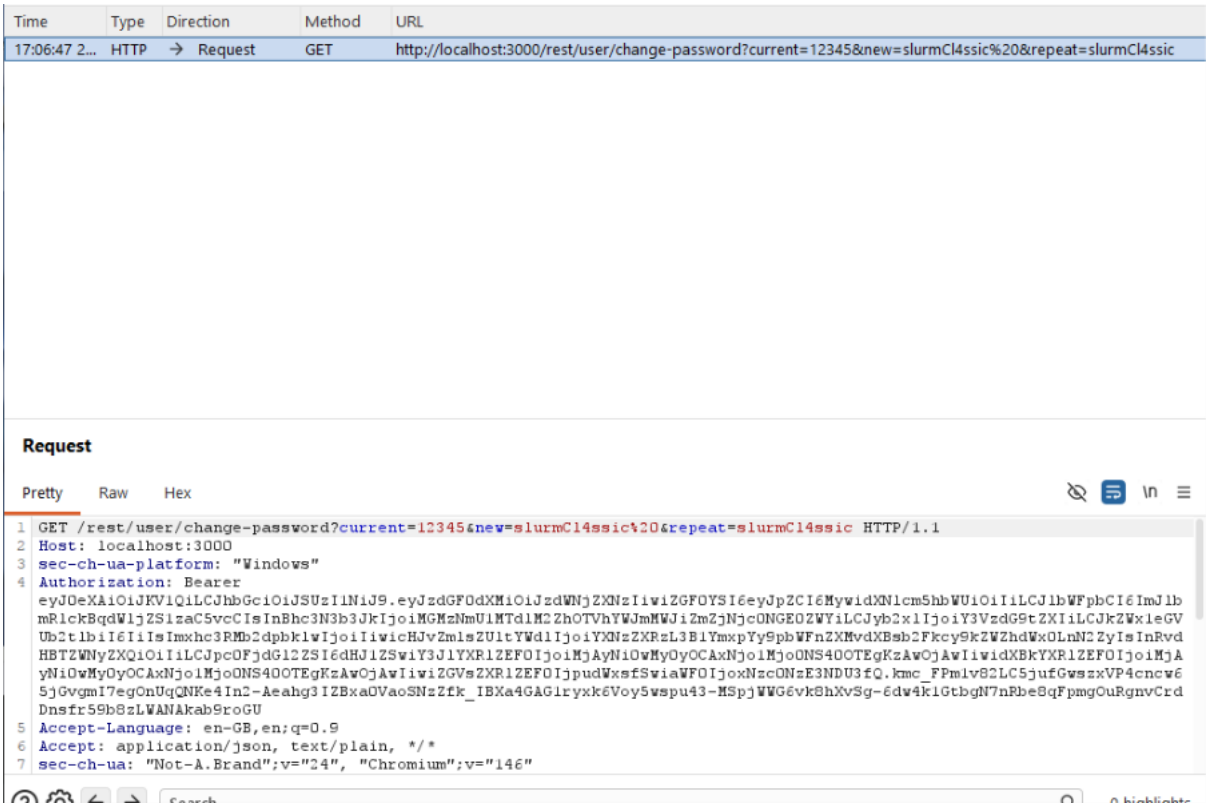
First, we log into Bender's account using the email "bender@juice-sh.op’ --” The password is 12345, but it won't be checked at all because ‘ --



The completed task proves that we have logged into Bender's account.



Next, all we need is to go to change password, enter a random password in the current password field and a new password slurmCl4ssic in other two fields.
Send a request, receive a refusal, and go to the Burb suite to intercept the request.

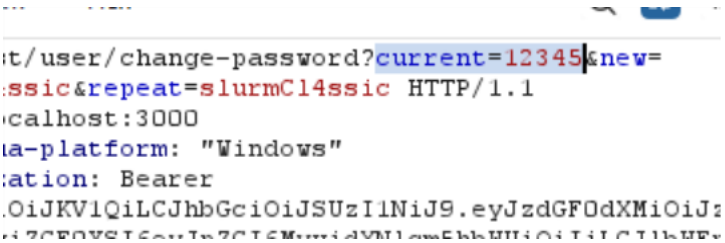


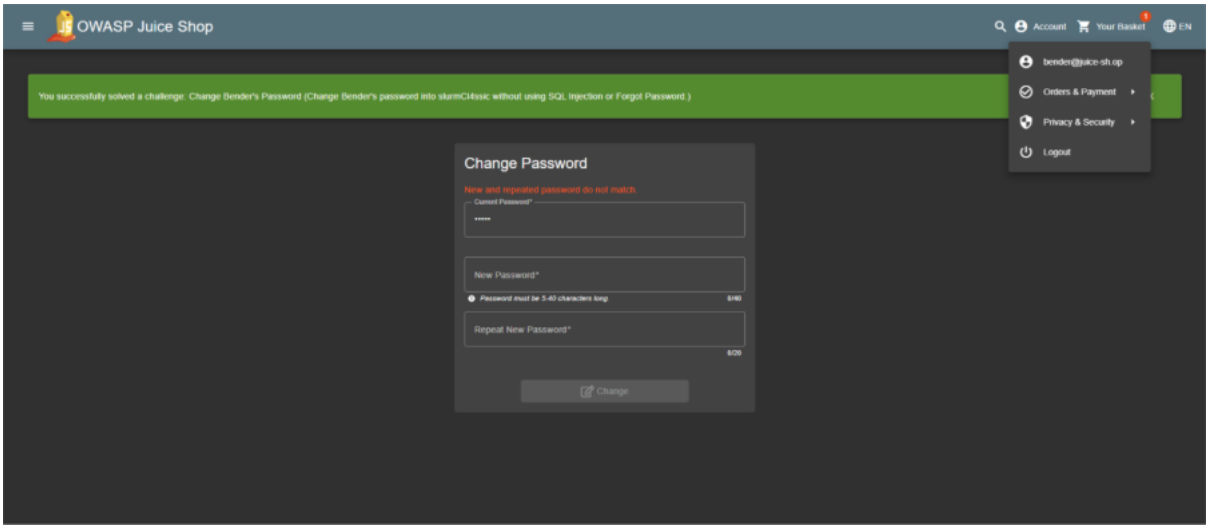
We transfer it to the repeater and see this response

```
HTTP/1.1 401 Unauthorized
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: text/html; charset=utf-8
Content-Length: 32
ETag: W/"20-6tKKLCLLgOnzR5qInvJyo/E13vg"
Vary: Accept-Encoding
Date: Sat, 28 Mar 2026 17:08:01 GMT
Connection: keep-alive
Keep-Alive: timeout=5

Current password is not correct.
```

I decided to test what would happen if I removed the “current” parameter entirely. After sending the request without it, the server returned a successful response and changed the password.

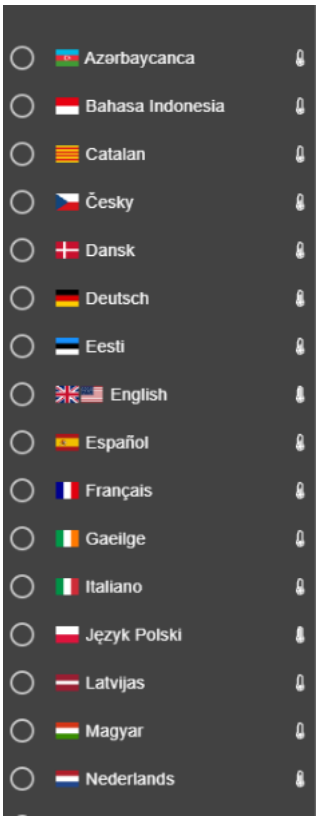




The password has changed, and the green completed challenge bar on the top proves it.

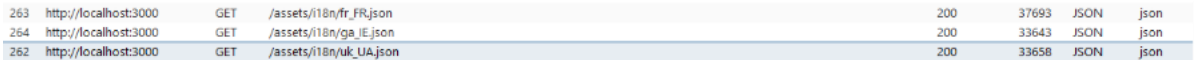
□ □ □ □ □ Extra Language:

When we open "select language", we see a list of all available languages.



In the Burb Suite, each such language has its own code, for example, Irish - ga_IE, French – fr_FR

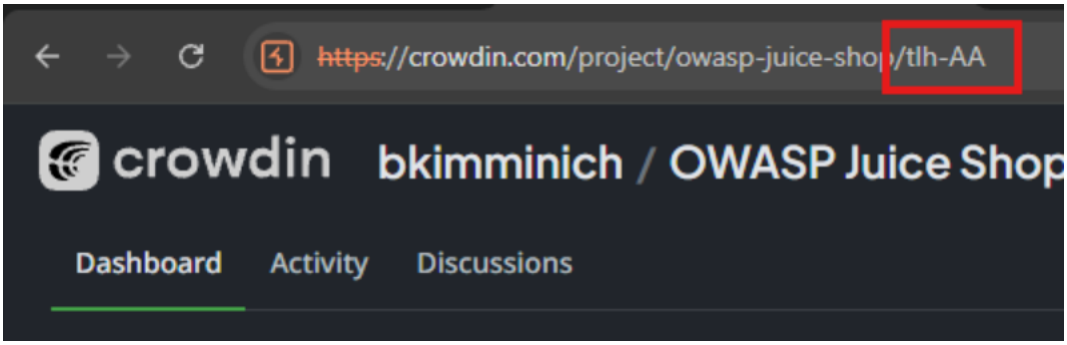
I decided to intercept the request in Burp Suite and check if it was possible to substitute an arbitrary value.



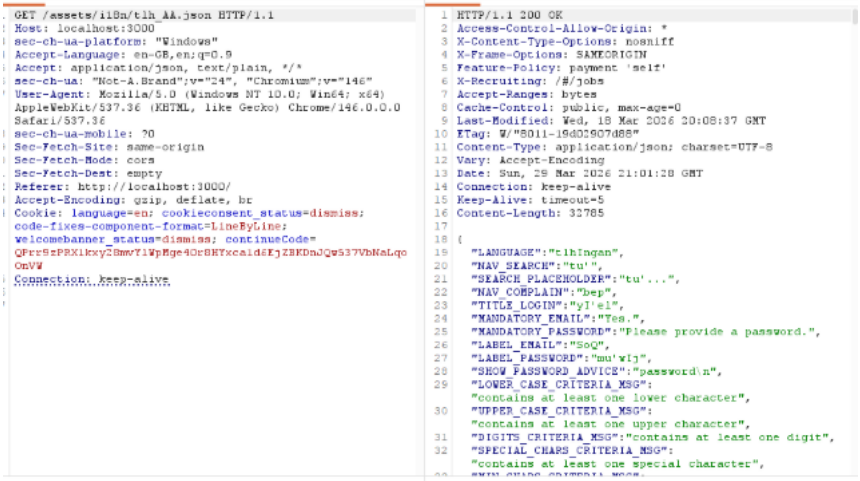
task hint said to search the browser for available languages. The language listed here is Klingon.



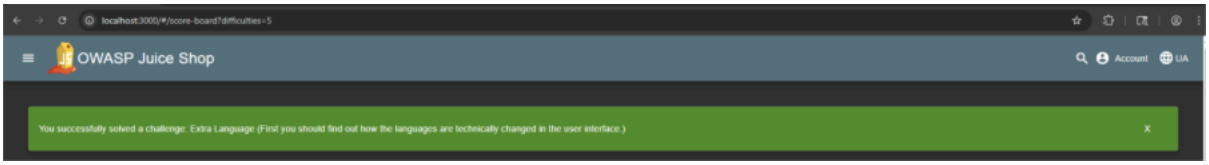
And the language page also has its own code.

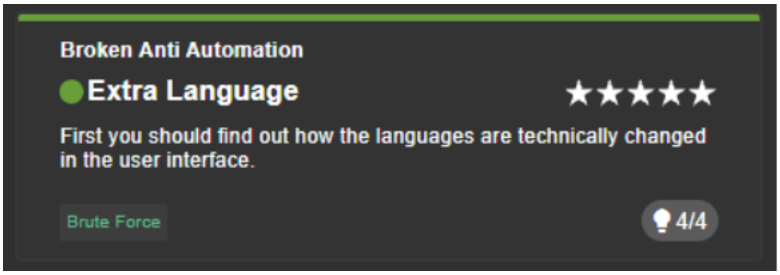


All that remains is to change the conditional "fr_FR" to "tlh_AA"



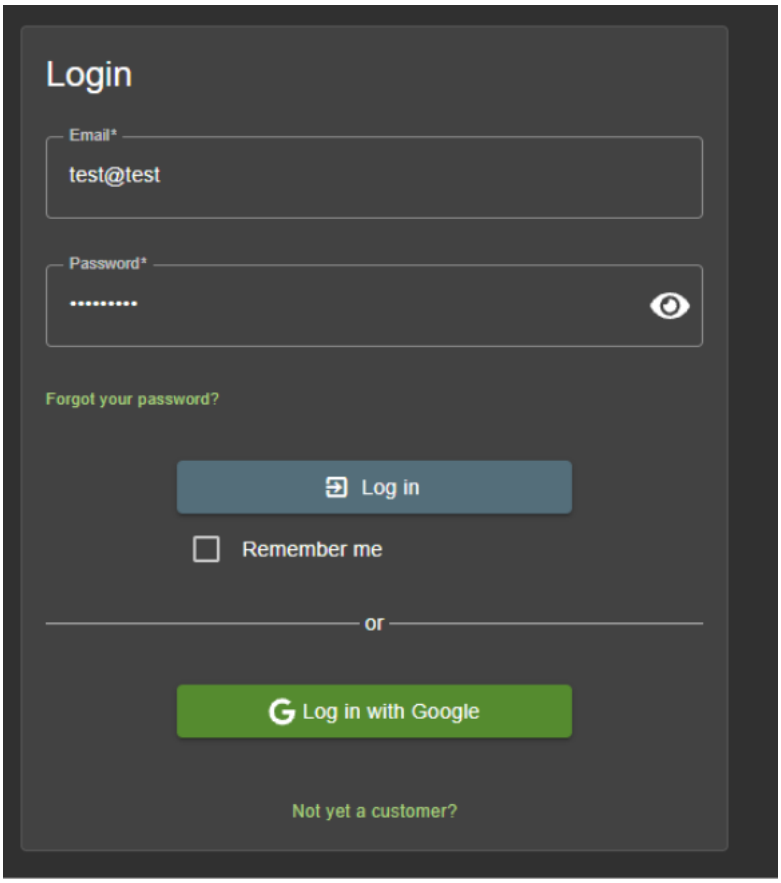
After submitting the request, the language was successfully changed and the task was accepted.

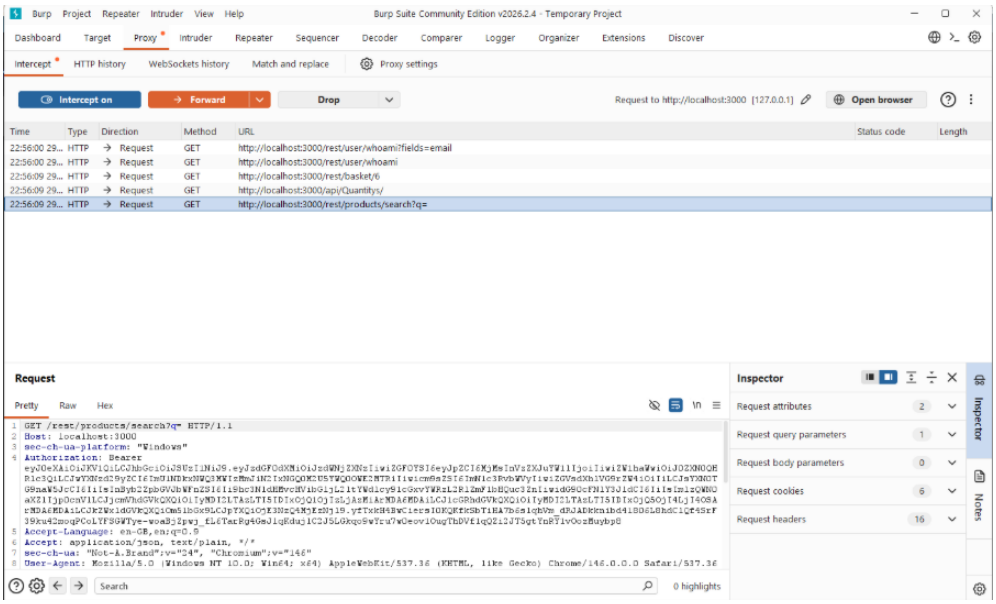




□ □ □ □ □ Email Leak:

When analyzing the authentication process, I started by intercepting the login request using Burp Suite to understand what data was being sent to the server and how the response was generated.





In one of the requests, I noticed that the Authorization: Bearer header was being used, which is responsible for user authentication. I removed Authorization: Bearer from the request and resent it. Despite the lack of a token, the server still returned a response with the user's data, including the email address.

```
sec-ch-ua-platform: "Windows"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZX
NzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lIjoiIiwiZW1haWwiOiJ0Z
XN0QHRlc3QiLCJwYXNzd29yZCI6ImU1NDkxNWQ3MWIzMmJiN2IxNGQ0M2U5
YWQ0OWE2MTRiIiwicm9sZSI6ImN1c3RvbWVyIiwiZGVsdXhlVG9rZW4iOiI
iLCJsYXN0TG9naW5JcCI6IiIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcH
VibGljL2ltYWdlcy91cGxvYWRzL2R1ZmF1bHQuc3ZnIiwidG90cFNlY3Jld
CI6IiIsImlzQWN0aXZlIjpOcnVlLCJjcmVhdGVkQXQiOiIyMDI2LTAzLTI5
IDIxOjQ1OjIzLjAzMiArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTAzLTI
5IDIxOjQ5OjI4LjI4OSArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYX
QiOjE3NzQ4MjE2Mzh9.xCERNJYGkRRwXN6v23YJo--mdQj9CJwN7IcHeEqU
ObX9HebKry7WOXpeaWiSFWA_VTfyjJ2jgSXJHfaMAoGvGOEteKyEbSYksCQ
6bRhqeH6AI5W8cRmdgypnp88cLDjGUNSGoMaPolNvarRPYW-wXs1xw5cIDA
5hGwhLawVeQBI
Accept-Language: en-GB,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
X
X
F
X
C
C
E
V
D
C
K


/

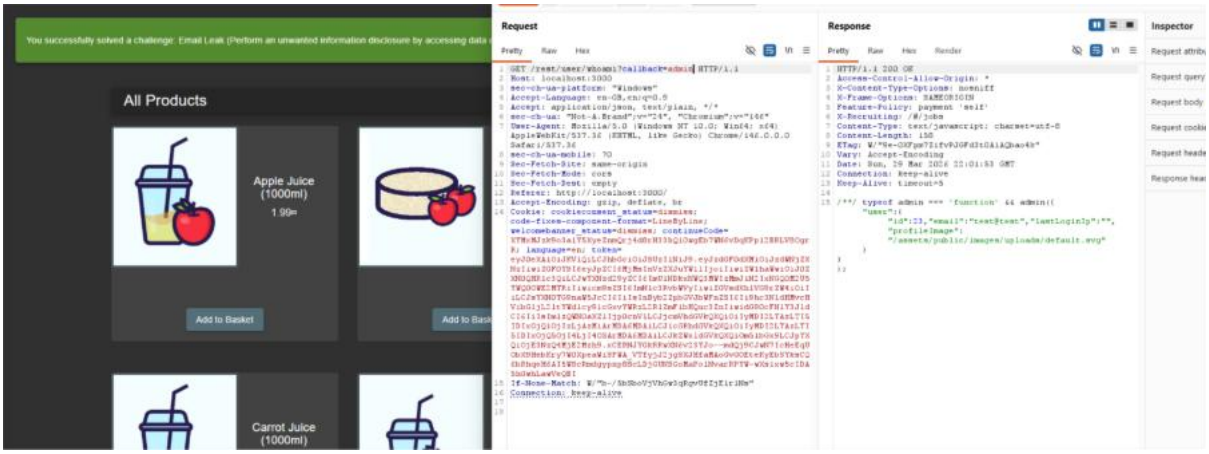

}
```

Then I noticed that the request used the fields=email parameter, which, judging by its name, affects the data returned by the server. I found some clues online. You need to specify the parameter itself and replace fields with callback.

I set callback=admin and sent the request.

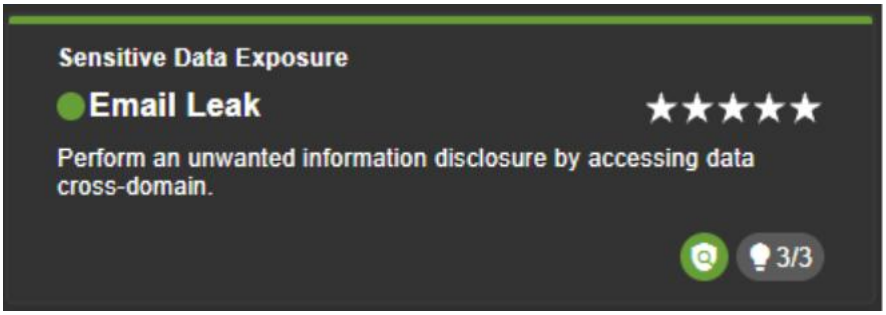
```
r/whoami?fields=email HT
st:3000
```

```
GET /rest/user/whoami?callback=admin HTTP/1.1
Host: localhost:3000
sec-ch-ua-platform: "Windows"
```



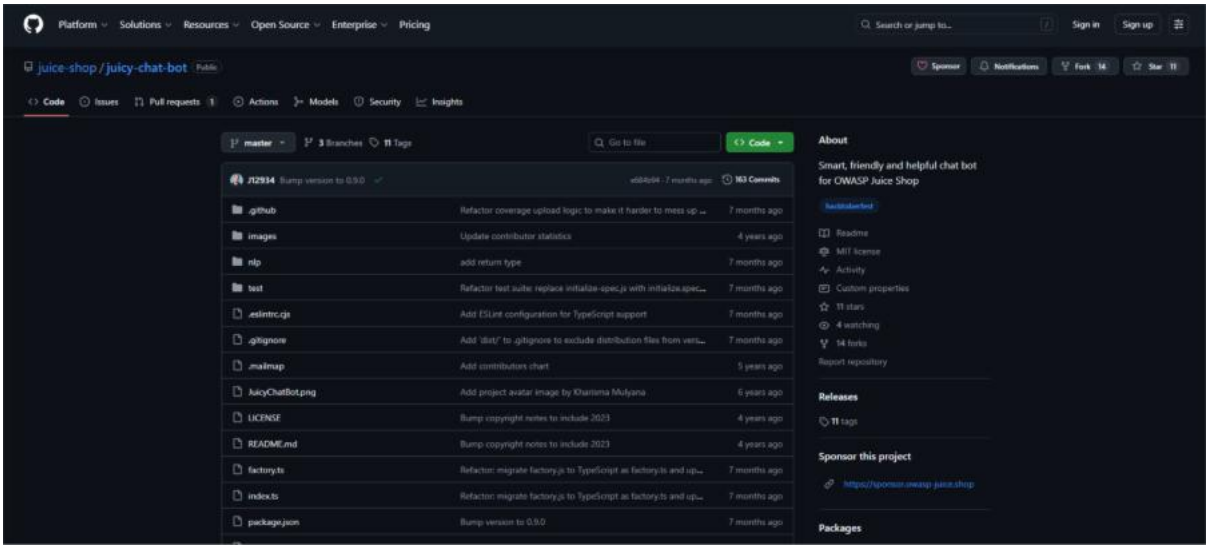
After submitting the request, the task was completed.

The site doesn't properly check whether the user is logged in, and it allows you to change the request parameters. This makes it possible to obtain someone else's email address even without access to the account.



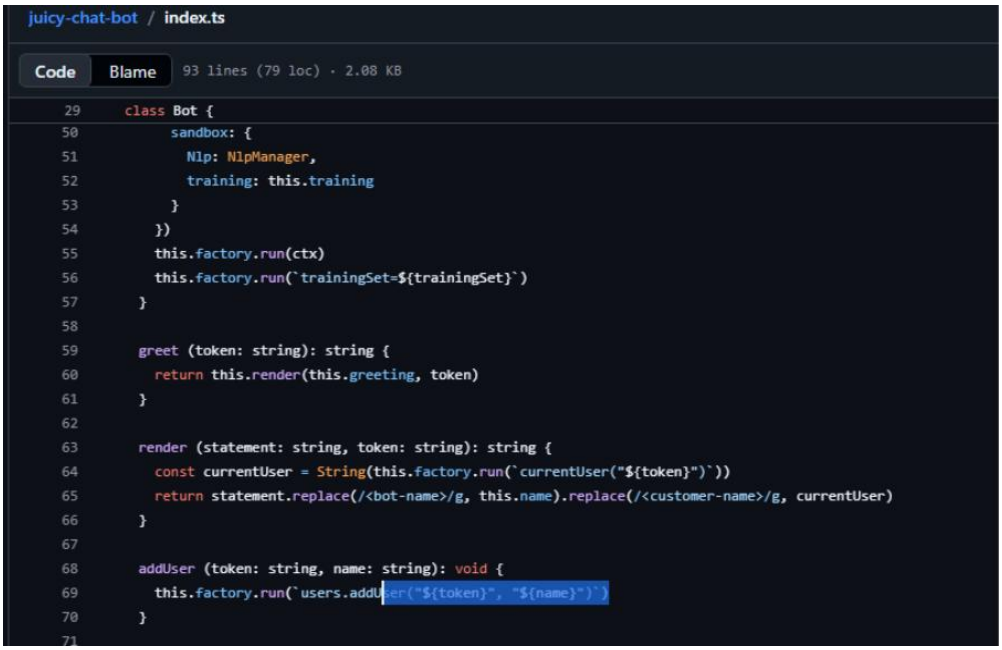
□ □ □ □ □ Kill Chatbot:

Found in source code in Juice-chat-bot (index.js):

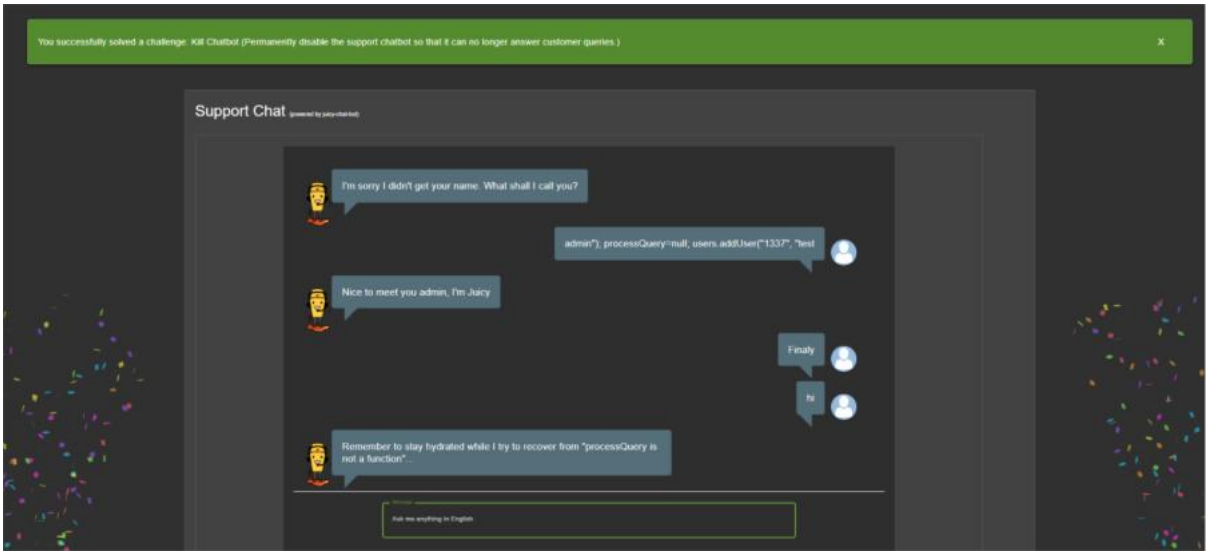


When looking at the source code, I noticed that the name parameter is directly inserted into the string without any processing.

I assumed that it was possible to execute arbitrary code if the input was formed correctly.



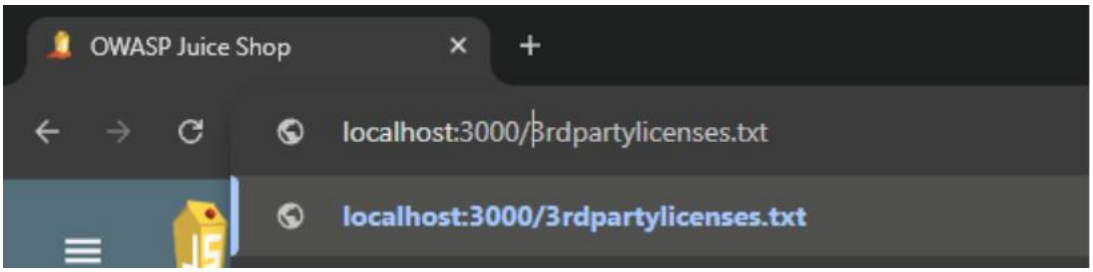
After much brainstorming and many hints, I found the solution: write to the chatbot - admin"); processQuery = null; users.addUser("1337", "test"



Interestingly, I noticed a bug while completing the task. The bot simply stopped working after the first message. This was because after re-logging into the site, my account "died", and I'd get an error message when I tried to log into my profile. I had to create a new account, and everything worked fine.

□ □ □ □ □ Frontend Typosquatting :

I started by analyzing the 3rdpartylicenses.txt file, as it contains a list of all the application's third-party dependencies.



F While browsing the list, I noticed the ngy-cookie package, whose name is very similar to the popular ngx-cookie library.

```
--------------------------------------------------------------------------------
Package: ngy-cookie
License: "MIT"

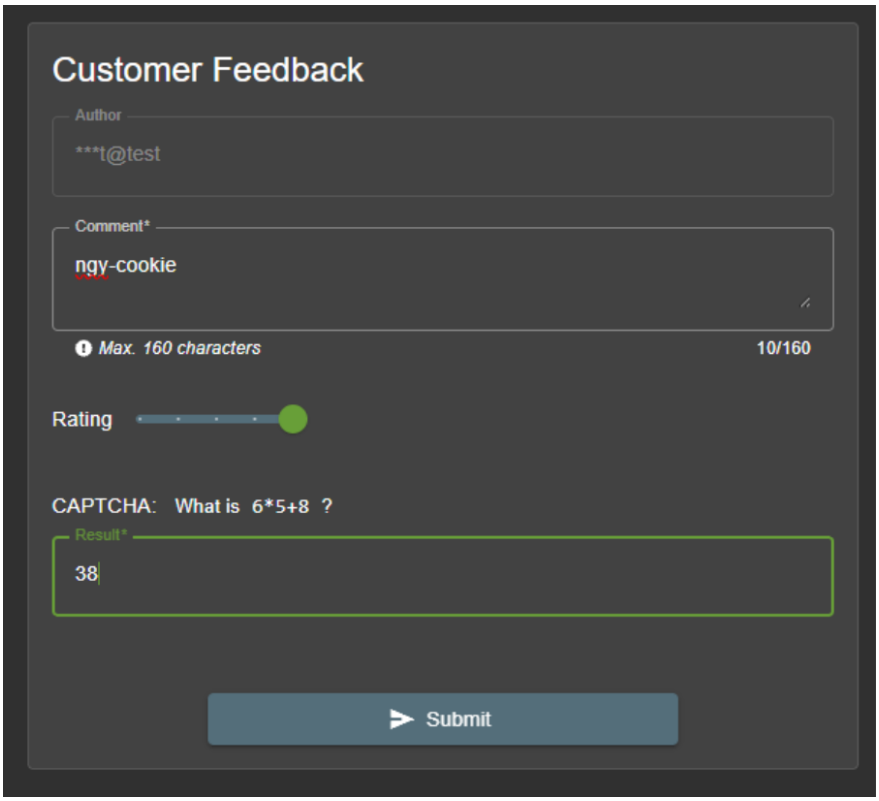
MIT License

Copyright (c) 2017 Samet Alemdar

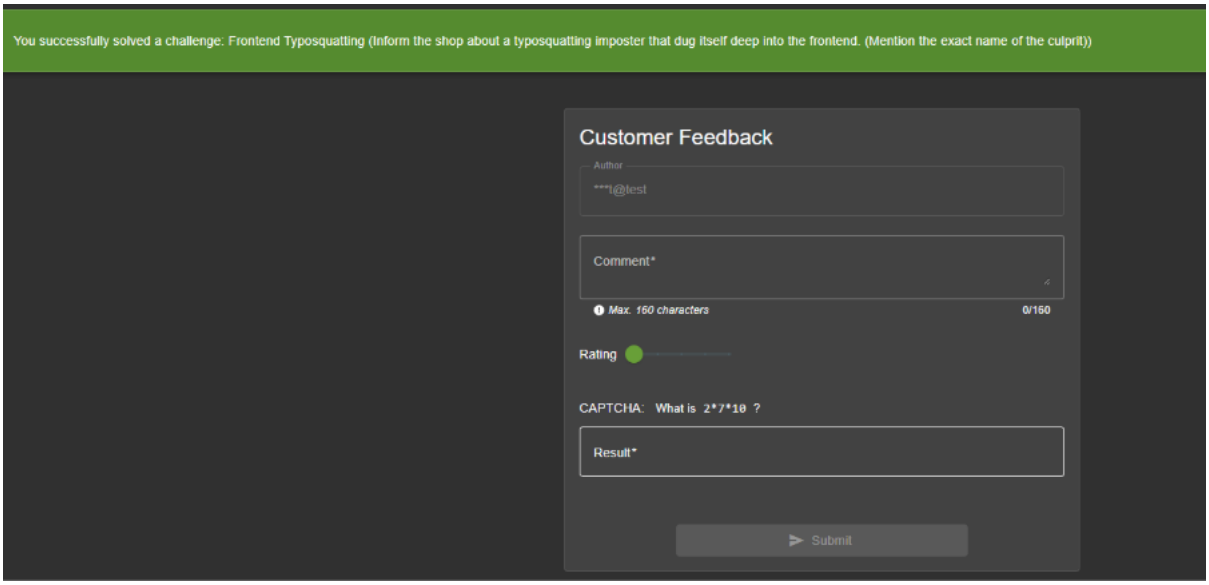
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
```

I decided to test whether the system responds to such things. To do this, I added "ngy-cookie" to the customer feedback comment. A captcha was also required.



After sending the task, the system counted it as completed which confirms that the application does indeed contain a vulnerability.



Vulnerability Analysis:

Broken Authentication:

1. Description and Root Cause:

Broken authentication was #2 on the 2017 OWASP Top 10 list. In 2021 the Broken Authentication category was renamed Identification and Authentication Failures and now includes Common Weakness Enumerations (CWEs) related to identification failures. Broken authentication is typically caused by poorly implemented authentication and session management functions. Broken authentication attacks aim to take over one or more accounts giving the attacker the same privileges as the attacked user. Authentication is “broken” when attackers are able to compromise passwords, keys or session tokens, user account information, and other details to assume user identities. [10]

Once authenticated, the attacker can perform sensitive operations such as changing account settings, including the user’s password. For example, an attacker was able to access the account of a user such as Bender without knowing the correct password.

Use of weak or easily guessable passwords, making accounts vulnerable to simple attacks. Absence of Multi-Factor Authentication (MFA), reducing the security barrier for unauthorized access. Exposed or predictable login endpoints, which attackers can easily discover and target. Poor session token management, such as not setting expiration times or allowing token reuse. Publicly accessible authentication endpoints without proper access controls. Lack of protections against brute-force and credential stuffing attacks, enabling automated login attempts. [9][10]

Rationale or investigative process

The vulnerability was identified by testing the login functionality with manipulated inputs. By inserting special characters such as quotation marks and SQL comment operators into the email

field, it became clear that the application was not properly sanitizing input. This indicated the possibility of SQL Injection. After successfully accessing another user's account, the password change functionality was examined. Using an interception proxy tool, the HTTP request sent to the server was captured and modified.

2. How the exploit works

An attacker exploits weaknesses in the authentication mechanism, such as predictable or weak passwords, lack of multi-factor authentication, or poor session management. For example, if a system allows unlimited login attempts, the attacker can perform a brute-force attack to guess a user's password. Once the correct credentials are obtained, the attacker can log in as the user and perform sensitive actions, such as changing account settings, updating the email address, or resetting security information. [11]

3. Potential impact

The impact of broken authentication attacks can be devastating for both an organization and its customers. When attackers exploit these vulnerabilities, they gain unauthorized access to user accounts, personal data, sensitive business information, and more. This not only leads to a breach of privacy and potential financial losses but can also severely tarnish the reputation of the impacted organization.

For an end-user, a broken authentication attack could mean unauthorized access to their account, leading to the theft of sensitive personal data such as credit card information, social security numbers, and more. This could further result in identity theft, unauthorized transactions, and other forms of personal harm.

For businesses, the consequences can be even more severe. A successful attack could potentially give cybercriminals access to privileged accounts, allowing them to manipulate data, perform malicious actions, or even take control of the entire system. This could lead to substantial financial losses, damage to the organization's reputation, loss of customer trust, and potential legal implications. [11]

4. Recommended mitigation

Don't reinvent the wheel in authentication, token generation, or password storage. Use the standards. Implement anti-brute force mechanisms to mitigate credential stuffing, dictionary attacks, and brute force attacks on your authentication endpoints. Where possible, implement multi-factor authentication. Require re-authentication for sensitive operations (e.g. changing the account owner email address/2FA phone number). Credential recovery/forgot password endpoints should be treated as login endpoints in terms of brute force, rate limiting, and lockout protections. [9][10][11]

Broken Access Control:

1. Description and Root Cause:

Access control, sometimes called authorization, is how a web application grants access to content and functions to some users and not others. These checks are performed after authentication, and govern what ‘authorized’ users are allowed to do. Access control sounds like a simple problem but is insidiously difficult to implement correctly. A web application’s access control model is closely tied to the content and functions that the site provides. In addition, the users may fall into a number of groups or roles with different abilities or privileges. [12]

2. Rationale or investigative process:

The vulnerability was identified by analyzing how the application handles access to restricted resources.

Testing involved modifying parameters such as user IDs in URLs and API requests to determine whether unauthorized data could be accessed. For example, by changing the user ID in a request, it was possible to retrieve data belonging to another user. [13][14]

3. How the exploit works:

For example, administrative functions might be linked from an administrator's welcome page but not from a user's welcome page. However, a user might be able to access the administrative functions by browsing to the relevant admin URL.

For example, a website might host sensitive functionality at the following URL:

https://insecure-website.com/admin

This might be accessible by any user, not only administrative users who have a link to the functionality in their user interface. In some cases, the administrative URL might be disclosed in other locations, such as the robots.txt file:

https://insecure-website.com/robots.txt

Even if the URL isn't disclosed anywhere, an attacker may be able to use a wordlist to brute-force the location of the sensitive functionality. [13]

4. Potential impact:

The impact of broken access control can be catastrophic for organizations. Unauthorized access to sensitive data can lead to data breaches, identity theft, financial loss, and damage to a company’s reputation. In worst-case scenarios, it can even result in total system compromise where attackers gain complete control over the system.

The risk associated with broken access control is high because it directly affects the confidentiality, integrity, and availability of data. An attacker exploiting this vulnerability can potentially access, modify, or delete any data on the system. This includes user data, system data, application data, and more. The larger the system and the more sensitive the data, the higher the risk. [12][14]

5. Recommended mitigation:

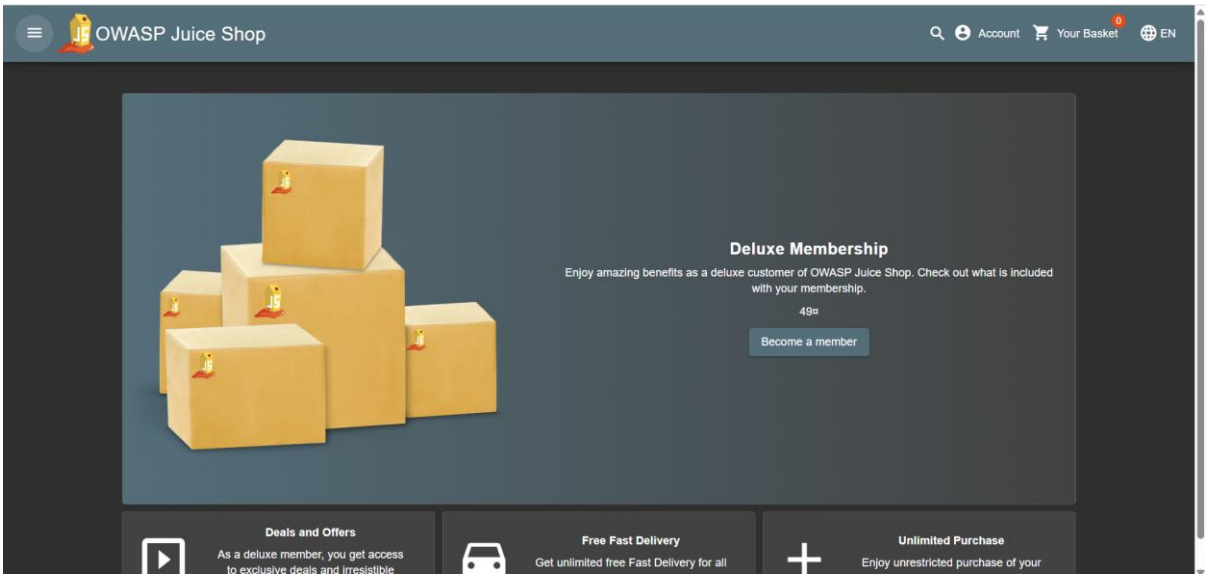
Secure session management and authentication controls are critical aspect of preventing broken access control. These measures ensure that only authorized users can access your system and that they can only access the parts of the system that they need to.
Secure session management involves creating a unique session for each user when they log in and destroying that session when they log out or after a period of inactivity. This prevents unauthorized users from hijacking a user's session and gaining access to their account.
Authentication controls verify that users are who they claim to be. This is usually done through a combination of usernames and passwords, but can also involve other methods such as biometrics or security tokens. [12][13][14]

Stepan Tretiakov B00177464

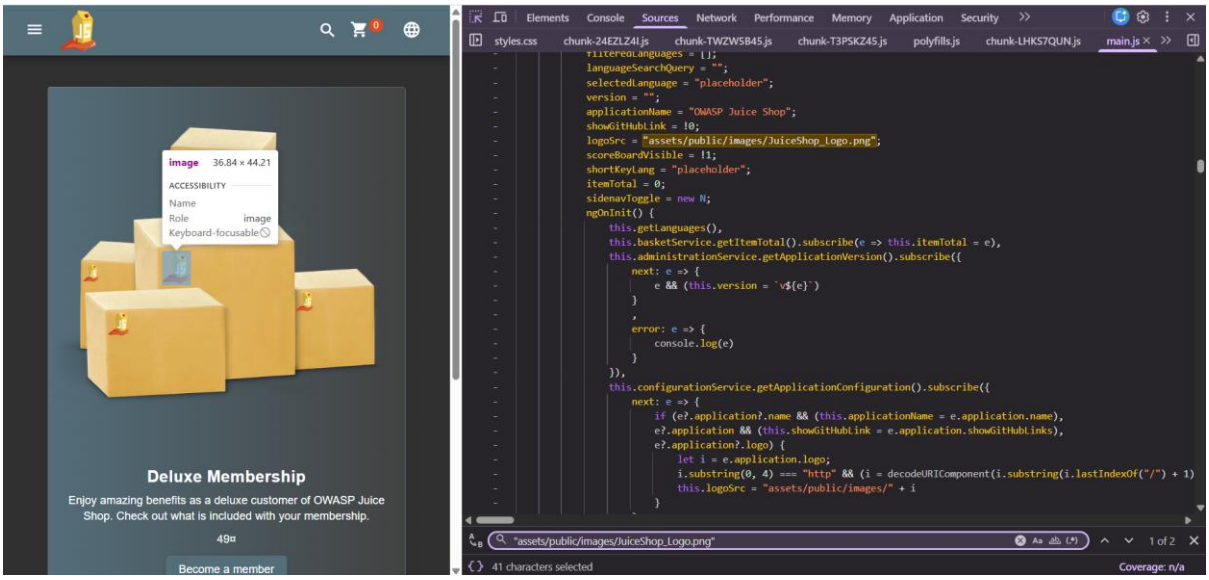
Solved Challenges

□ □ □ □ □ Cross-Site Imaging:

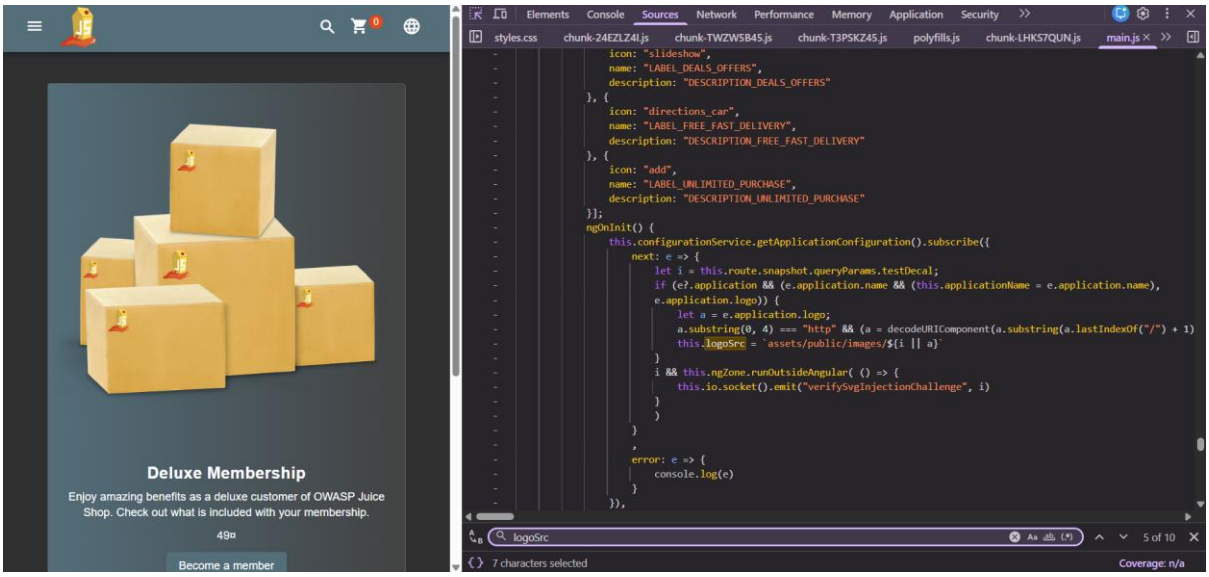
The challange is reffering to Deluxe Membership tab and logo images displayed on them. To start, let's analyze the images and source code of the page.



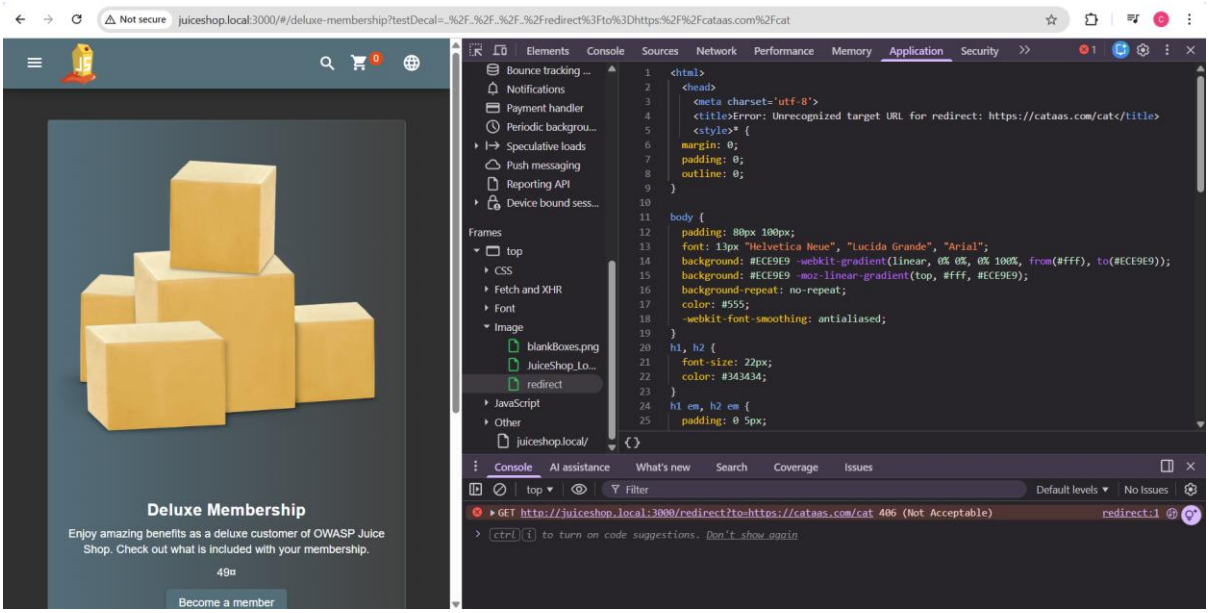
By inspecting the image, I am able to find that the path towards it is href="assets/public/images/JuiceShop_Logo.png", which then allows me to find the name of the attribute of logoSrc which is used to load the image of the logo in main.js of the page in Source tab of Dev tools.



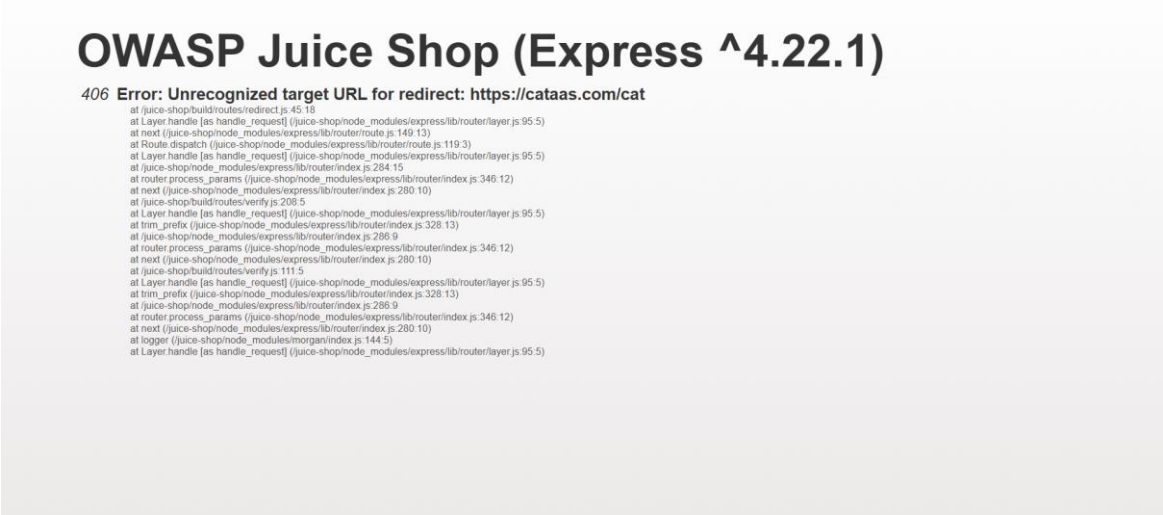
By analyzing the code, we can spot the the logoSrc can have default or secondary path, but only if testDecal parameter is provided in URL.



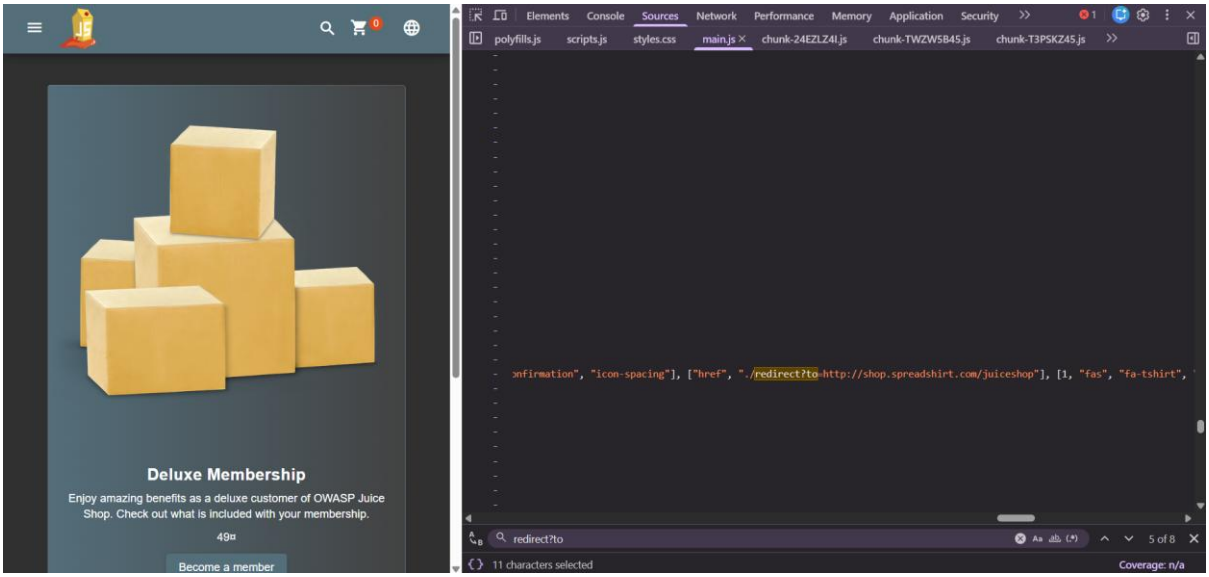
This means that by creating URL query such as “http://juiceshop.local:3000/#/deluxe-membership?testDecal=...” it’s possible to use custom path of the image, instead of the default one by redirecting the path to an external source – Cat as a Service website. Since, in the JavaScript code above, logoSrc custom path is this.logoSrc = `assets/public/images/${i || a}`, which mean it starts with “assets/public/images”, We first need to do some directory traversing to get to the root directory with “../../../../”.



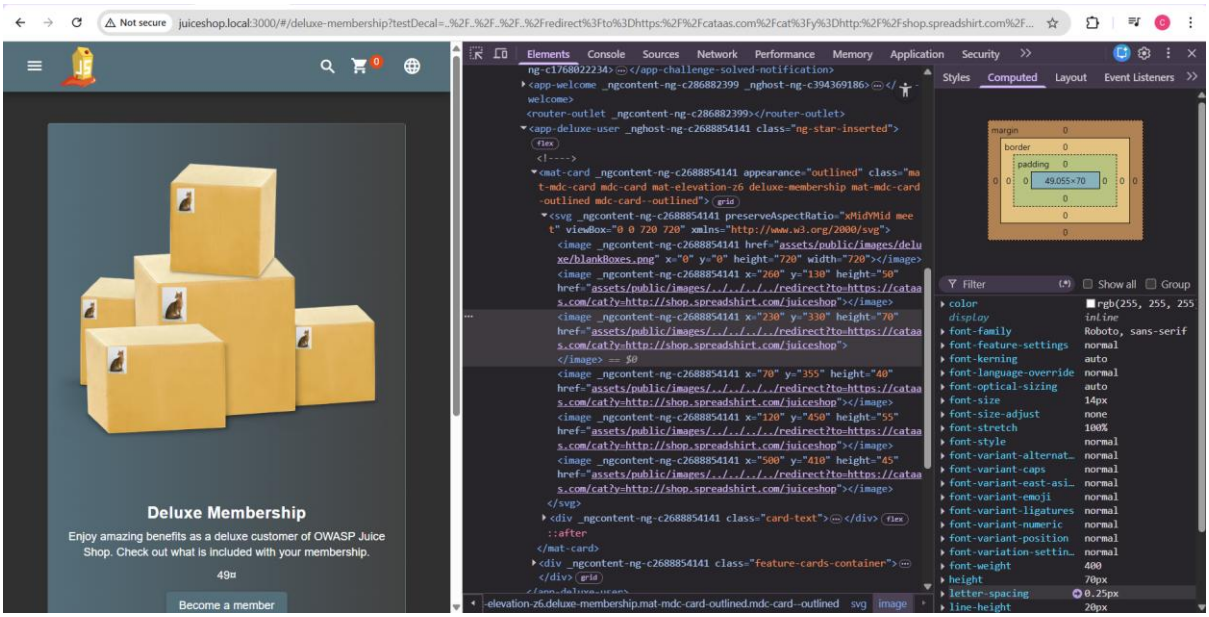
URL such as “http://juiceshop.local:3000/#/deluxe-membership?testDecal=../../../../redirect?to=https://cataas.com/cat” results in an allowlist 406 error, but as seen from the Inspect function, it correctly changes the path of an image to “href="assets/public/images/../../../../redirect?to=https://cataas.com/cat"”.



Next step would be bypassing the allowlist which can be done by including one of the whitelisted URLs into the crafted URL request as noise parameter. The whitelisted URLs can be found in main.js file by searshing for “redirect?to=”. One of the options would be “http://shop.spreadshirt.com/juiceshop”

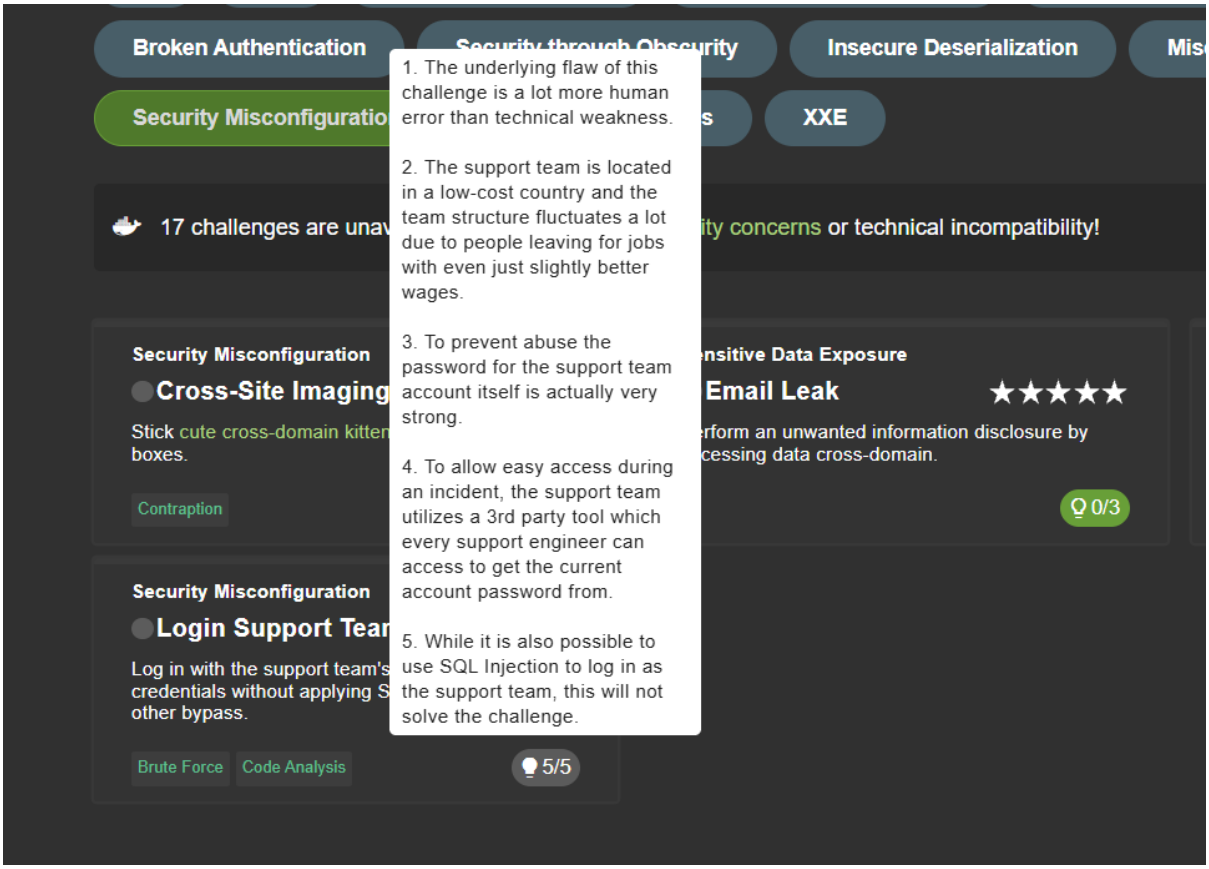


Now, by combining all of the steps above, the final crafted URL would look like this: “http://juiceshop.local:3000/#/deluxe-membership?testDecal=../../../../redirect?to=https://cataas.com/cat?y=http://shop.spreadshirt.com/juiceshop”. This URL correctly replaces the Juice Shop logo on boxes with cats.



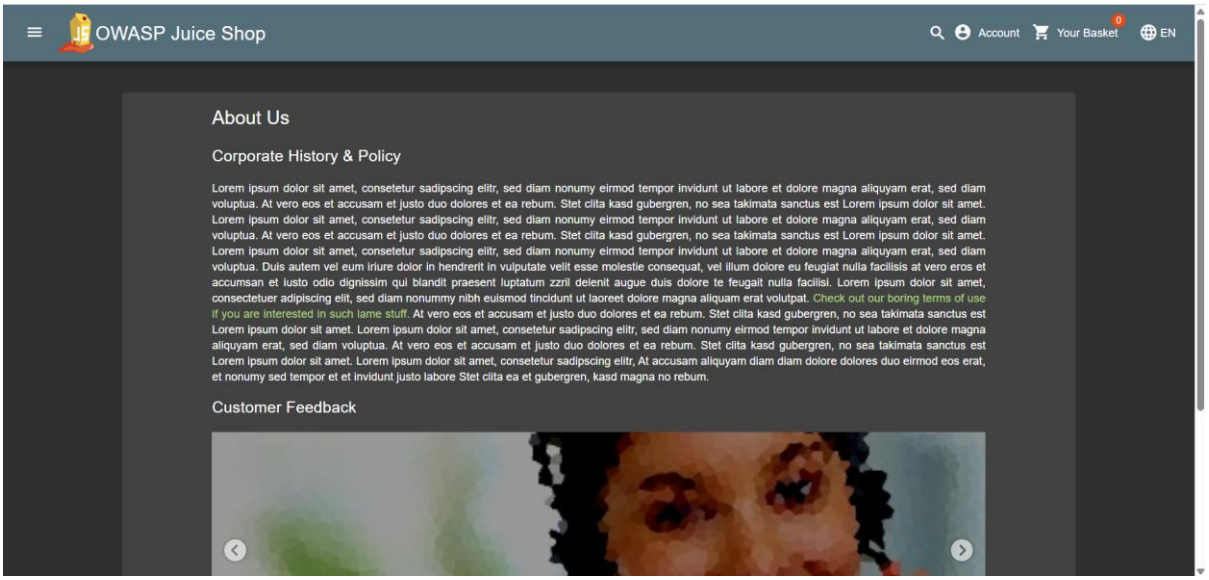
□ □ □ □ □ □ Login Support Team:

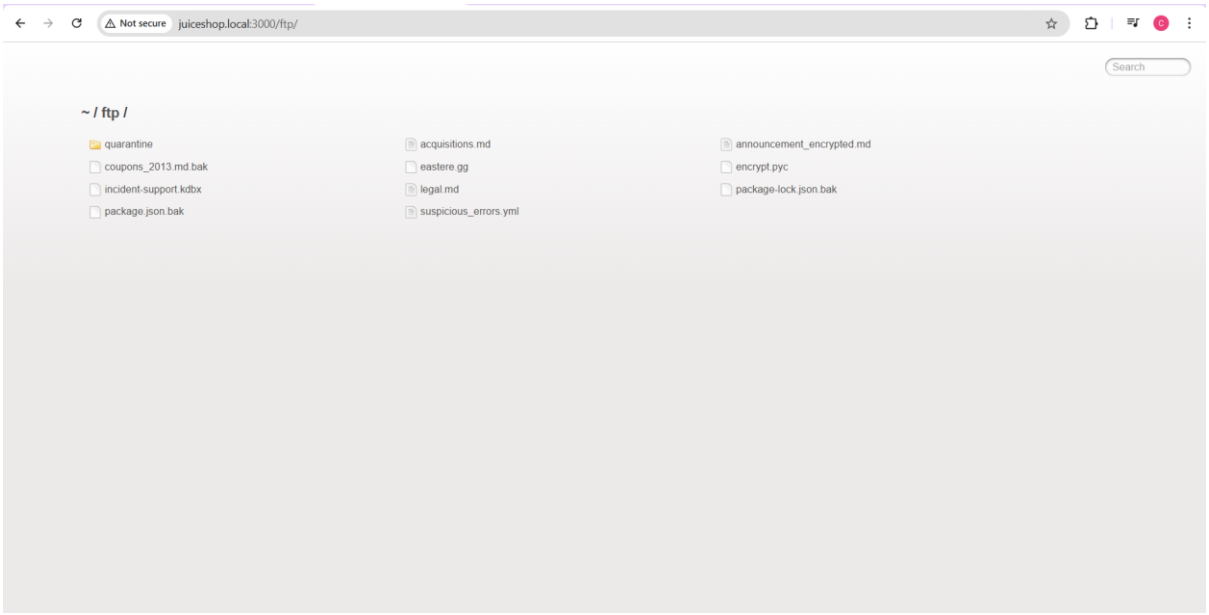
By unlocking a few hints for a challange, important information about the nature of support team account is discovered.



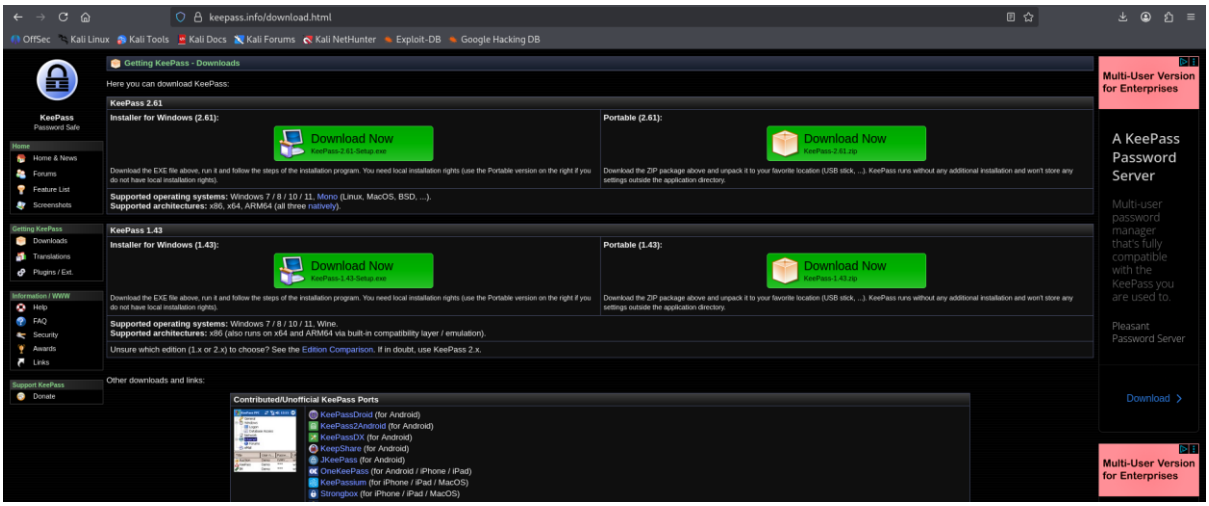
The support team is using a 3rd party password manager to store credentials for the account so that many different people could access the account.

Earlier, by going through the pages onn the Juice Shop website, I discovered that the legal notice for the website leaks the path towards FTP page with website's files are stored.

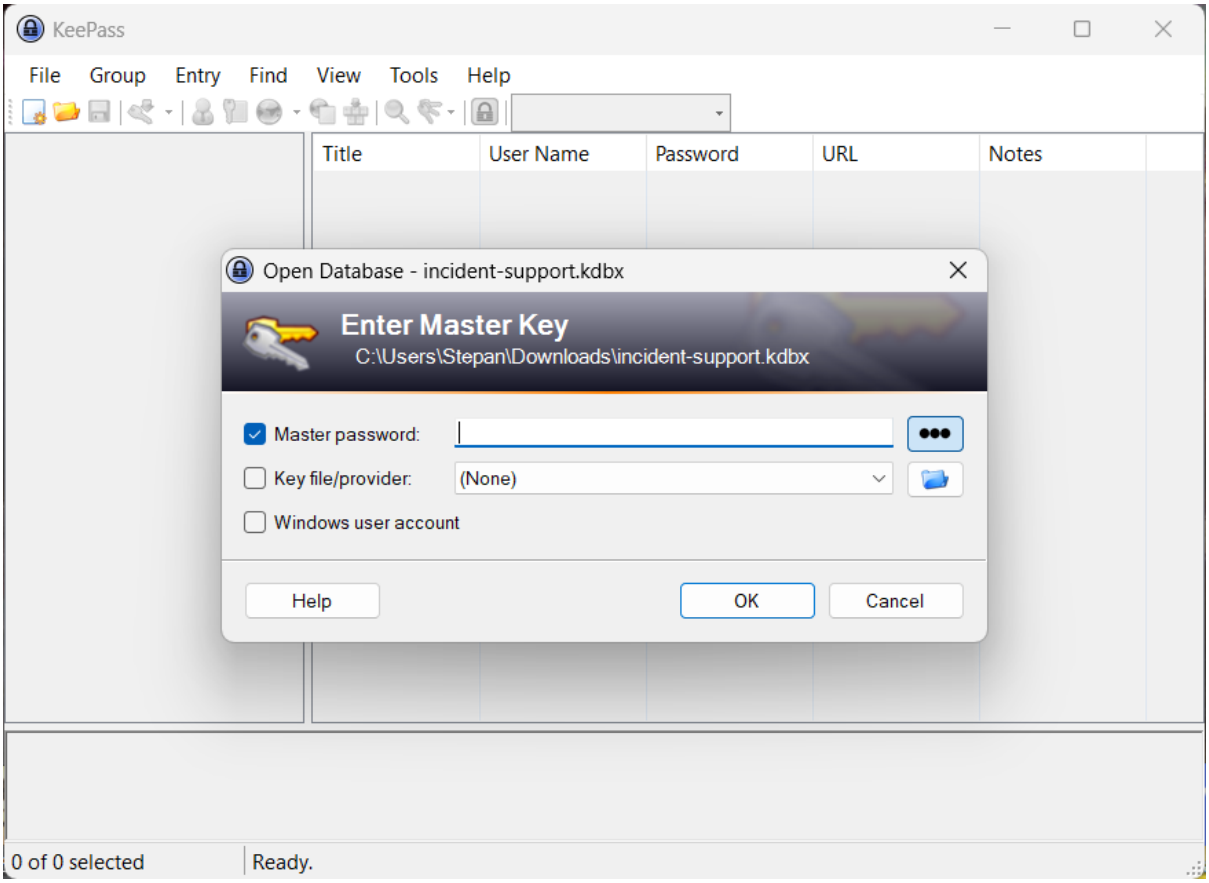




On the /ftp page there is file with .kdbx extension, which after some googling turns out to be a file format used by KeePass, a free open source password manager. Install the password manager to try to open the file.



However, the file is locked with a master password. This however is not a big issue since now that we have the file on the system we can use affline passwod cracking tool like John the Ripper to try to find the password.



To start password cracking, however, we first need to convert .kdbx file into a format John the Ripper will accept. This can be achieved with one of the tools which will convert KeePass database into a format of John the Ripper. After that we can start cracking process.

```
┌──(kali㉿kali)-[~/keepass2john]
└─$ python keepass2john.py ../incident-support.kdbx > ../output_john.txt

┌──(kali㉿kali)-[~/keepass2john]
└─$ cd ..

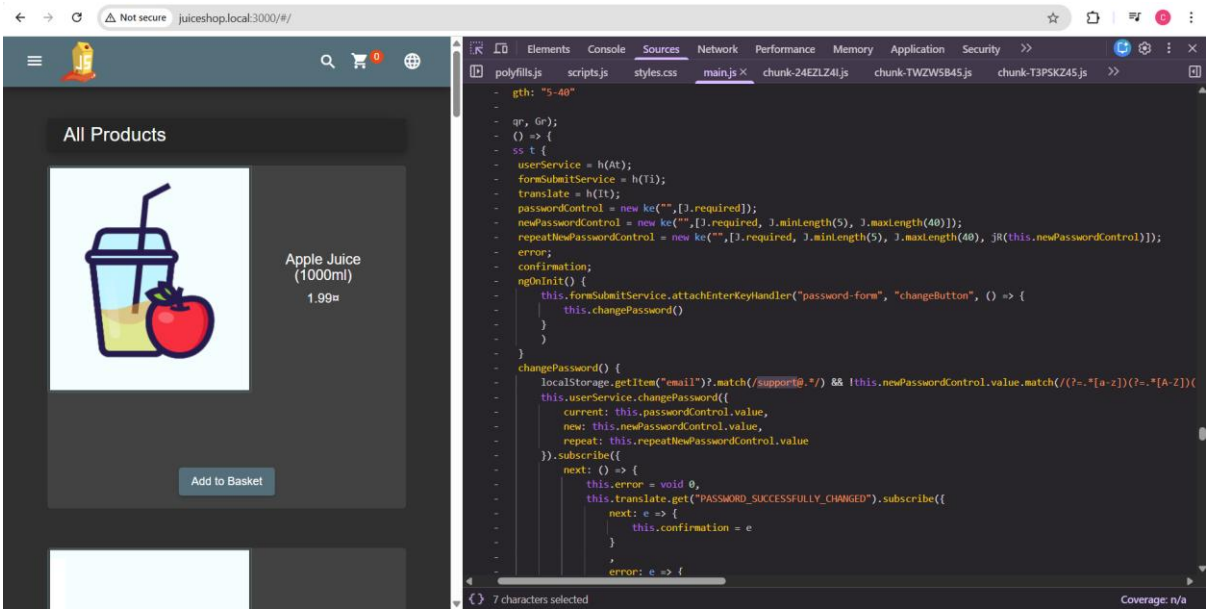
┌──(kali㉿kali)-[~]
└─$ ls
chall_key.pem  Desktop  Documents  Downloads  incident-support.kdbx  keepass2john  Music  output_john.txt  Pictures  Public  Templates  Videos
```

After trying the rockyou.txt and it not resulting in any matches. It's better to not guess blindly. But to try to find some hint on to what the password might look like.

```
┌──(kali㉿kali)-[~]
└─$ sudo john --format=keepass --wordlist=/usr/share/wordlists/rockyou.txt output_john.txt
Using default input encoding: UTF-8
Loaded 1 password hash (KeePass [SHA256 AES 32/64])
Cost 1 (iteration count) is 1 for all loaded hashes
Cost 2 (version) is 2 for all loaded hashes
Cost 3 (algorithm [0=AES 1=TwoFish 2=ChaCha]) is 0 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:07 DONE (2026-04-17 15:54) 0g/s 1870Kp/s 1870Kc/s 1870KC/s           ..*7¡Vamos!
Session completed.

┌──(kali㉿kali)-[~]
└─$ john --show output_john.txt
0 password hashes cracked, 1 left
```

After searching for "keyterm" support in main.js file of a website, something notable came up. "changePassword" policy which limits how the password will look like. According to JavaScript code the password will look like "/(?=.[a-z])(?=.[A-Z])(?=.\d)(?=.[@$!%?&])[A-Za-z\d@$!%?&]{12,30}/". This means that a password must contain 12 to 30 characters, 1 lowercase,1 uppercase,1 digit, and 1 special character.



This help a lot since now, after reading John the Ripper documentation, extra characters can be appended to words from a wordlisst. It can be done in to ways: either by adding –rules or –mask. For each case the command will look like this:

Rules:

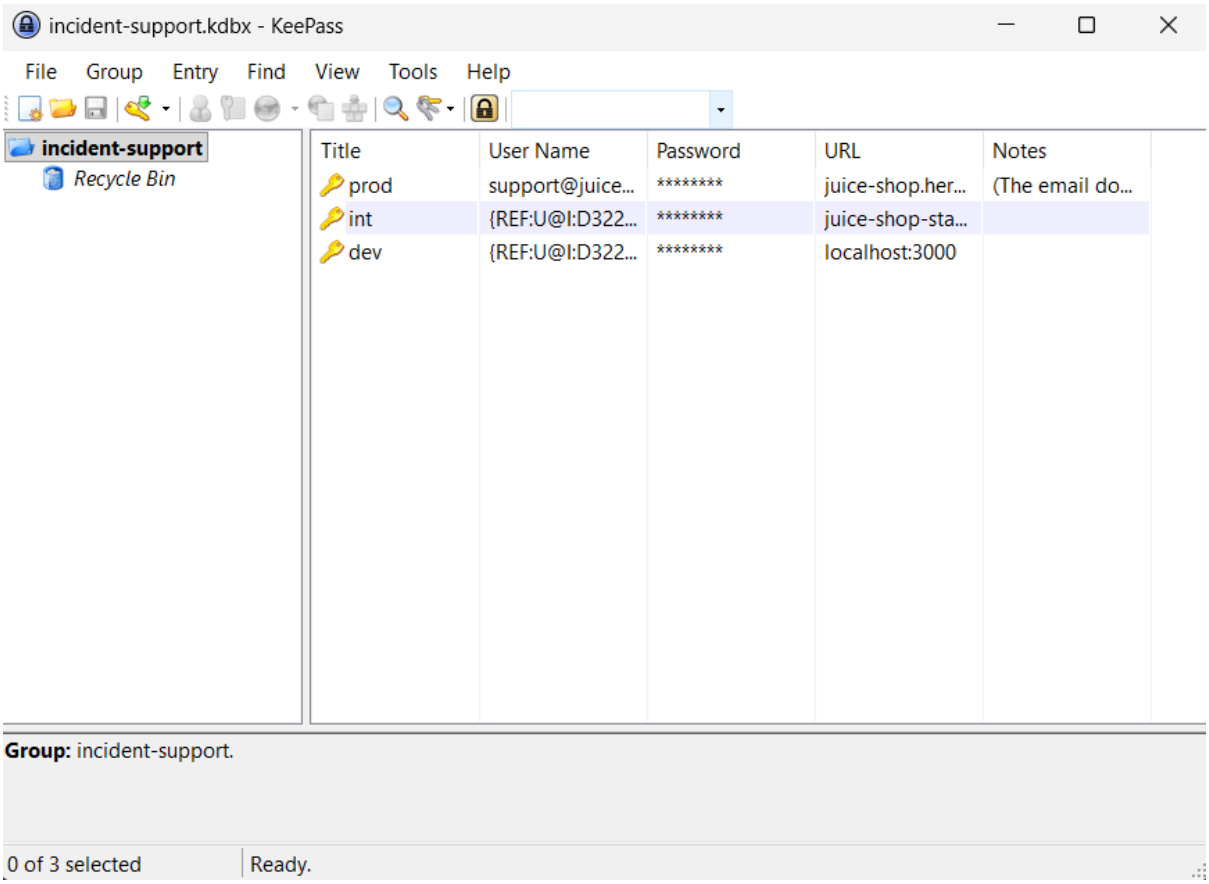
john --format=keepass --wordlist=/usr/share/wordlists/rockyou.txt --rules=All output_john.txt

Mask:

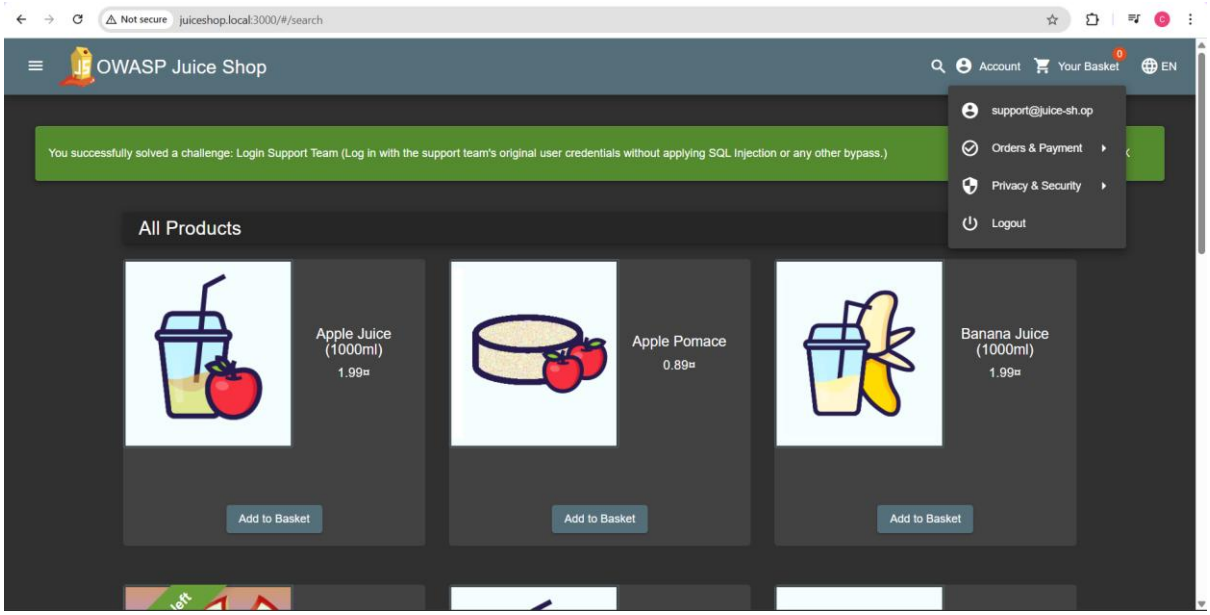
john --format=keepass --wordlist=/usr/share/wordlists/rockyou.txt --mask='?w?d?d?d?d!' output_john.txt

Rules attack combines existing John the Ripper rules which add/tranfrom the wordlist entry, but due to sheer number of combinations, it will take a very long time. After the John the Ripper is finished with cracking process, the password is “Support2022!”.

After inserting it into the KeePass database, the content inside can be accessed.

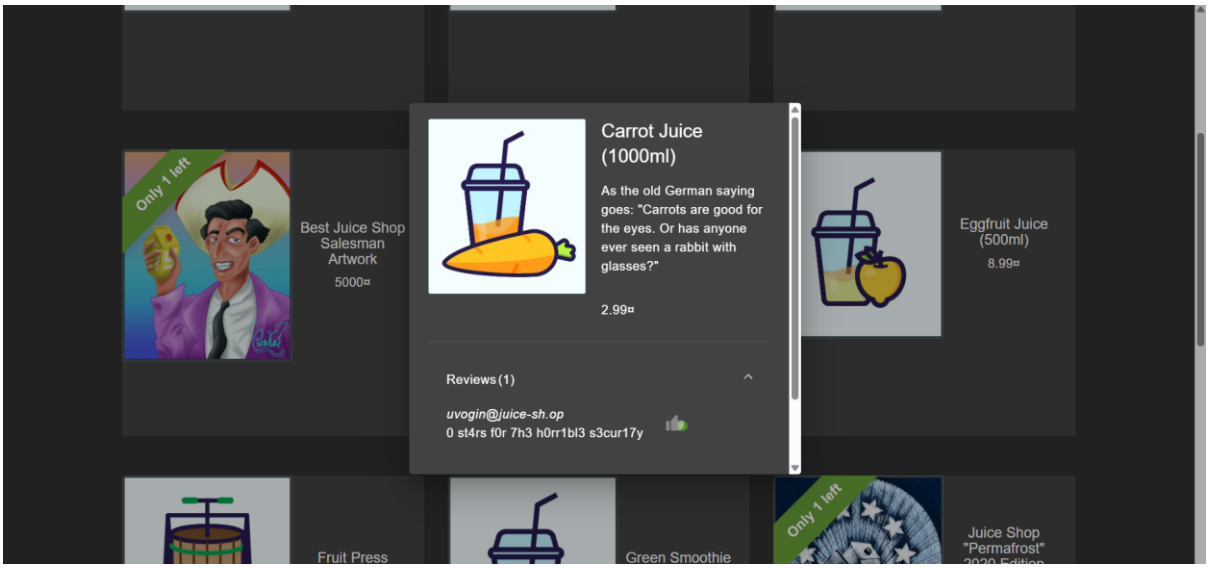


Entry whose owner is support@juice-sh.op is what we are looking for. After copying the password and inserting the credentials into the Juice Shop login page the challange is done.

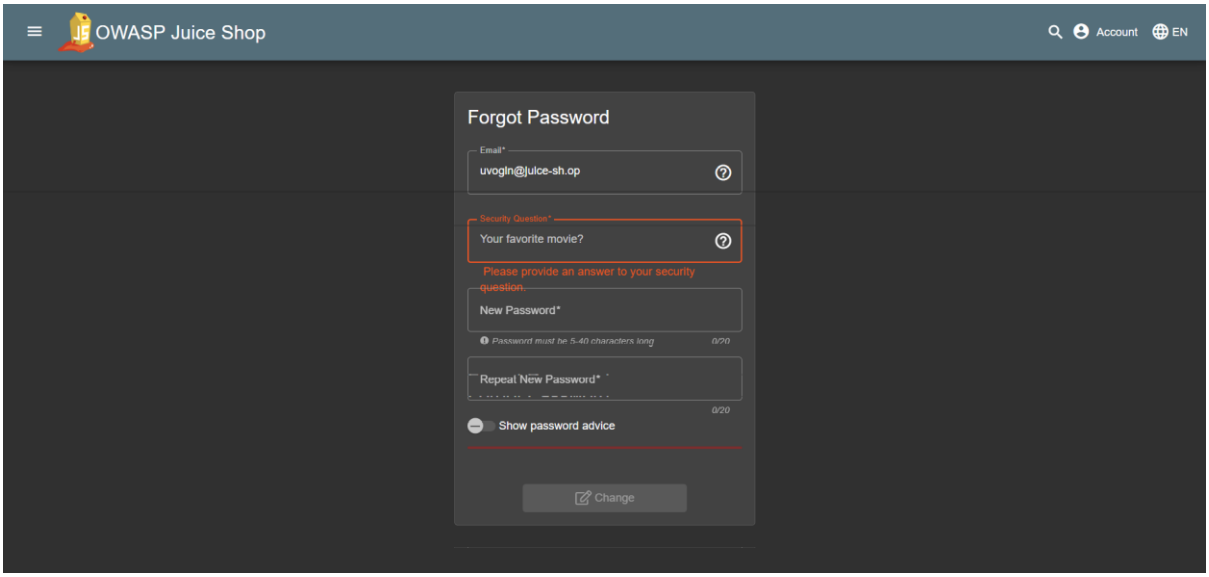


□ □ □ □ Reset Uvogin's Password:

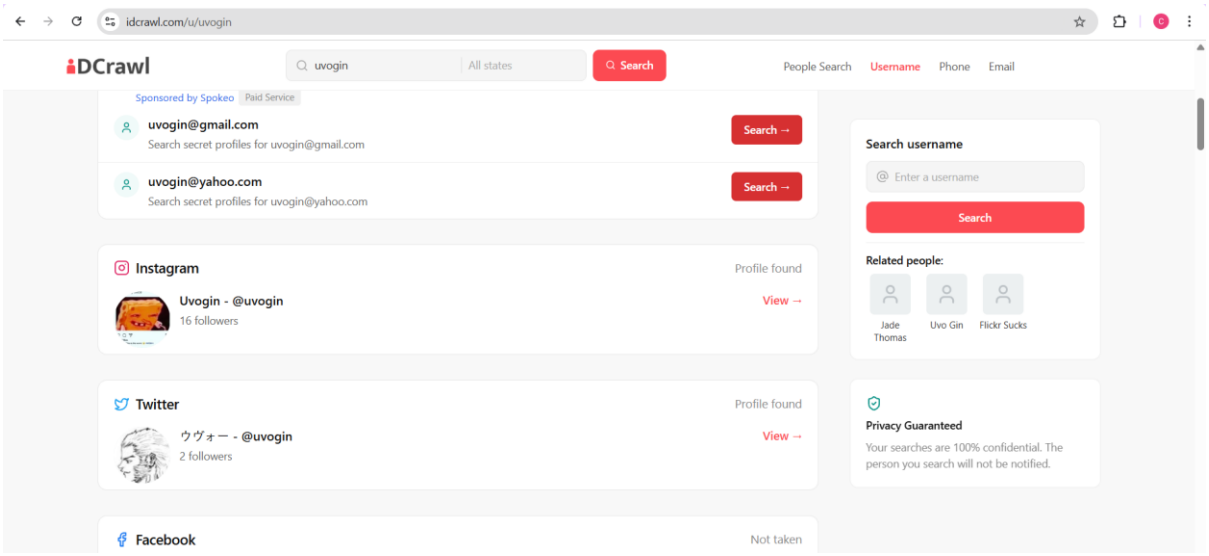
To begin, we need to find an email of Uvogin, under which they have registered in Juice Shop. This can be done easily by looking through the reviews under some of the products on the website, since on the website, emails of users are displayed in reviews instead of the usernames, which helps to quickly find Uvogin's email address.



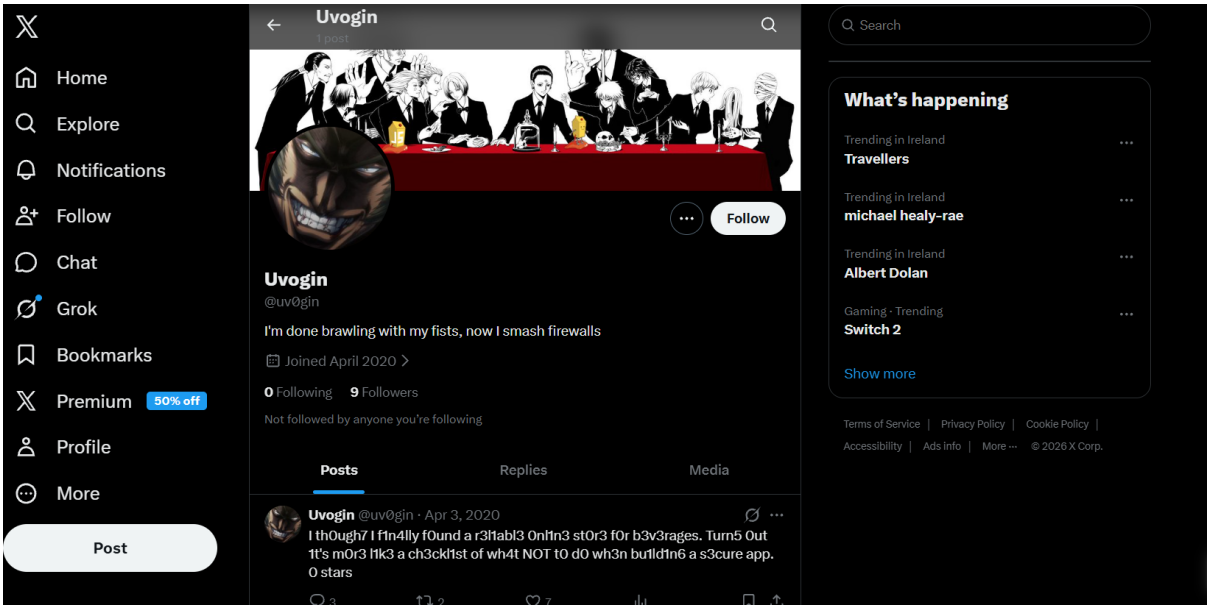
With this, all that is left is to find the answer to the security question.



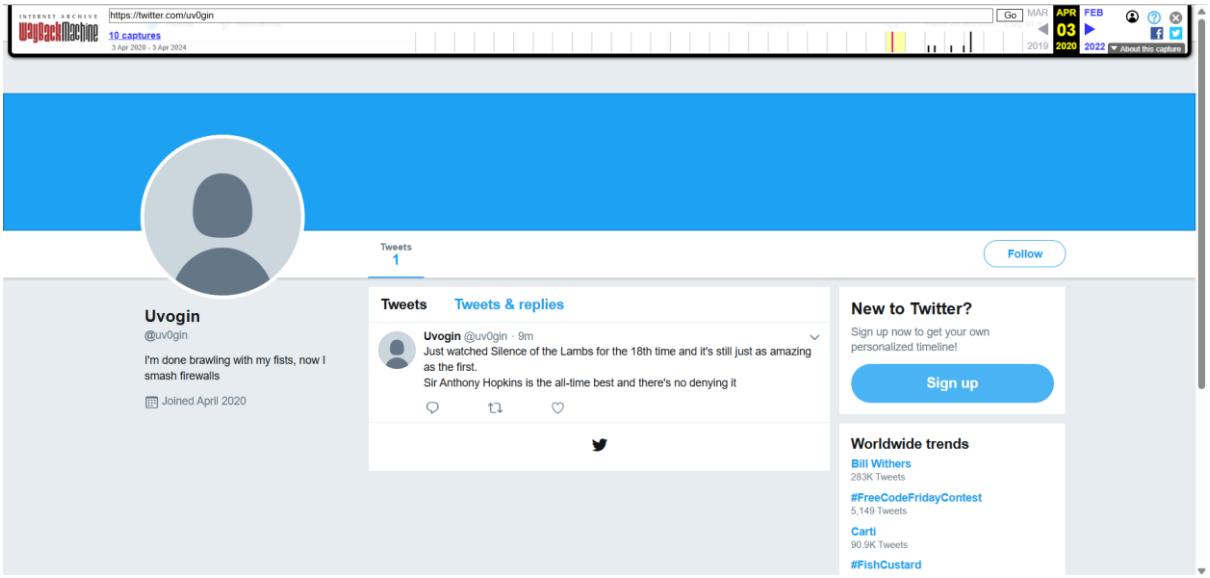
By searching directly for the username of “Uvogin” in of the OSINT resources, nothing particular comes up. And for a good reason, Uvogin – name of a character from a very popular anime series, which explains why account under this username is present almost on every social medai platform with seeming no indication that one person own same account among platforms.



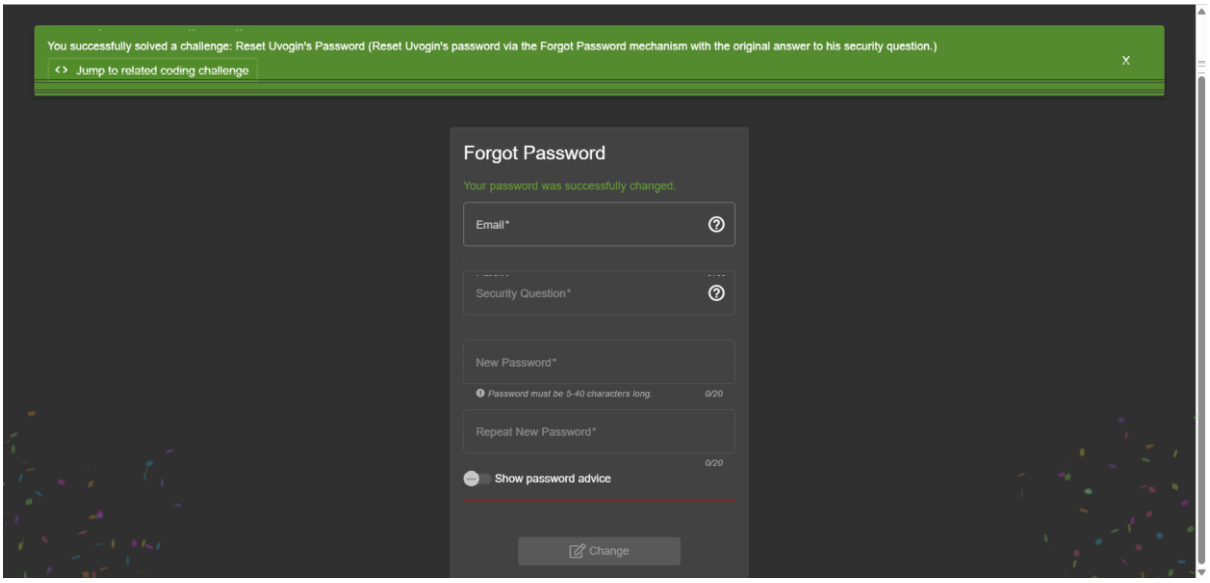
However, by coming back to reviews left under the products on the Juice Shop, it is possible to notice that Uvogin, we are looking for has a distinctive way of typing where they replace certain letters with similar looking numbers, such as: i – 1, t – 7, 3 – e, etc. By searching for variations of name “Uvogin” with numbers in place of certain letters, I found a particularly interesting account on Twitter, where both typing style and the content of the post matched perfectly the user I am trying to look for.



But there is only one post on this account which does not give any information about the security question. I decided to try my luck with the Wayback Machine wo see whether there are any snapshots of this page of an earlier date, since the user might have posted something and later deleted it.

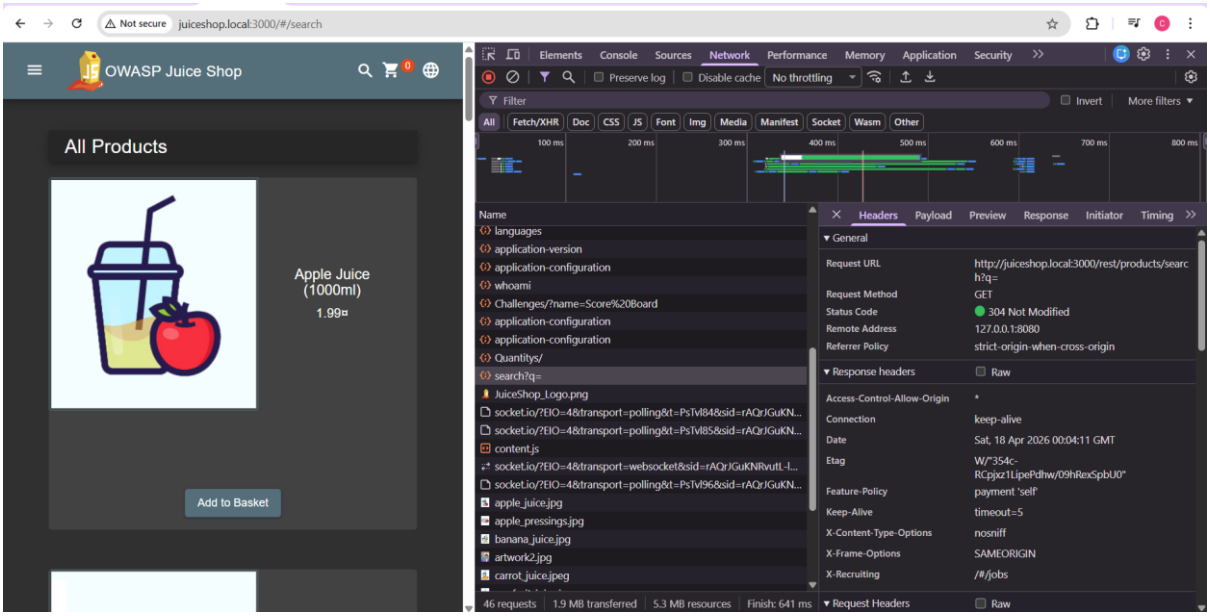


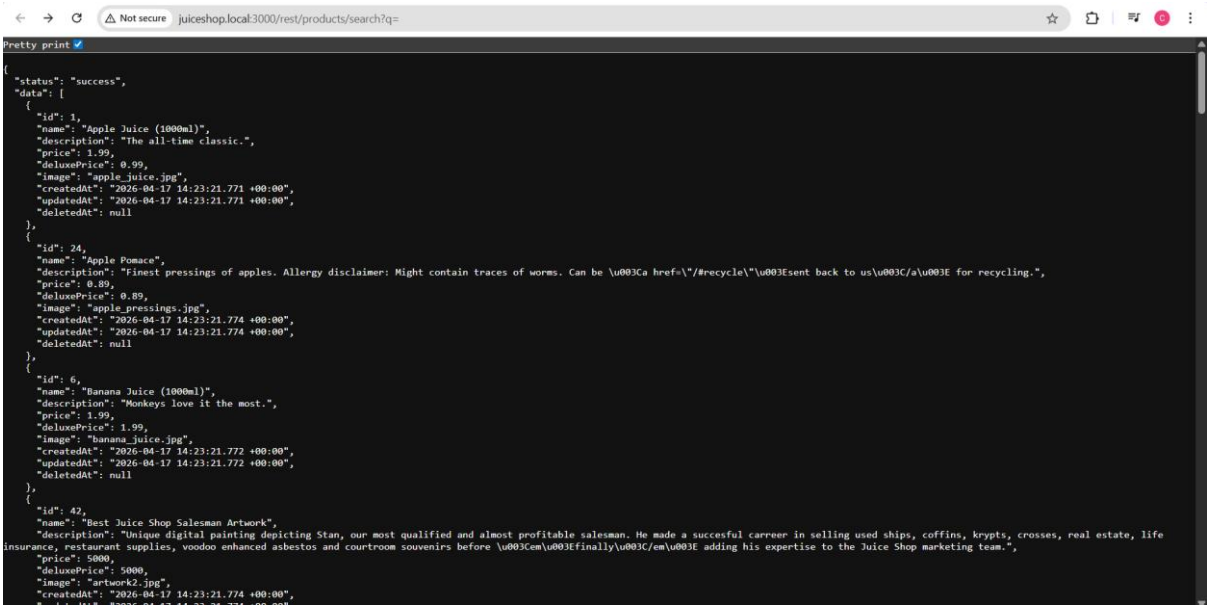
In the end, there was indeed a post about a film recently watched by Uvogin. After pasting Silence of the Lambs into the security question field, we've successfully completed the challange.



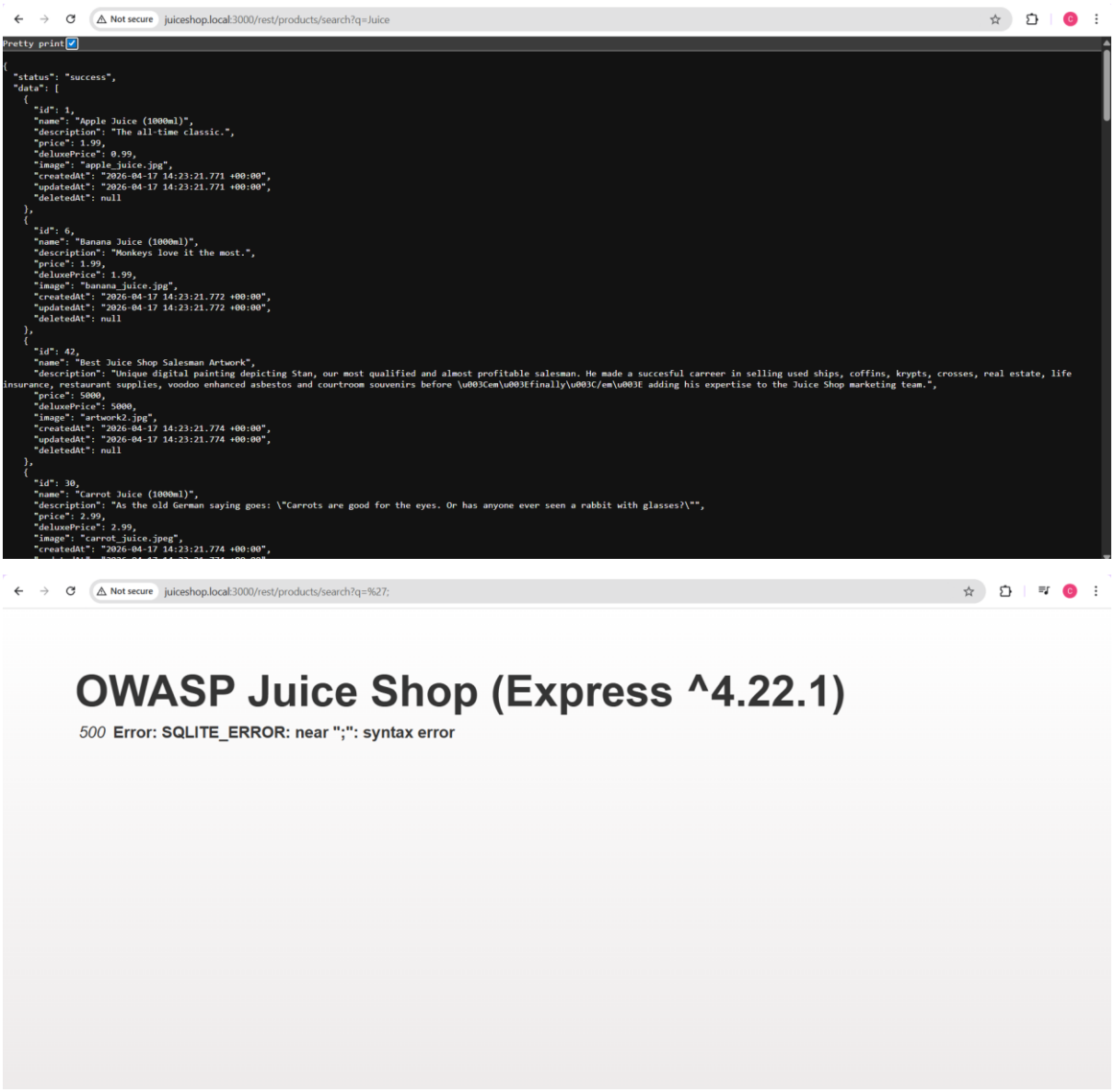
□ □ □ □ Leaked Unsafe Product:

We can see in Network tab of Developer tools that when web page is loading, many GET requests are done, among which one titled “search?q=” is what is used to load products to the front page.

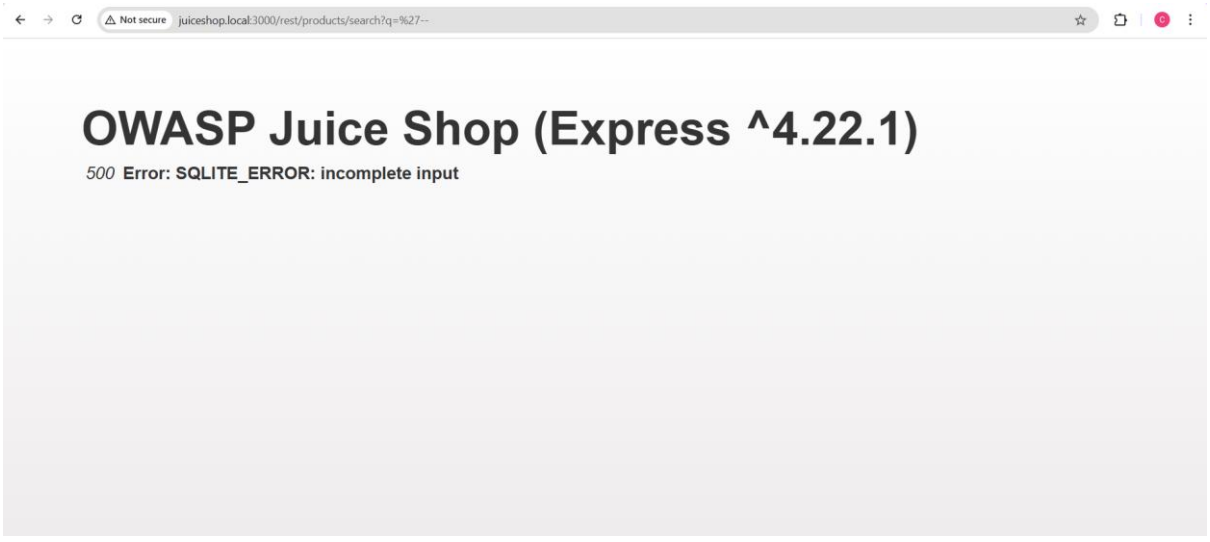


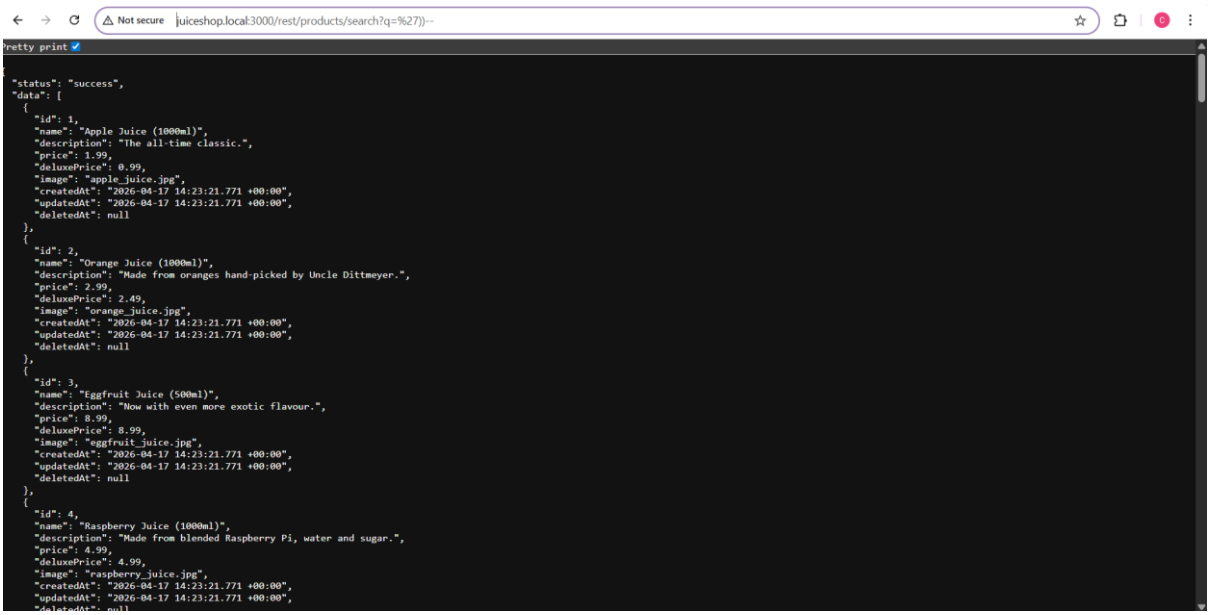


By opening the request in a new tab, we will try to see whether an SQL injection is possible and whether it will be possible to view all products from database. To begin with, I added a genuine search query to test if it works. Afterwords, I decided to test whether an SQL nijection is possible by passing ‘; in the end of a request which resulted in an SQL error and confirmed that there was a response from the server.

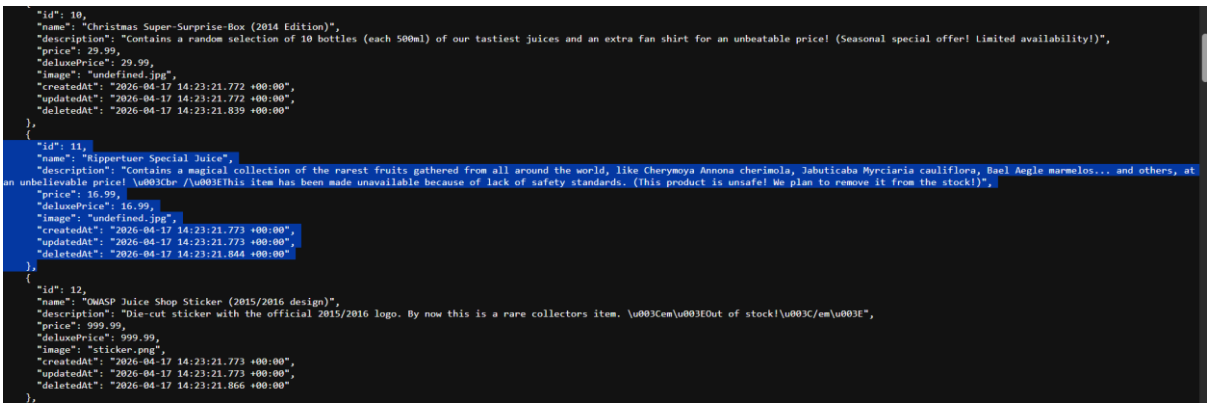


Now, it's a matter of trying to make a payload to access the list of all products. I started with passing ?q='-- which resulted in 500 Error: SQLITE_ERROR: incomplete input. This means that the syntax of query in incomplete. After finding more about this type error, the way to fix it is to add two paranthesis into the payload which fixes it, so it will look like this: "http://juiceshop.local:3000/rest/products/search?q='))--". Now we have the list of all products at Juice Shop.

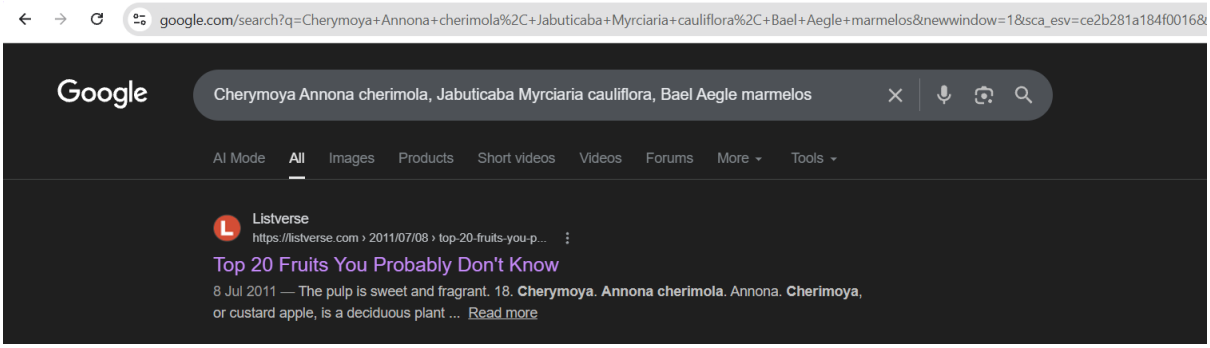


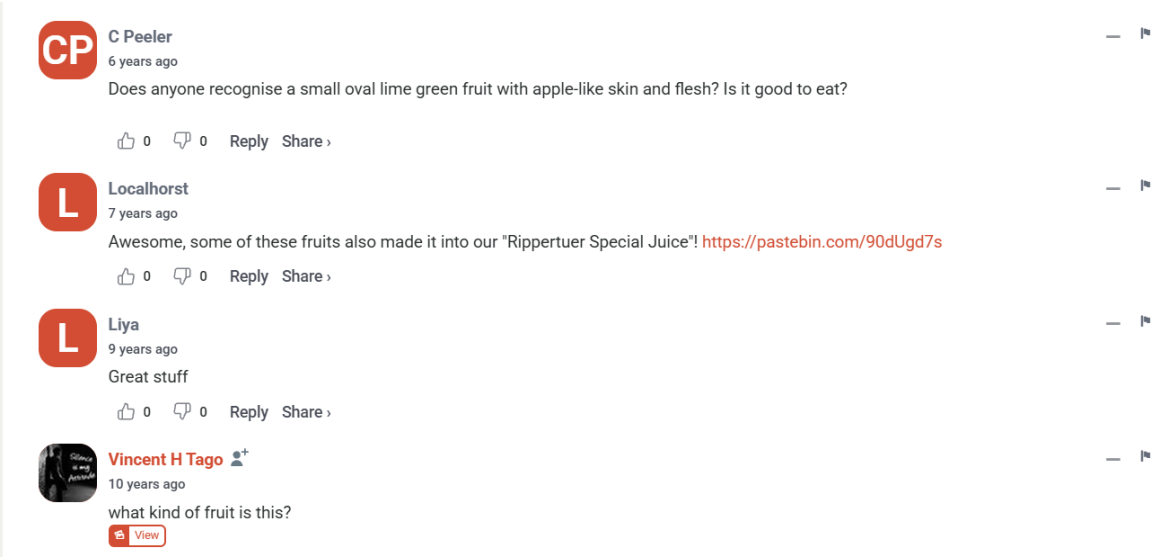


By looking through the list of products, one of them with product id=11 has a note in its description reading: “This item has been made unavailable because of lack of safety standards. (This product is unsafe! We plan to remove it from the stock!)”

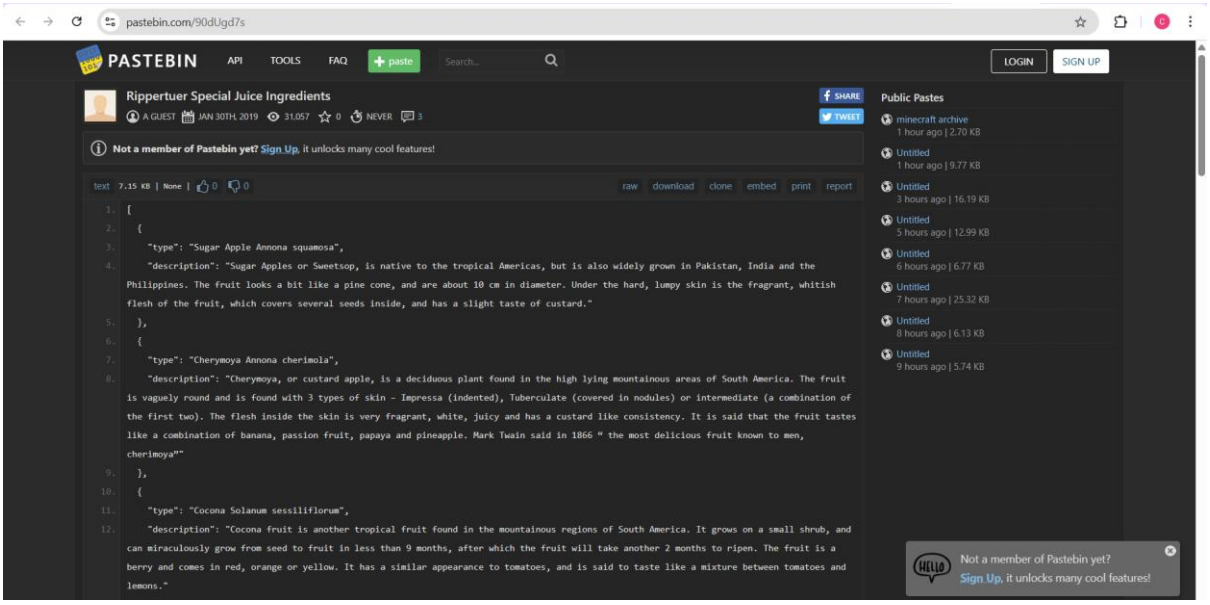


To solve the challang ewe need to submit all of the dangerous ingredients of the drink. To start off, a good idea would be to google the ones that are already provided in the description. By googling the names of the fruits for this juice, the top result in Google show a an article of “Top 20 Fruits You Probably Don't Know”. Moreover, on the same websites, in the comment section, one of the users refers to the Rippertuer Special Juice, with the link for a full list of ingredients for the juice.

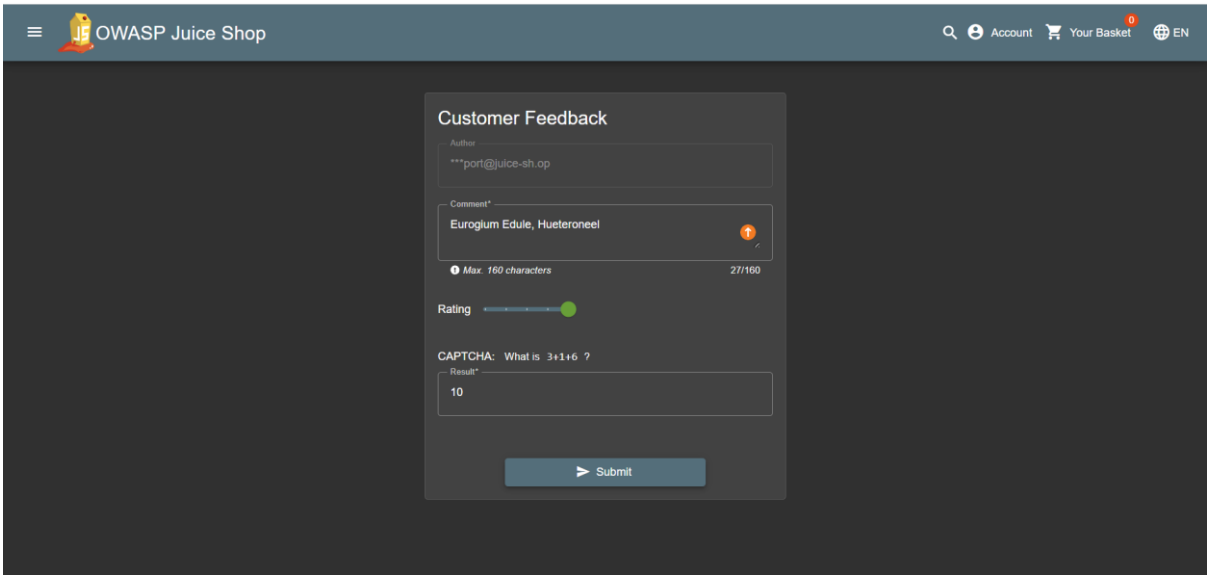




All that's left now is read the descriptions and to google each of the ingredients present in the full list and determine which ones are potentially harmful to solve this challange. In the end, as is was written in the description, two potentially harmful ingredients if combined with one another are Eurogium Edule and Hueteroneel.



By pasting them into the contact page, the challange now is solved.

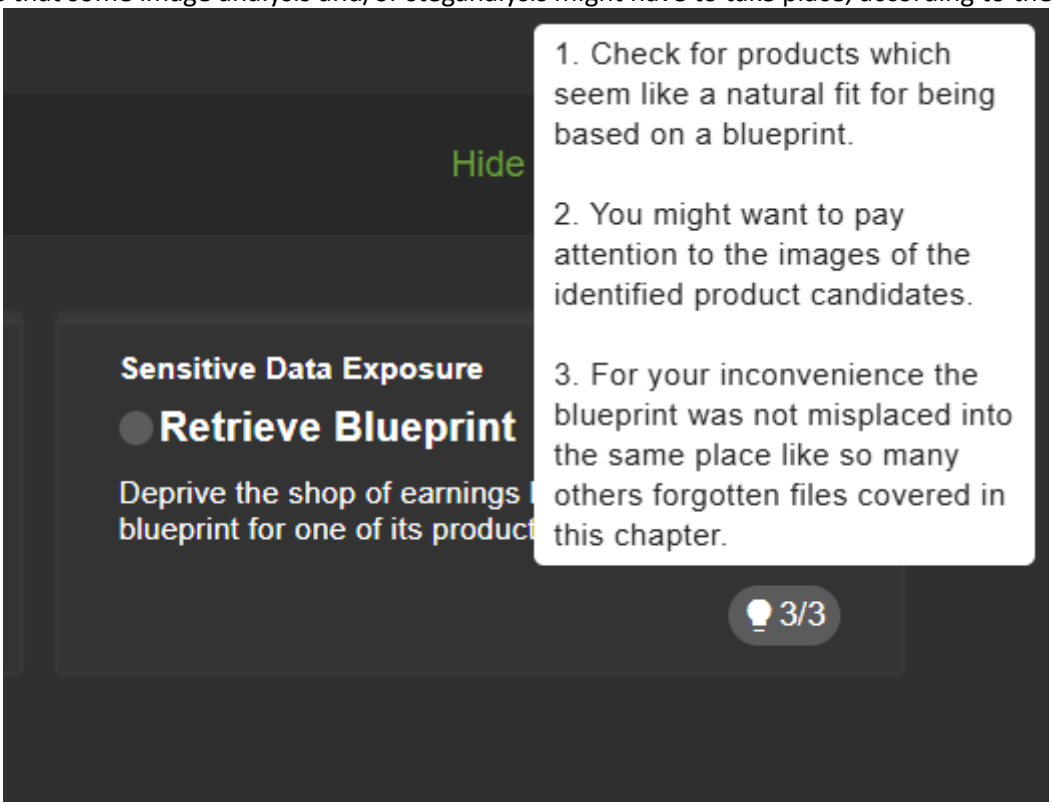


□ □ □ □ □ Retrieve Blueprint:

Thanks to hints for the challange, it is very possible that the following is true regarding the hunt for a product blueprint:

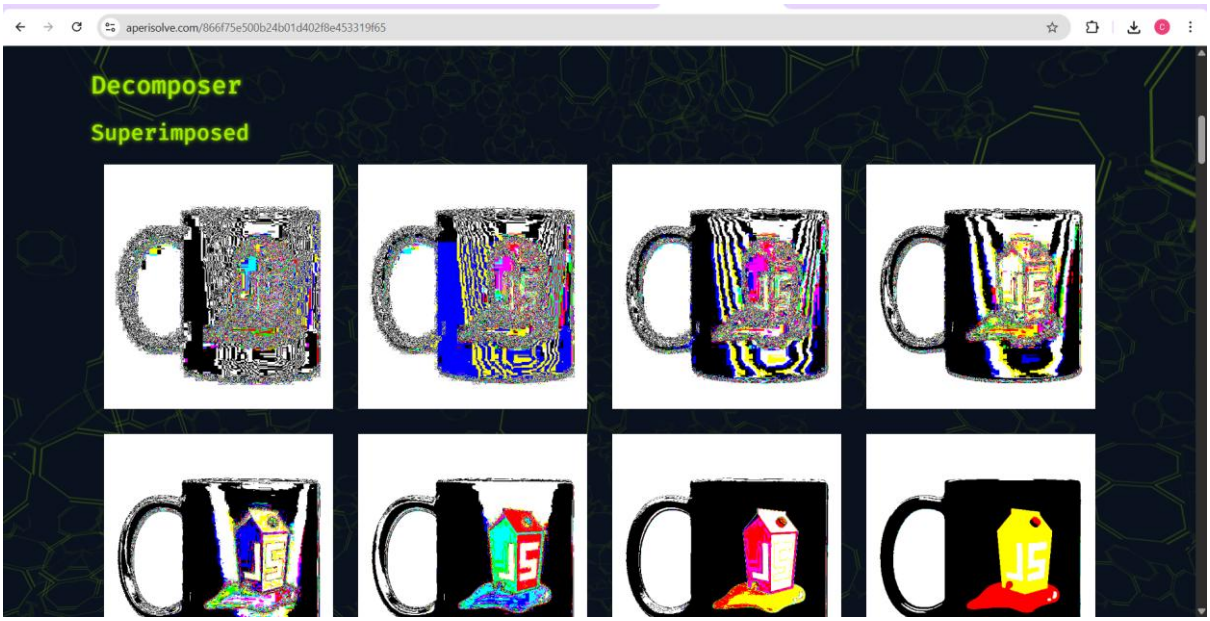
There won't be any kind of file with a blueprint in the usual /ftp/ folder on a website.

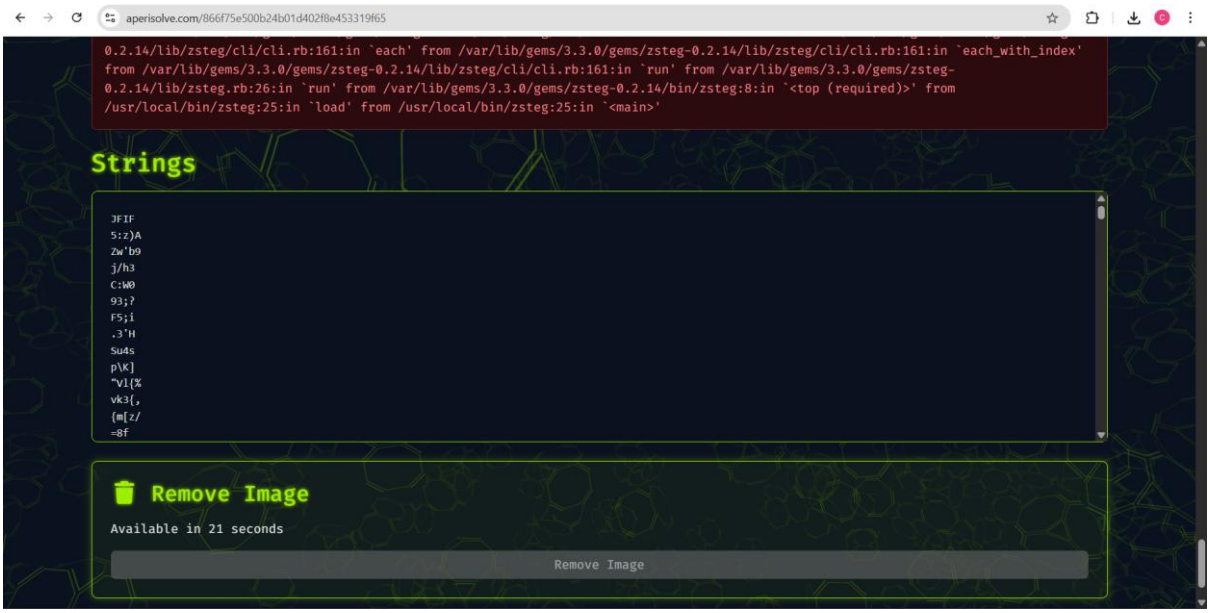
It's possible that some image analysis and/or steganalysis might have to take place, according to the hint №2.

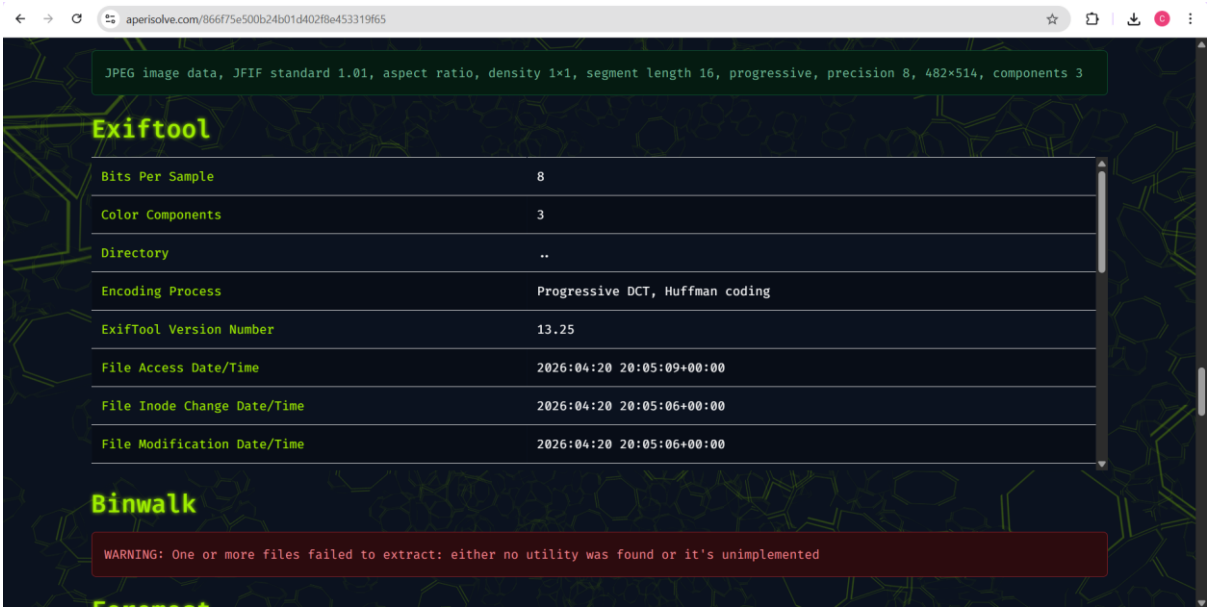


So, I decided to go through the products on the website, downloading images and then analysing them with powerful online image analysis tool – Aperi'Solve. After analysis is done, the areas of importance to which I paid most attention, are Exiftool's report on file's matedata, which is integrated into Aperi'Solve own report, image strings, and visual representation of an image with various filters applied, in case there some visual hidden features.

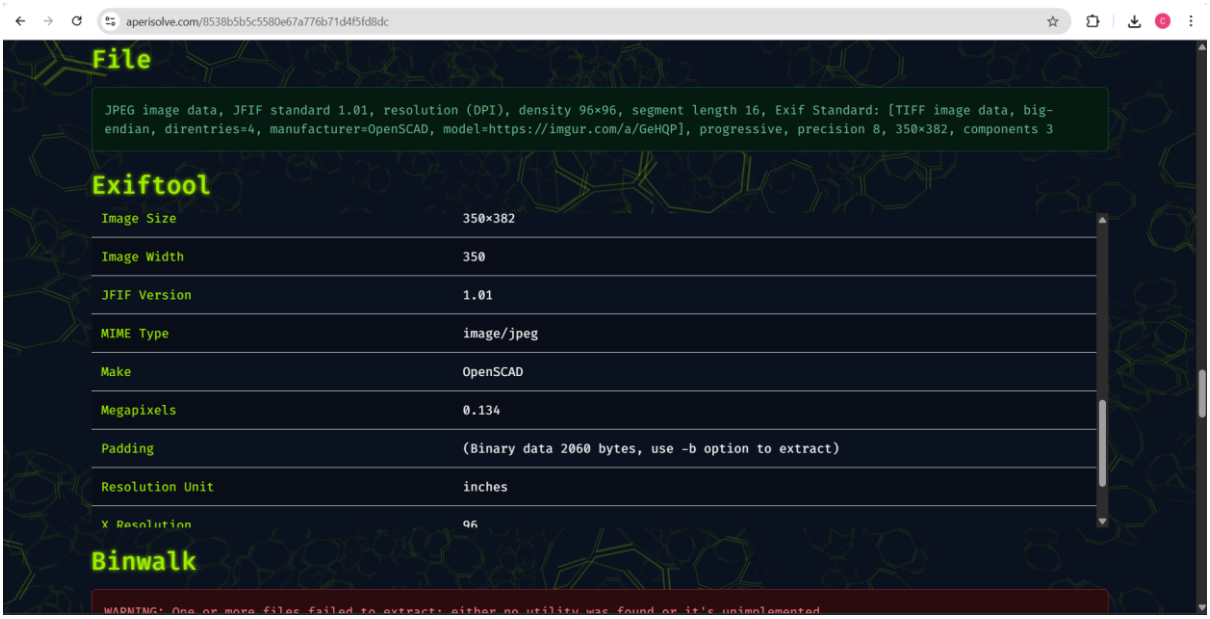
As an example, there are pictures of a report of Aperi'Solve on fan_mug.jpg of OWASP Juice Shop Mug product.





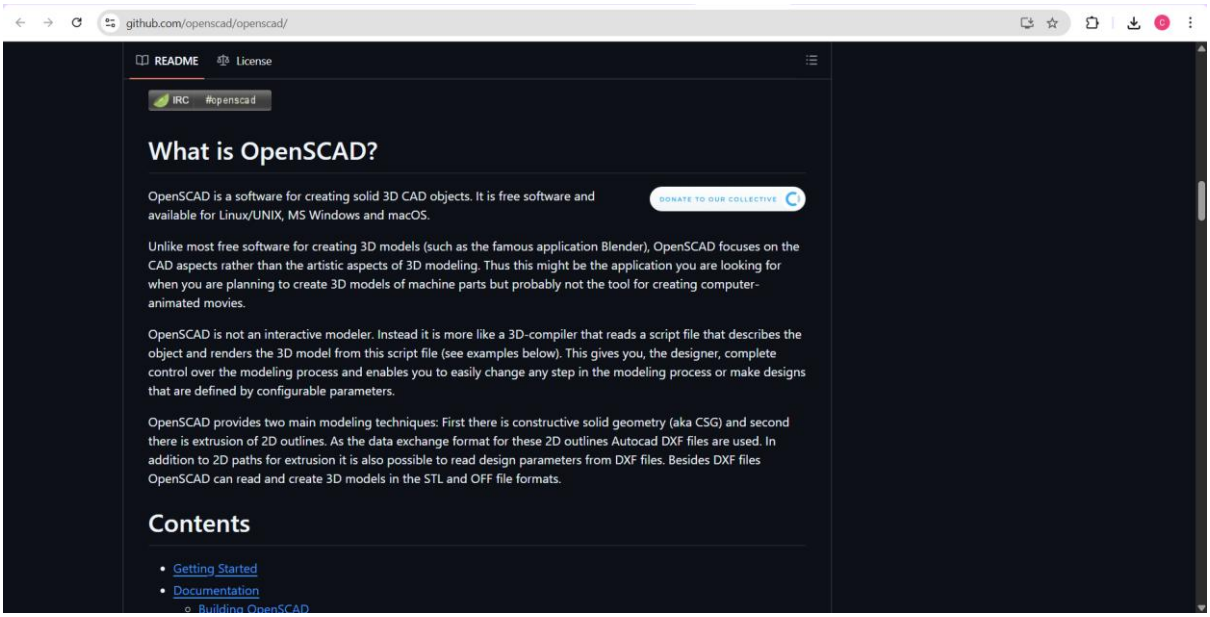


Repeating this process, I finally stumbled upon a product which had some unusual features present in the report. 3d_keychain.jpg had a separate section present in it's exif and strings.



From the metadata we can see, two important fields: manufacturer and camera model. By following the link to https://imgur, nothing came up, likely because the photo has been hosted only for a limited time on that website.

Following another clue, OpenSCAD – the “manufacturer” which in reality is a software for creating solid 3D CAD objects, gives some important imforamtion about the blueprint. From their official github page, we can find that the file types used by this software are .stl and .off.



Now, in order to retrieve the blueprint and complete the challange, we need to get our hands on the file with one of the aforementioned formats. Since I couldn’t find any further hint on the blueprint filename, a lucky guess is the best bet. In the end, by downloading the file at http://localhost:3000/assets/public/images/products/JuiceShop.stl the challange is complete.

Vulnerability Analysis

Sensitive Data Exposure

- Vulnerability description and root cause

  Sensitive Data Exposure, categorized as A3:2017-Sensitive Data Exposure (or as A02:2021 in newer 2021 version of OWASP Top 10), occurs when applications fail to protect sensitive information such as Personally Identifiable Information (PII), financial data, health records, and authentication credentials [1]. This vulnerability maps to CWE-200 (Exposure of Sensitive Information) [3]. The root cause comes from either lack of cryptographic controls: transmitting data over unencrypted channels, or when crypto is employed: weak key generation and management, and using weak or deprecated algorithms (MD5, SHA1), and employing hardcoded keys [2]. Developers often treat cryptography as an afterthought rather than properly implementing it into architectural design, leaving sensitive data exposed.
- Rationale or investigative process

  The investigation process focused on specific Juice Shop challenges. For "Reset Uvogin's Password," the Forgot Password mechanism was analyzed, revealing that security questions relied on publicly discoverable information, which requeired use of some OSINT tools. For "Leaked Unsafe Product" and "Retrieve Blueprint," while using Burp Suite, the /rest/products/search endpoint was intercepted which was used to load products on the home page. Injecting a SQL payload ')) -- retrieved the complete product database, including deleted items that remained accessible via the API . Further investigation involved external reconnaissance: searching for the unsafe product name led to a Pastebin link containing dangerous ingredient information.
- How the exploit works

  To reset Uvogin's password, the attacker researches the public account on Twitter to determine the answer to his security question, then submits it via the Forgot Password flow to gain account access. For the unsafe product, the attacker injects ')) -- into the search API to retrieve the deleted Rippertuer Special Juice, then submits the dangerous ingredients Eurogium Edule and Hueteroneel discovered via the leaked Pastebin on a website’s comment section.
- Potential impact

  Exploitation of this vulnerability further harms the business as it enables identity theft, financial fraud, and targeted phishing. Moreover, reputational damage causes customers’ lack of trust, while exposed data creates

competitive disadvantage. For example, the 2022 Optus breach demonstrated how a single configuration failure can cascade into catastrophic organizational crisis when a misconfigured API exposed millions of customer records with sensitive information [4].

- Recommended mitigation

  Implement TLS 1.2+ for all data transit and enforce HTTPS using HSTS headers [1]. Use AES-256-GCM for encryption and Argon2id for password hashing with unique salts. Establish automated key management using AWS KMS or Azure Key Vault, eliminating hardcoded secrets. Apply OWASP ASVS requirements V7, V9, and V10 as mandatory development checkpoints. Classify data to identify protection requirements and encrypt data at rest using transparent database encryption. Conduct regular penetration testing and TLS scanning using SSL Labs to verify cryptographic controls, aligning with OWASP Proactive Control C8: Protect Data Everywhere [1].

Security Misconfiguration

- Vulnerability description and root cause

  Security Misconfiguration is A02:2025 in the OWASP Top 10, covering improperly implemented security settings across the entire application. Mapped to CWE-16 (Configuration) and CWE-276 (Incorrect Default Permissions), it includes default accounts, unnecessary services, debug modes, and verbose error messages [5]. The root cause is the absence of repeatable, automated hardening processes. Organizations might prioritize fast deployment over proper security, leaving configuration gaps as software changes and shifts toward, for example, cloud architectures with expanding configurations, which adds more attack surfaces [6].

- Rationale or investigative process

  The investigation examined Juice Shop for configuration weaknesses. For "Cross-Site Imaging," the image embedding features were analyzed through main.js file, revealing misconfiguration that allowed unintended cross-domain content loading. For "Login Support Team," the /ftp directory was discovered through one of the website's pages containing terms (legal.md), to be publicly accessible, containing an unprotected KeePass database file (incident-support.kdbx). Analysis of main.js revealed a regex pattern enforcing the certain password policy. Using this intelligence, a brute-force attack was done against the KeePass database, successfully cracking it with the weak but policy-compliant password "Support2022!".

- How the exploit works

  For Cross-Site Imaging, the attacker exploits misconfigured image handling to embed cross-domain images into the delivery box, demonstrating improper content security controls. For the support team login, the attacker downloads the exposed KeePass database from /ftp/incident-support.kdbx, then applies the regex pattern found in JavaScript to use in brute-force John the Ripper's parameters and rules. The password Support2022! cracks the database, revealing production credentials for support@juice-sh.op. The attacker logs in with these credentials, achieving privileged access.

- Potential impact

  Misconfiguration often serves as the initial foothold for broader compromise, enabling unauthorized administrative access and mass data exfiltration. Exposed source code accelerates vulnerability discovery, while cloud misconfigurations permit lateral movement across organizational infrastructures [7]. Service disruption occurs when attackers abuse admin interfaces to delete resources or deploy cryptominers. Compliance violations under SOC 2 or PCI DSS trigger audit failures and penalties. The 2022 Uber breach demonstrated how hardcoded credentials in scripts enabled attackers to access privileged systems and sensitive data stores [8].

- Recommended mitigation

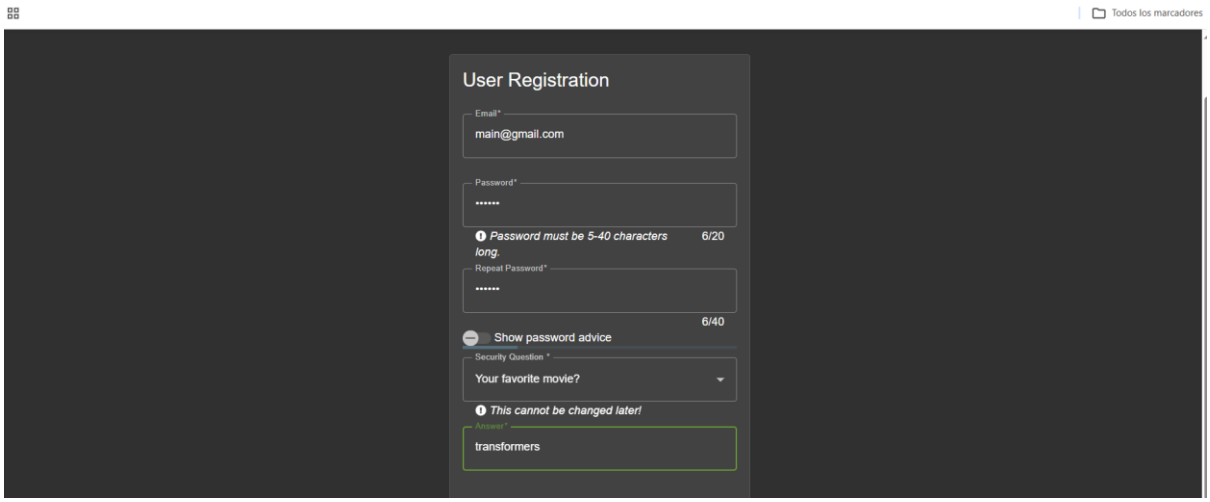
  Establish automated, repeatable hardening processes using the minimal platform principle—remove unnecessary features, services, and default accounts before production deployment . Enforce immediate credential rotation and MFA for all administrative interfaces. Configure standardized security headers and implement a WAF with targeted rulesets [5]. Adopt Infrastructure as Code with security baselines encoded as policy-as-code to prevent configuration drift. Implement generic error handling that logs diagnostics to centralized systems without exposing stack traces to users [6]. Conduct continuous configuration monitoring using cloud security posture management tools [5]. These measures align with OWASP ASVS V14, NIST Server Hardening Guidelines, and CIS Benchmarks.

Miguel Caracuel Millan B00176896

Solved Challenges

□ □ Empty user registration:

First, we start try to create a new account with a fake email and password:



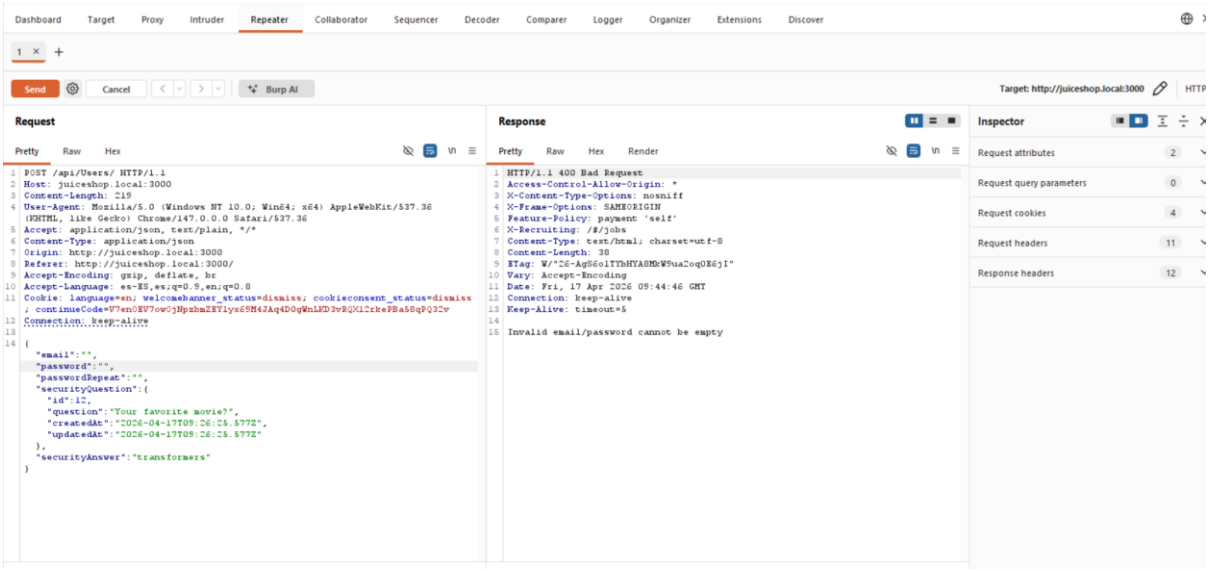
Then we intercepted the request using a proxy to see how we can make this empty email registrate. At first I tried putting an input that will not be valid like “‘” or “–” in the fields of email and see what happened but still did not work, then I tried just putting an empty string in both fields.

Request

Pretty Raw Hex

```
1  POST /api/Users/ HTTP/1.1
2  Host: juiceshop.local:3000
3  Content-Length: 219
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
5  Accept: application/json, text/plain, */*
6  Content-Type: application/json
7  Origin: http://juiceshop.local:3000
8  Referer: http://juiceshop.local:3000/
9  Accept-Encoding: gzip, deflate, br
10 Accept-Language: es-ES,es;q=0.9,en;q=0.8
11 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
   ; continueCode=V7en0EV7ow0jNpzbmZEYlyx69M4JAq4D0gWnLKD3vRQX12rkePBa58qPQ32v
12 Connection: keep-alive
13
14 {
     "email":"",
     "password":"",
     "passwordRepeat":"",
     "securityQuestion":{
       "id":12,
       "question":"Your favorite movie?",
       "createdAt":"2026-04-17T09:26:25.577Z",
       "updatedAt":"2026-04-17T09:26:25.577Z"
     },
     "securityAnswer":"transformers"
   }
```

We tried setting the email and password as empty string.

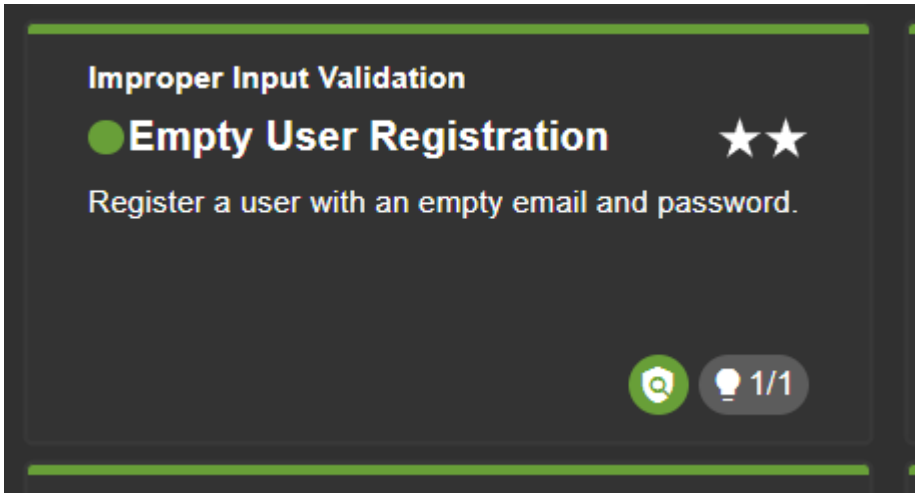


And now we have it completed:

As we can see inputting a string with no value in it (empty) worked to registrate the email.

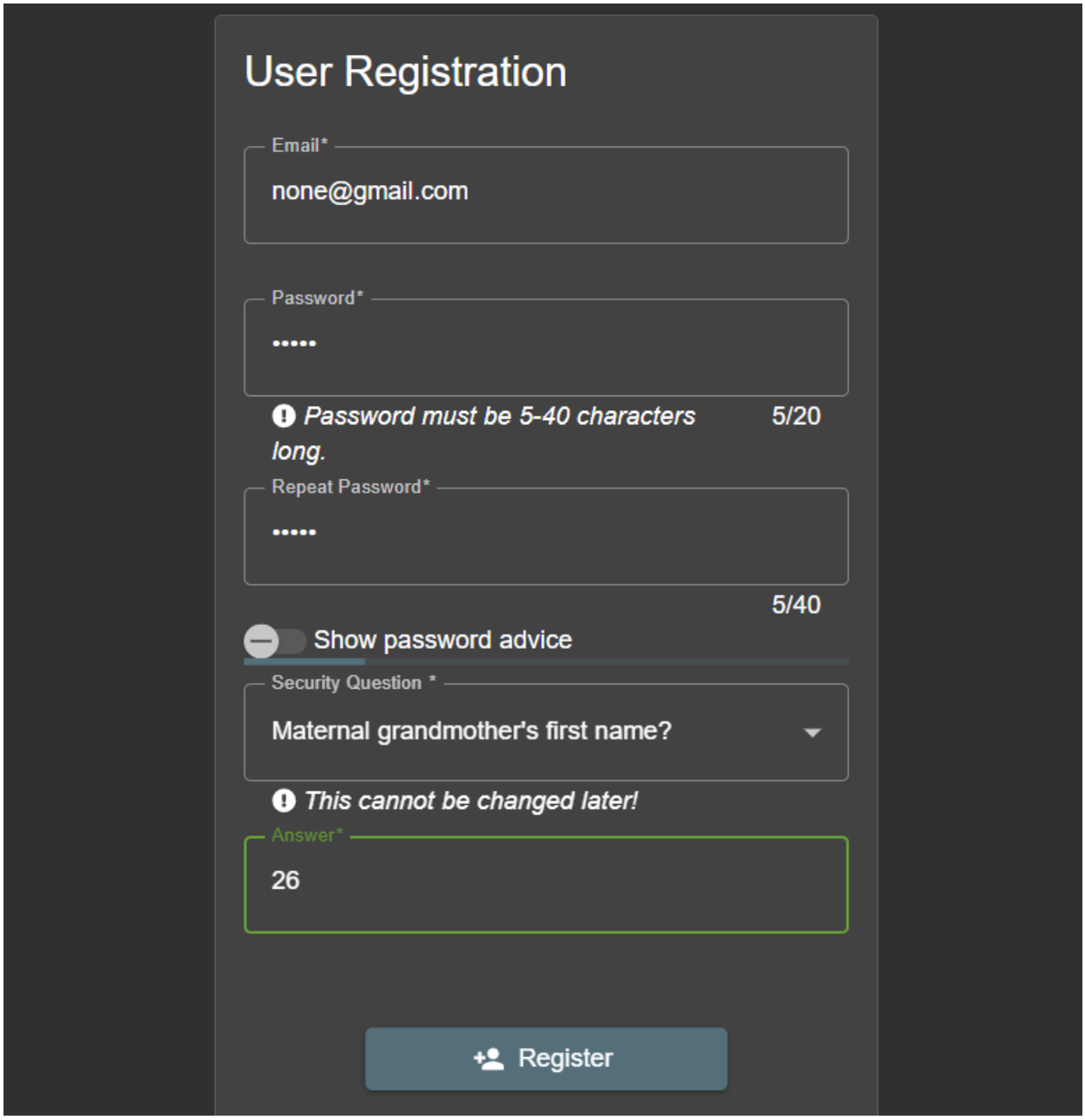
This is because the background of the web page sees the empty string as a valid value, then the database stores it because it is configured to store valid values even if it is empty.

The website has wrong configuration, and it validates empty values without even checking it.



□ □ □ Admin Registration:

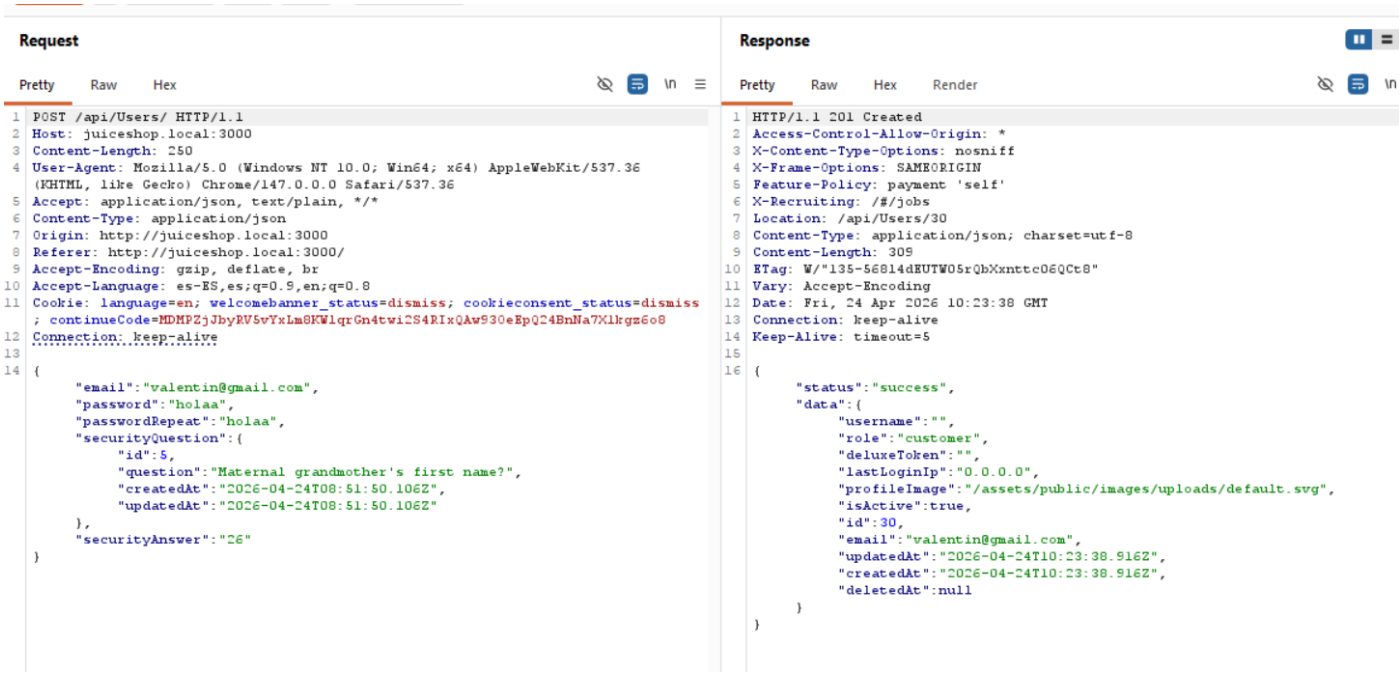
At first we start with the email registration:



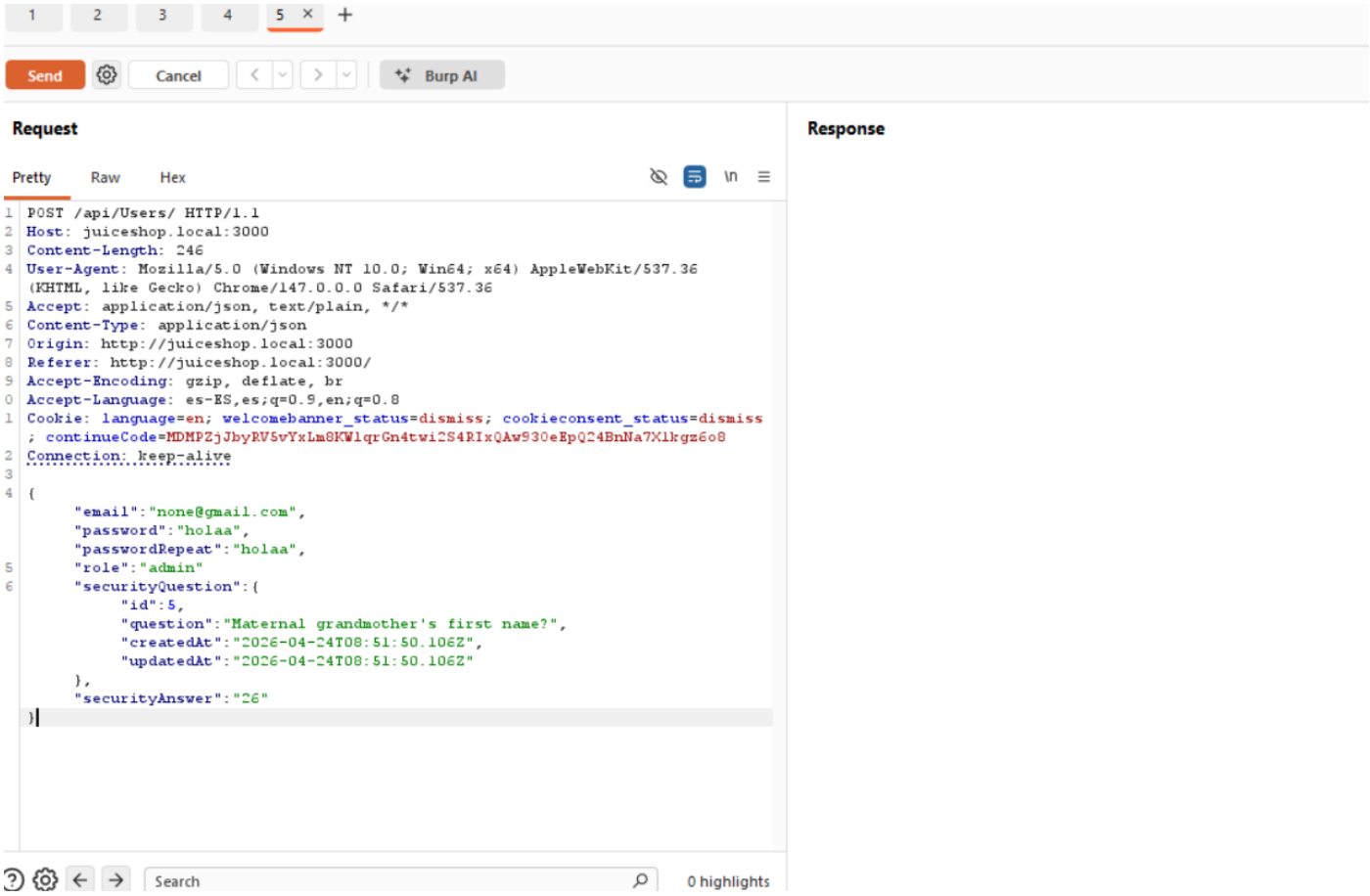
When we send the registration API request we intercept it with burpsuite proxy:



As we can see there are all the information in the request

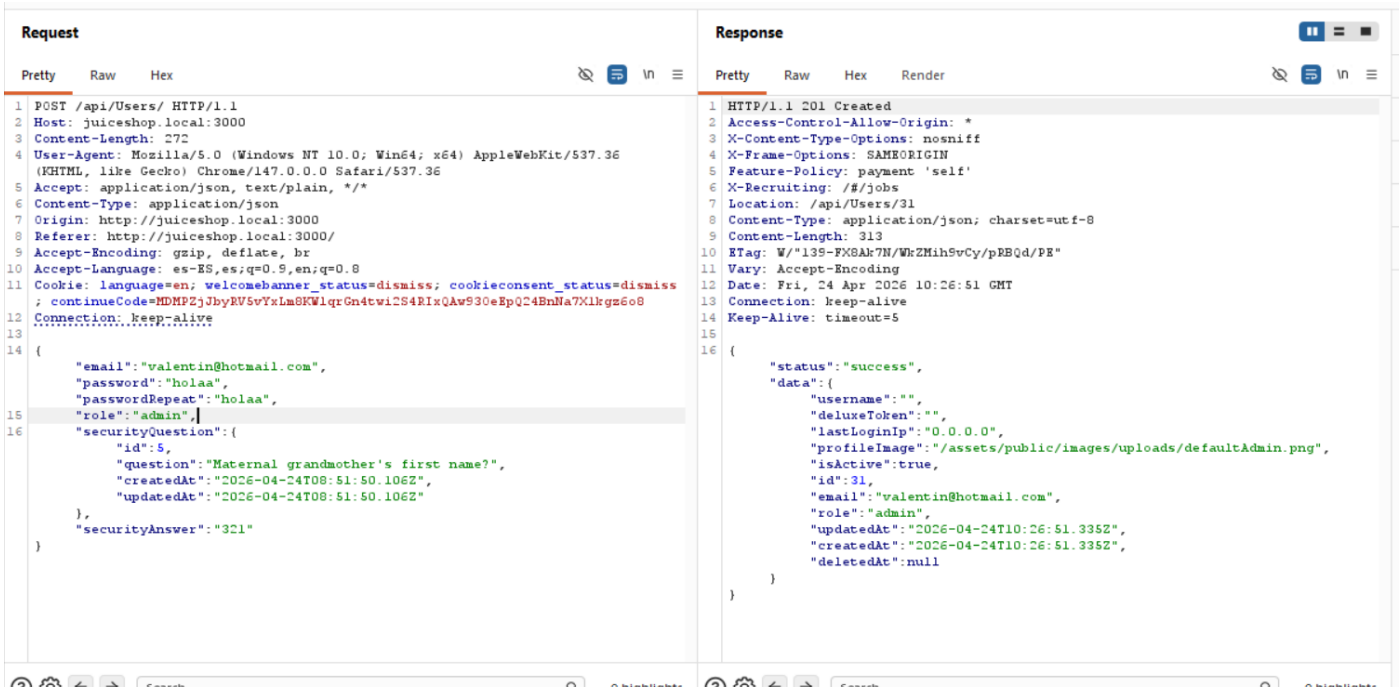


After intercepting the request, we can see that there is a field called role that in this request has the output “customer” when you forward it.

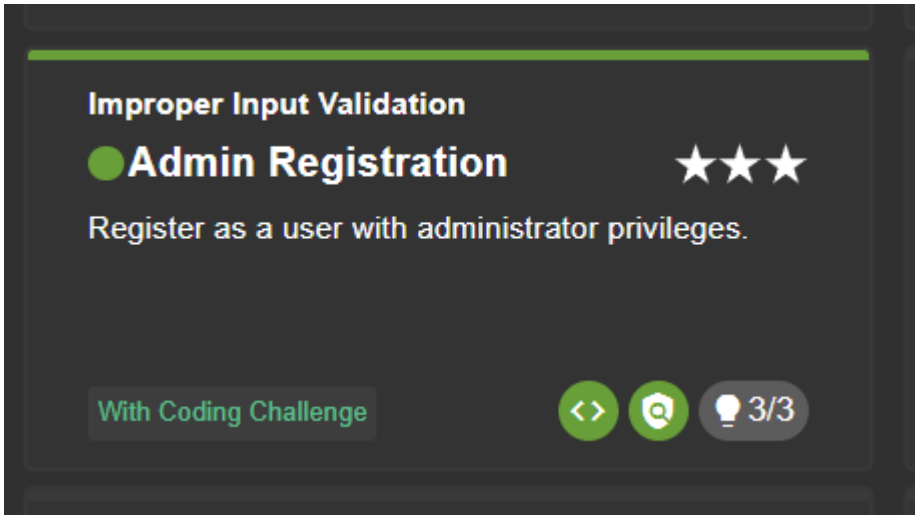


I tried adding the role into the fields in my request and at first did it wrong because I did not put the comma.

But then I entered the role correctly and as we can see in the next image I finished the task



In the forwarded request we can see that the role field has admin on it.

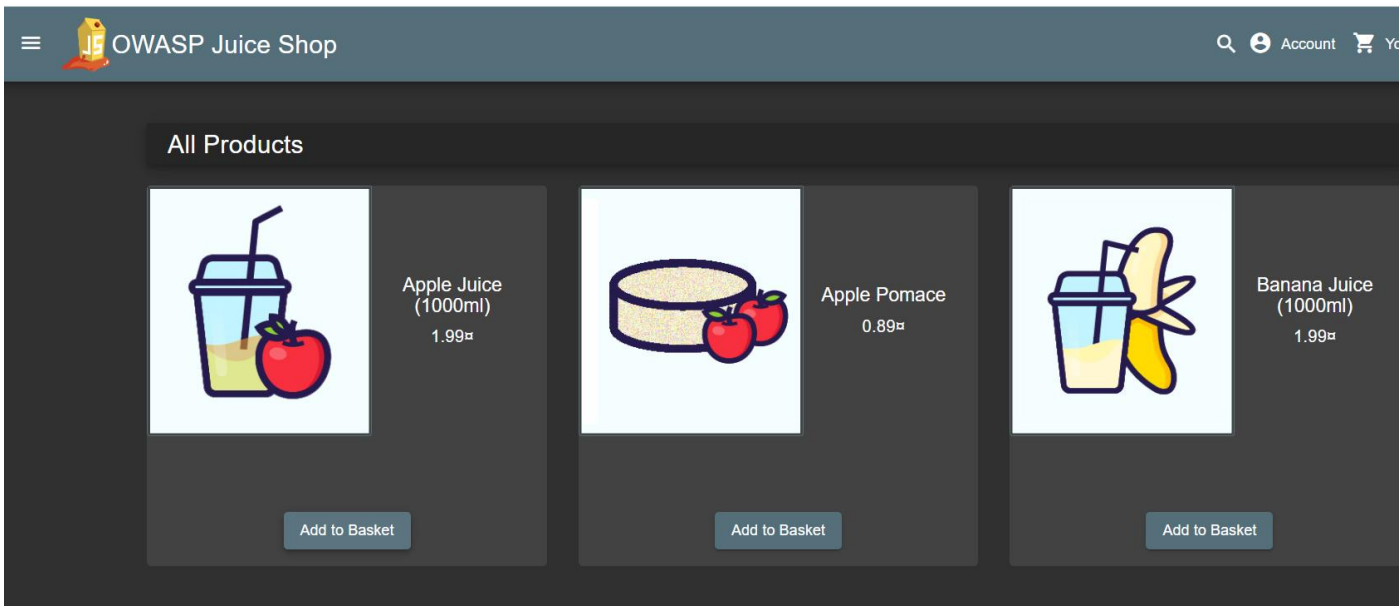


And we finish with the task.

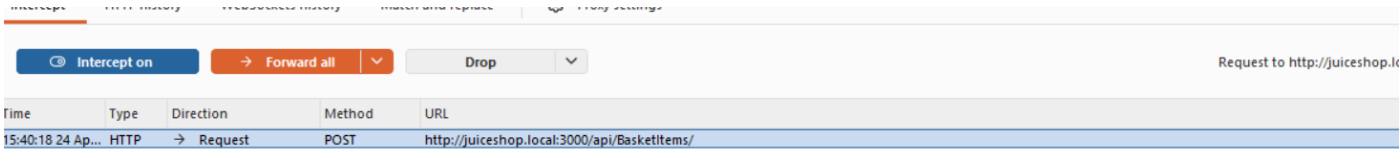
□ □ □ Payback Time:

In this task the objective is to “Place an order that makes you rich”, meaning that we want to instead of paying for making an order we want to gain money exploiting the basket function in the web page.

First we put a product into our basket.

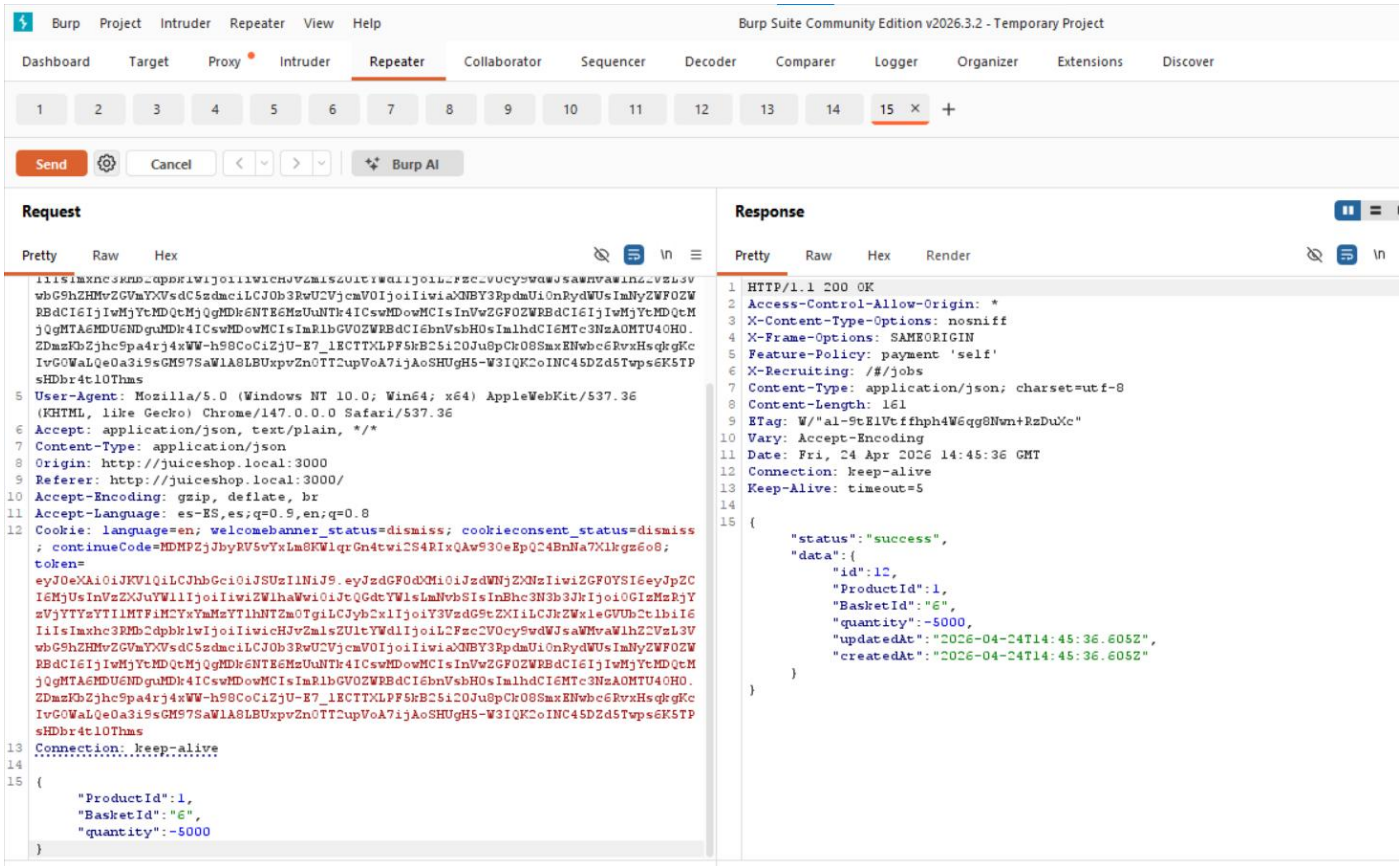


Once we click in the add to basket buttom and then we intercept the request in the proxy:

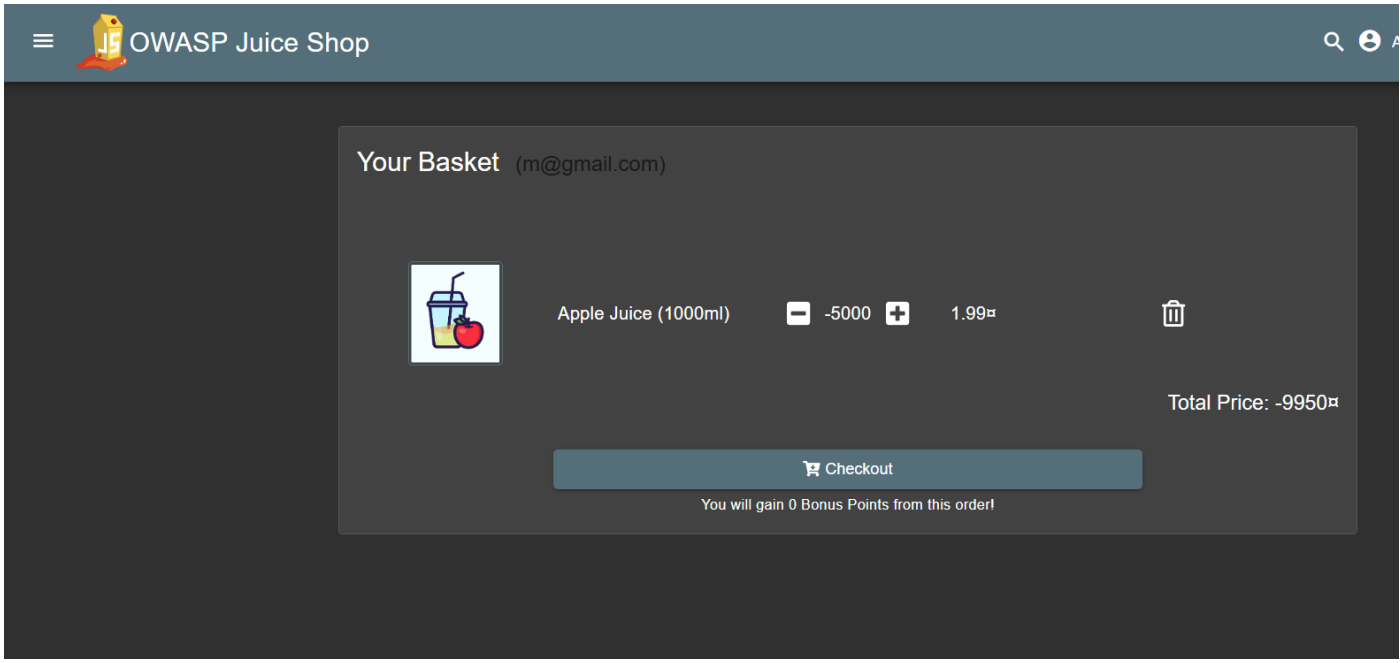




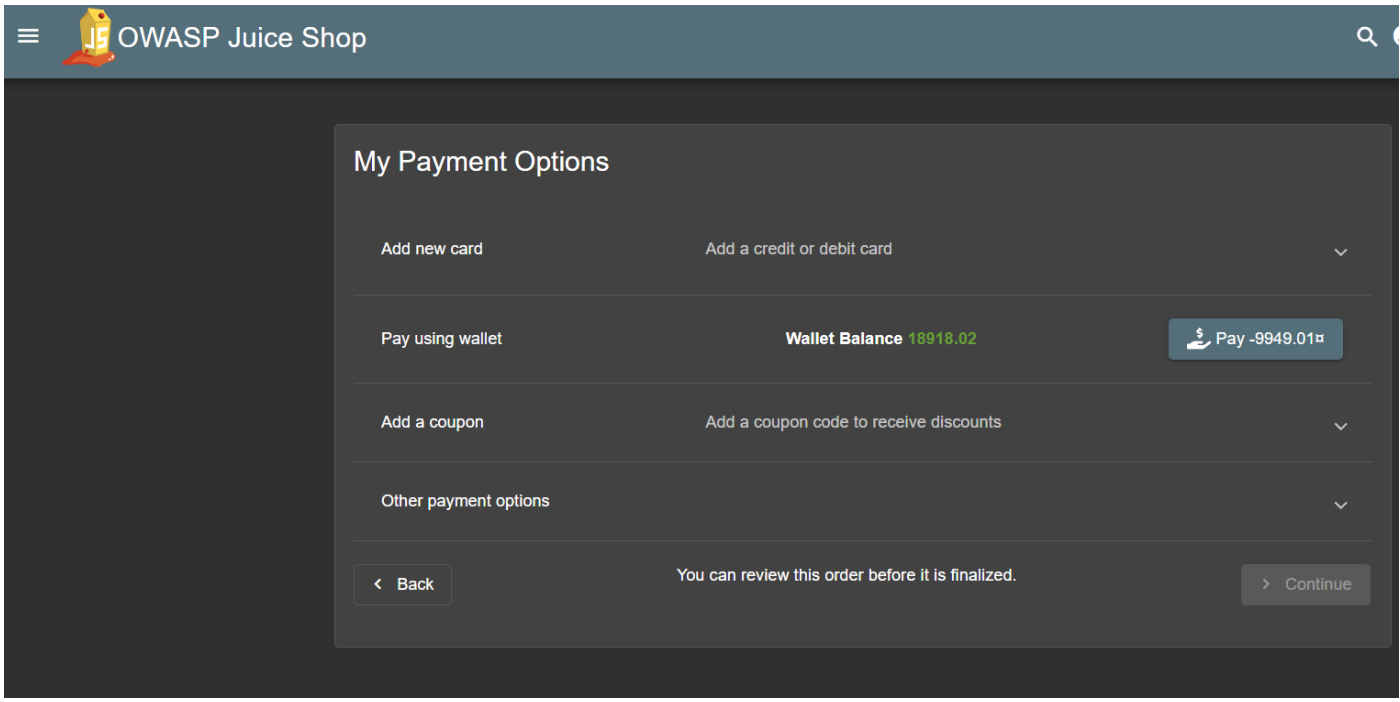
we see that it has 3 fields: ProductId, BasketId and quantity, then I tried setting the quantity to a negative value in the repeater to try to get money from the purchase:



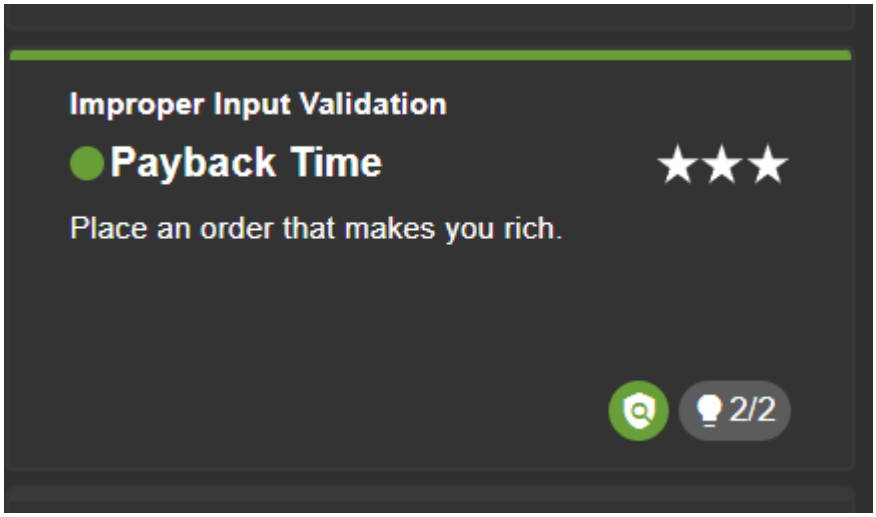
I set the quantity to –5000 which would mean that instead of paying 5000 units of this product, I will be receiving the money that this product is worth 5000 times.



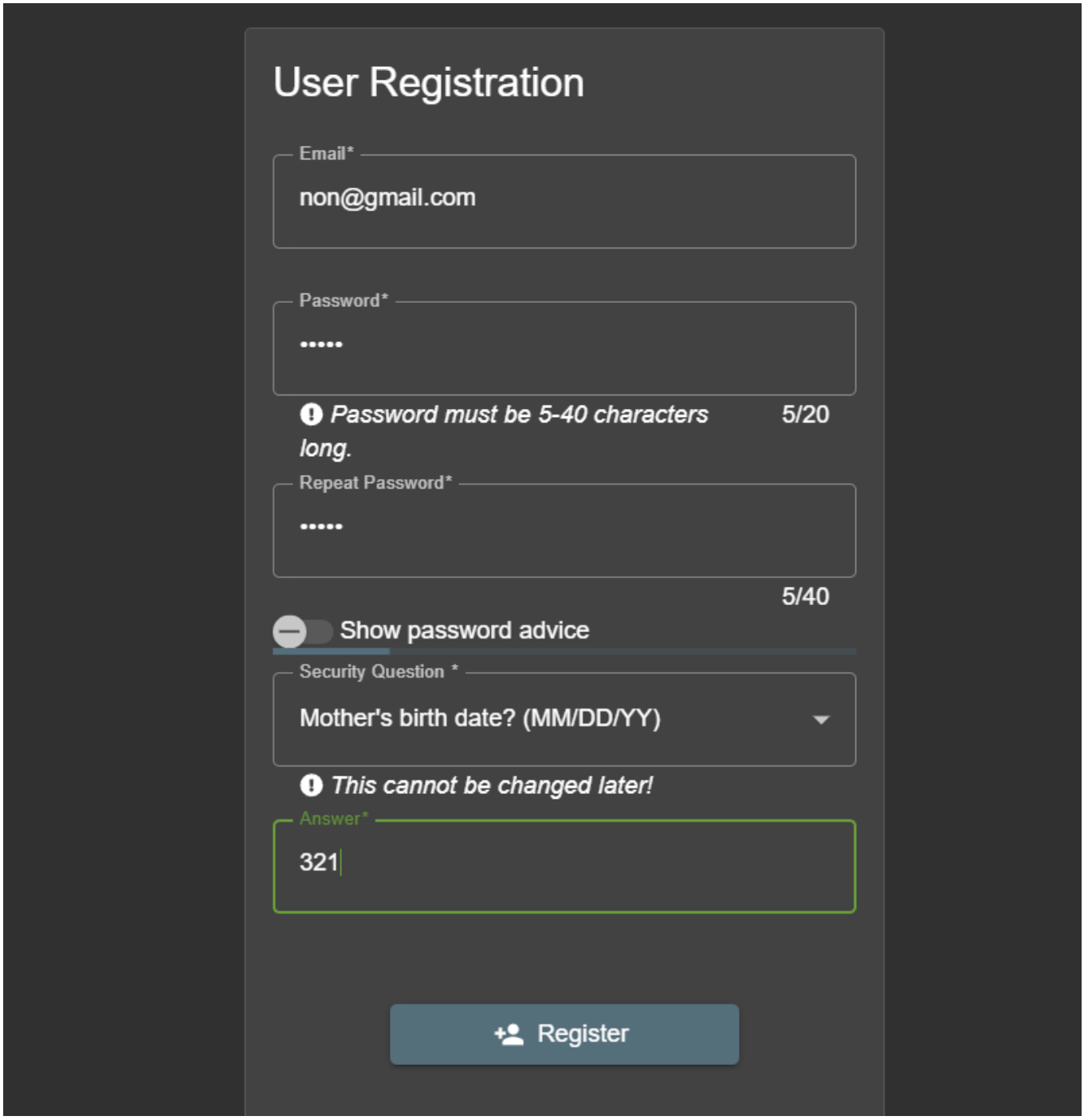
As we can see the purchase page also displays this change of –5000.



In the checkout that the price is also displayed as negative which means that I am not spending money but receiving it.



We completed the task.

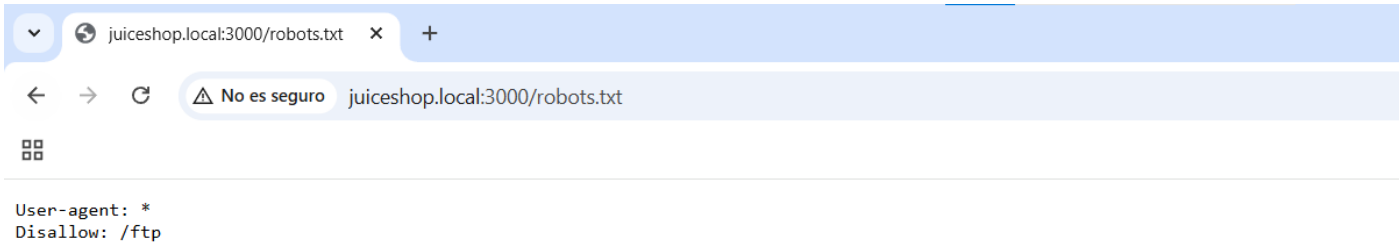


First we start with the email registration in juice shop.

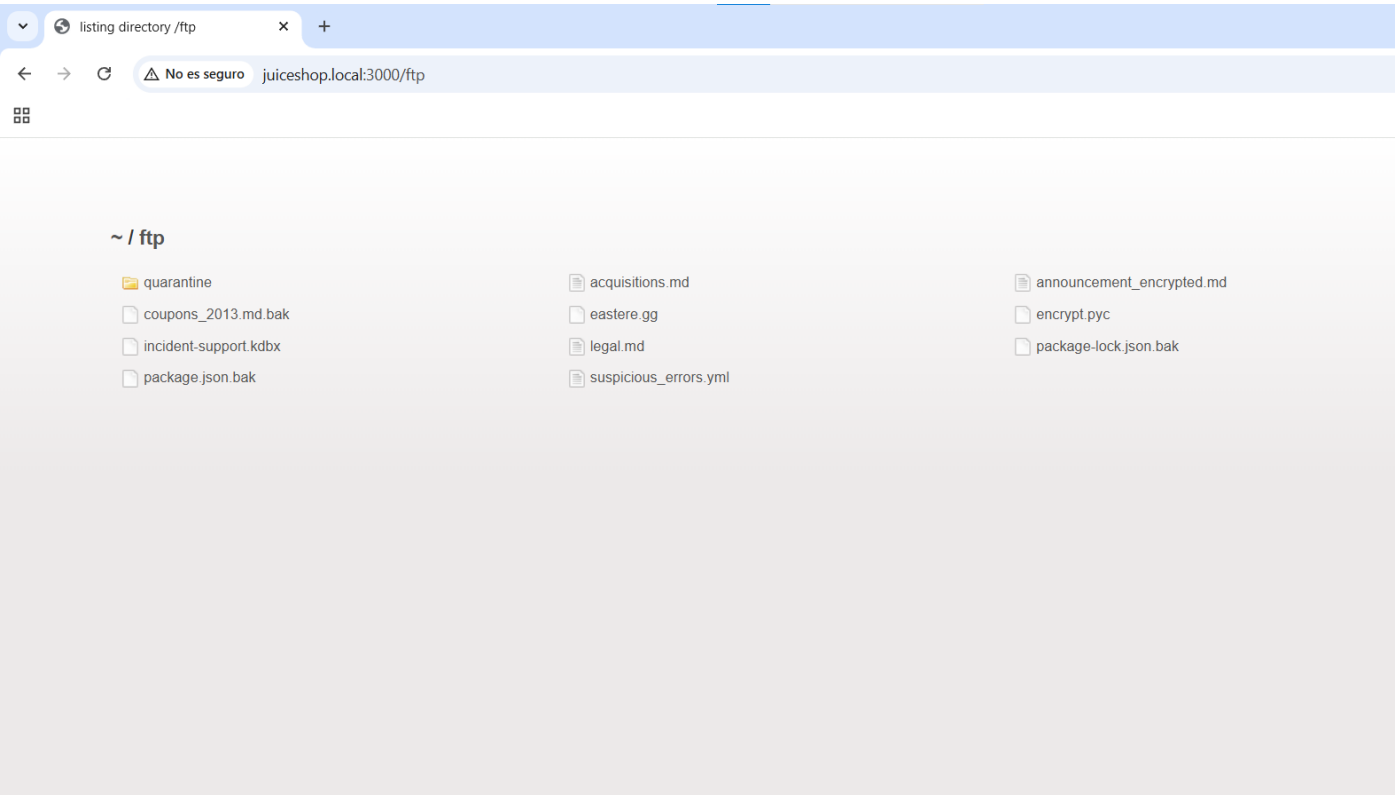
□ □ □ □ Access Log:

In this challenge our objective is to find the Access logs from juice shop stored in some place of the web page:

First, we try looking for usual places to find the file and while trying I looked at robots.txt and found this:

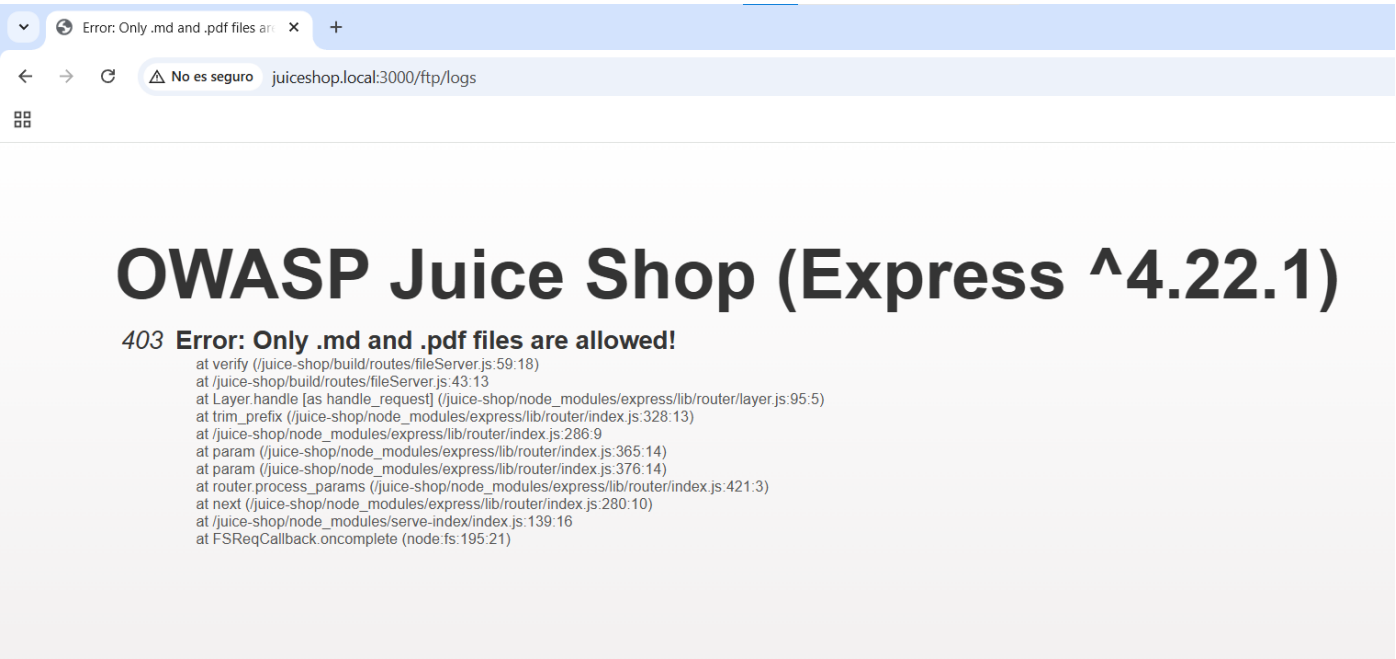


which was saying disallow /ftp and I tried to search for that and found some files:

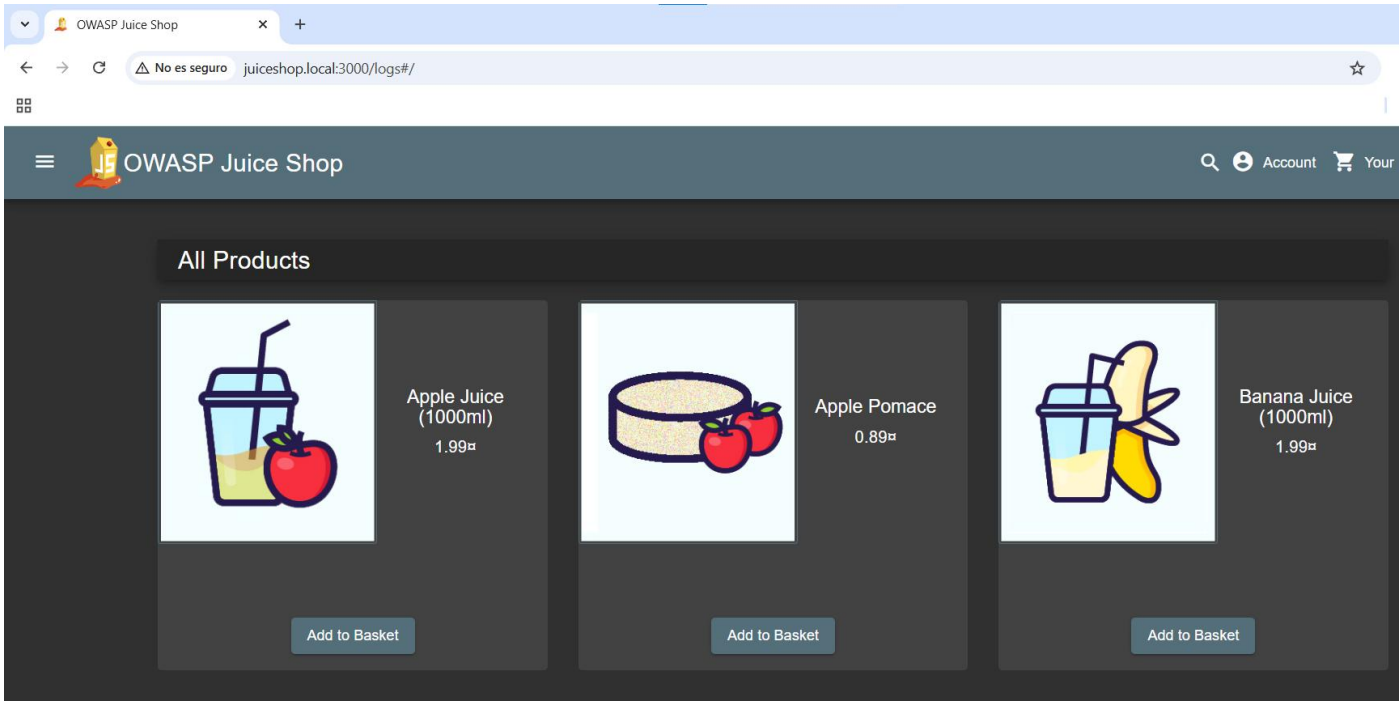


Then I saw that there were several files but the one that looked more important was incident-support.txt.

Then from that I tried changing the URL for different extensions like /ftp/logs or /ftp/access.log but it was returning error 403 and saying that only .md and .pdf files were available.

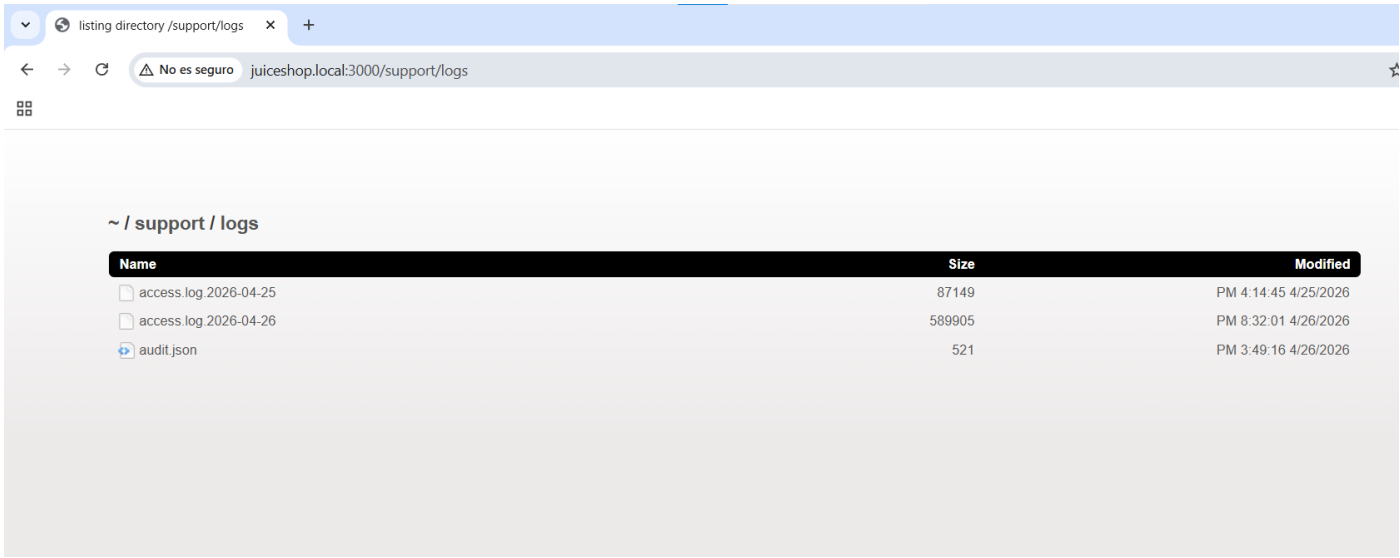


After not being successful with the URL tries, we tried without using /ftp, we tried different URL and always kept getting the same response:

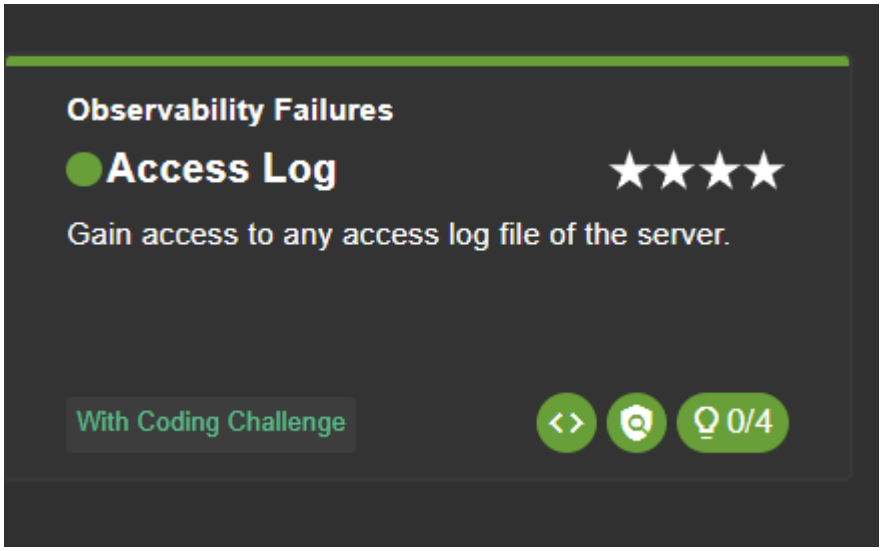


It was always getting the main page as return of the request for each page, after looking to each URL, we see that in the file we saw in /ftp, the name was: incident-support.kdbx.

And we assume that the directory support exists because it does not give any error but returns us into the main page so then we tried /support/logs and it returns us the access log page:



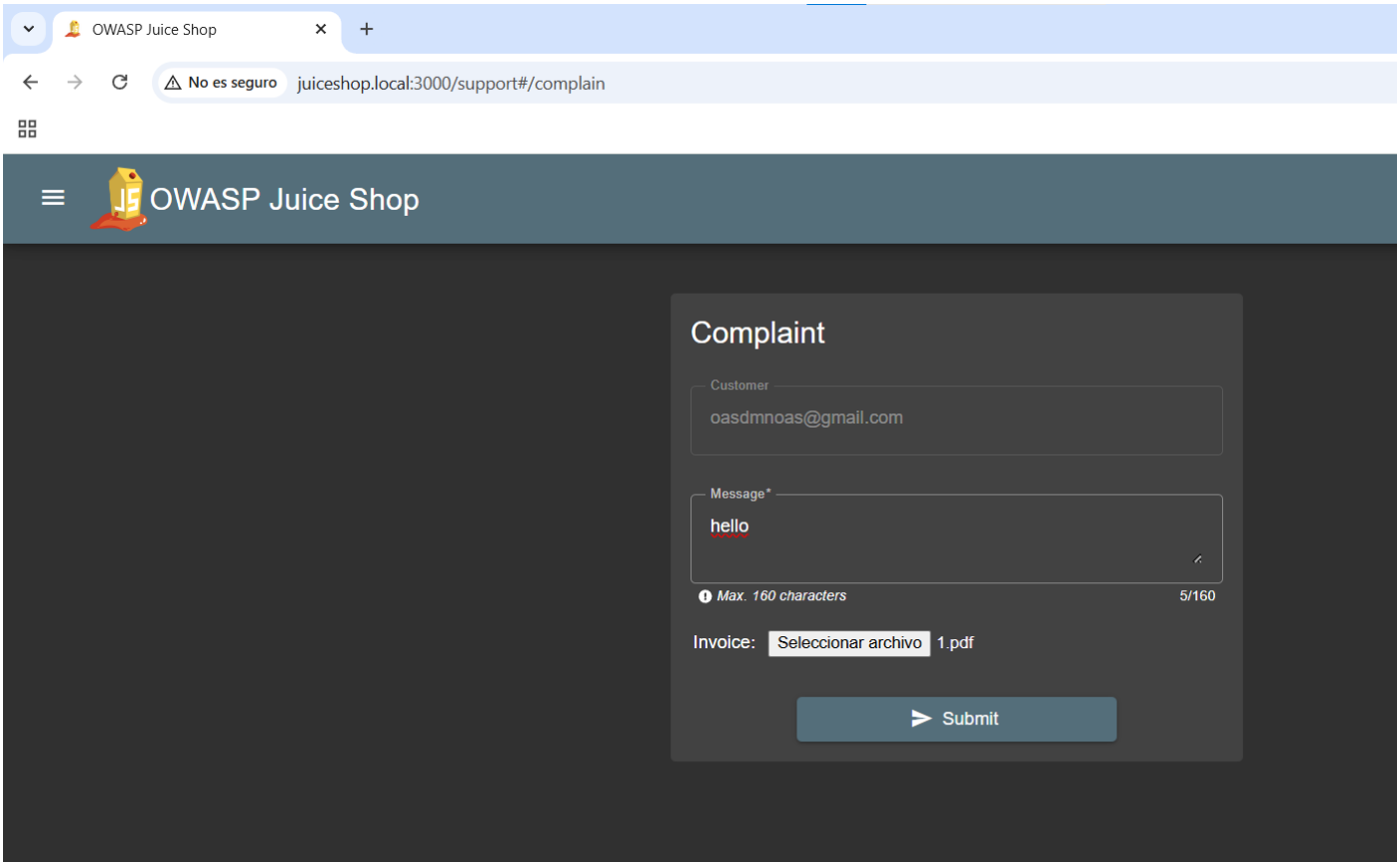
And then we download the access.log file and we have completed the challenge.



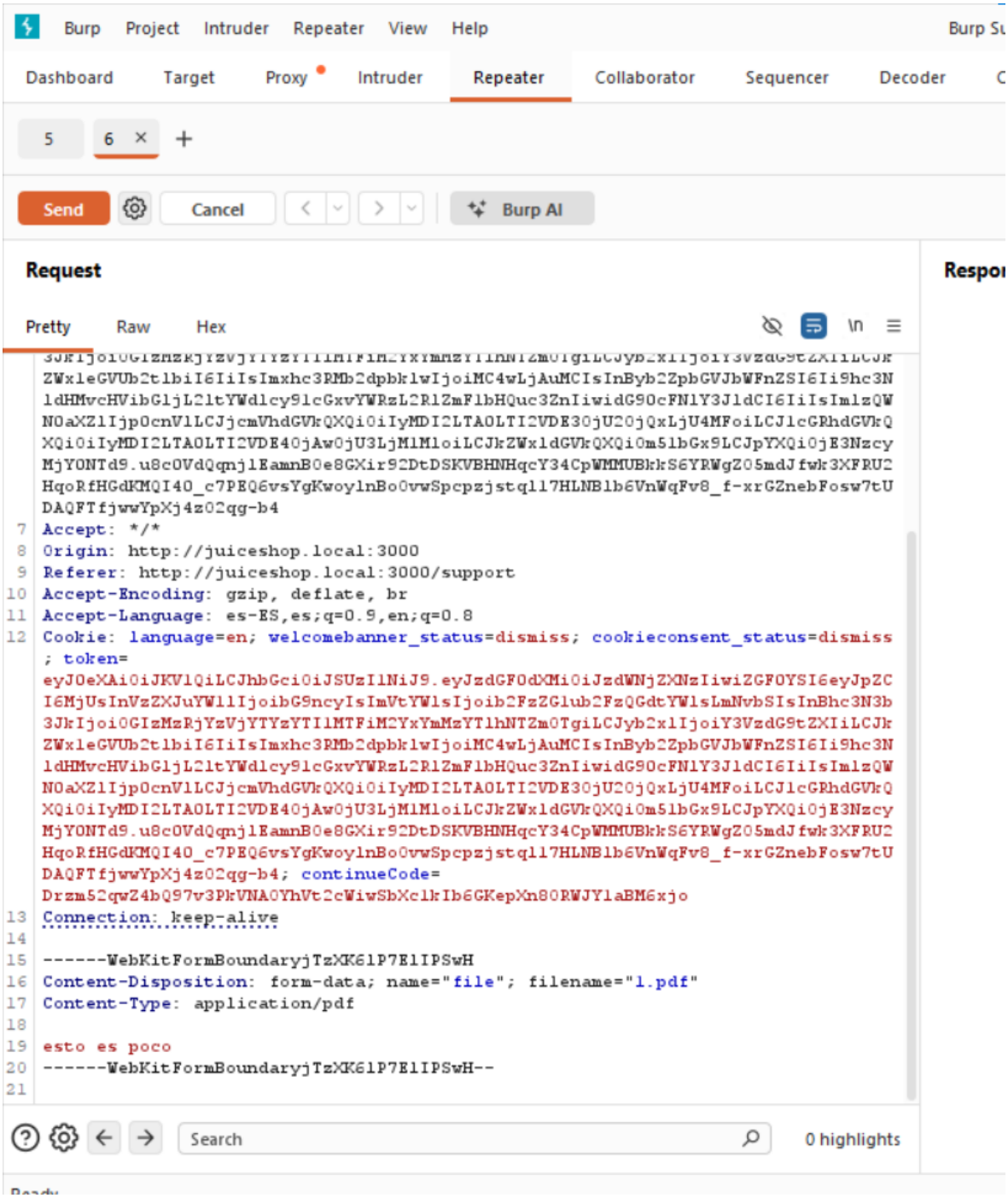
□ □ □ Upload Size:

First, we must try to upload a file bigger than 100kB in the complaint section:

We submit a file that is smaller than 100kB because the page does not accept any file bigger than that size.



Then we intercept the POST request using burp suite's proxy and see the content of the submitted file, we send it to the repeater:

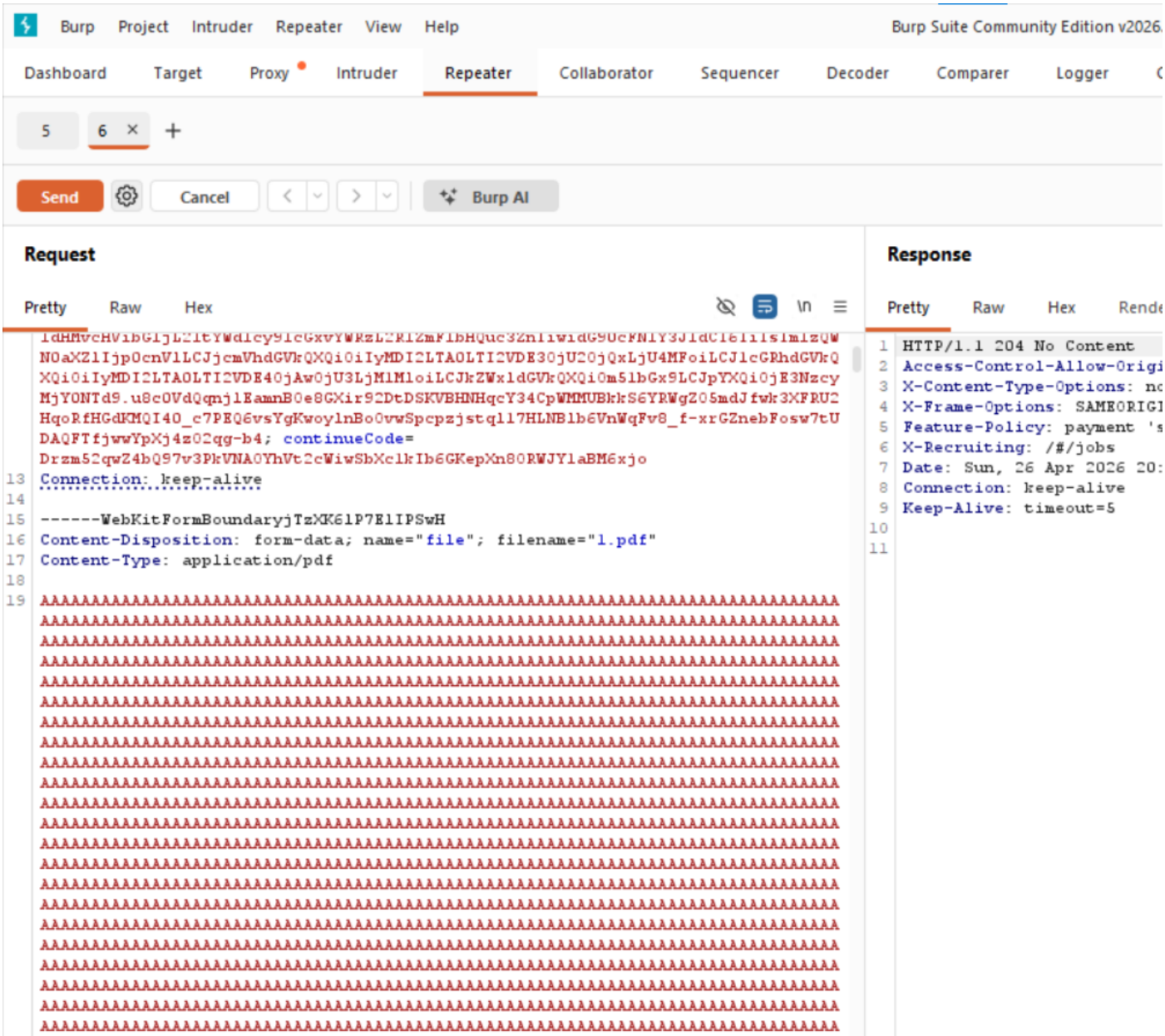


As we can see there is the content of the file submited: name 1.pdf and the content “esto es poco”.

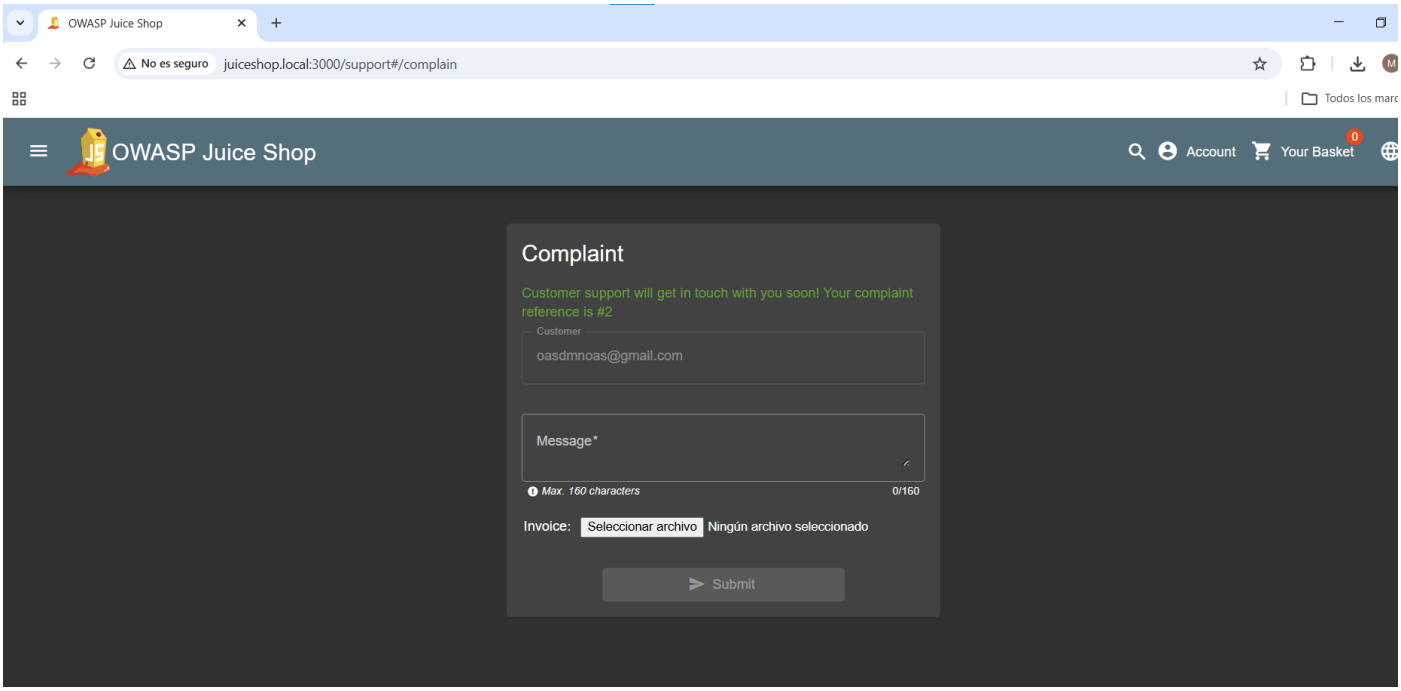
Then we created a file that is bigger than 100kB:



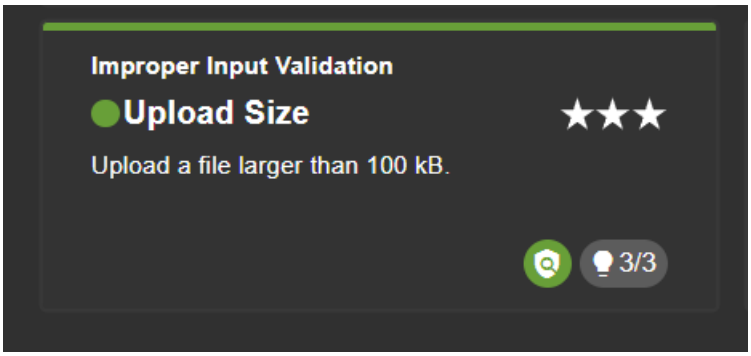
And copied the content and replaced it in the POST request instead of the old file's content.



Once we have copied, we send the POST request.



As we can see the POST request has been successful because the message in the web page is of a successful attempt to submit the complaint.



We have finished the task.

Vulnerability Analysis

Observability Failures

To explain this vunerability we need to know what is observability, observability is the capacity to understand the internal state of a system from the data generated from the system it allows to analyse the behaviour of applications and infrastructure using metrics and logs. It is different from traditional monitoring because observability does not rely in only alerts but it relies on new questions to the system and finds the root cause of any problem.

This vulnerability is part of OWASP Top 10 vulnerability number 9 (Security logging and Monitoring Failures), without logging and monitoring many things could happen:

- Auditable events like loggings, failed loggings and high-value transactions are not logged.
- Errors and warnings generate no, inadequate or unclear log messages

- Penetration testing, and scans by DAST tools do not trigger alerts
- You are vulnerable to injection attacks on the logging or monitoring systems if log data is not correctly encoded.

To complete the task “Access Log” in juice shop, what I did was, first look for any file from configuration that normally exists, and I found robots.txt that had an output which showed “user-agent: * Disallow:/ftp” which means that /ftp existed and then I tried looking into “juiceshop/ftp” and found a page where files that are not meant to see were, I found a file called incident-support.kdbx and then download it but I was not able to read it, then I thought that if the file was named with support then a /support directory should exist and tried looking for it. After trying several URL I finally got to “/juiceshop/support/logs” and there were the log files.
The only tools used in for vulnerability was the juice shop page and the inspection tools in the own juice shop site.

Scenario: A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.

This shows a similar thing from the challenge solved due to, not detecting the attack for a long time and there is not effective logging or monitoring. In the scenario there is no effective observability.

Potential impacts in observability failure are that attackers are not detected as they could enter the system, move laterally and scale privileges without the monitors even realising it and other potential impact is that without a good log monitoring system the investigation process for attacks is more difficult for digital forensics.

Also, the enterprise that got the access logs stolen could face legal problems for not compliement of some laws or regulations.

As mitigation strategies: what we could do to prevent this vulnerability from happening is

- Ensure all login, access control, and server-side input validation failures can be logged with context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
- Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- Ensure high-value transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables.

[20] [21] [24] [25]

Improper input validation

To explain this vulnerability first we must understand what is an untrusted input, an untrusted input is an input of code whose source is an untrusted application or user such as data, query strings and POST request, any data that is not generated or managed by our application should be considered untrusted.
Untrusted input has the ability to interact with our application and modify its execution being able to change a decision made from this input or perform an action.
When the application receives untrusted input from an untrusted source that it does not expect such as an incorrect format or a value out of the range, there can be unexpected consequences.

The investigation process to solve this task (Admin Registration)in juice shop first I thought about what I wanted to achieve (here I wanted to login a new user email with admin role), then I try intercepting the request and somehow change the request that I previously sent to accomplish the objective of the task, while doing the admin registration task, first I sent a POST request to register and looked at how the POST request was using burpsuite proxy, then I realized that there were some parameters that where shown in the response from the page which were not visible in the POST request, so I tried adding that field and changing the value it had from “user” to “admin” and then I got the admin role access with a new account.

The tools used to do this are a google chrome browser to access juice shop and the burp suite proxy, which was used to intercept the request and being able to change it to the desired state to get through the registration as an admin.
I used improper input validation to make this task. This attack works because I was able to intercept the POST request using burp suite proxy and I was able to see the page’s output through the proxy and I was also able to change my request’s values in order for it to work.

Potential impact:

- Data integrity: there can be alterations in some registers that have important information.
- Security Bypass: an attacker can bypass the security layers by sending a null value or inputting unexpected characters.
- Denial of Service: if a extremely big file is sent the server can ran out of memory.

Recommended Mitigation:

- Server-side validation: this consists in never trusting in JavaScript validation from the browser because is easy to bypass.
- White-List: instead trying to find what could be “bad” you should try to only permit the things that are considered “good”.

- Strong typing: ensure that data is in the correct format for example (integer, Boolean...)
- Sanitization: clean characters that are eliminated and dangerous before they arrive to the database.

[22] [23]

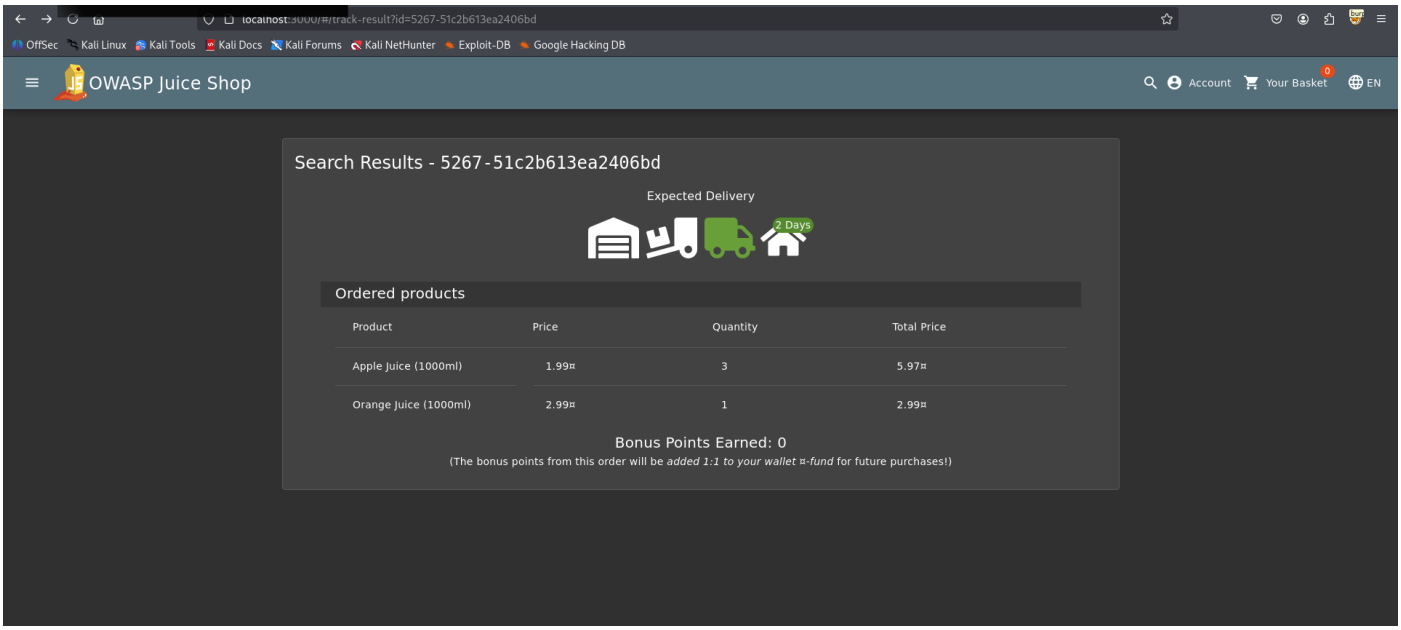
Roman Oleinikov B00177480

Solved Challenges

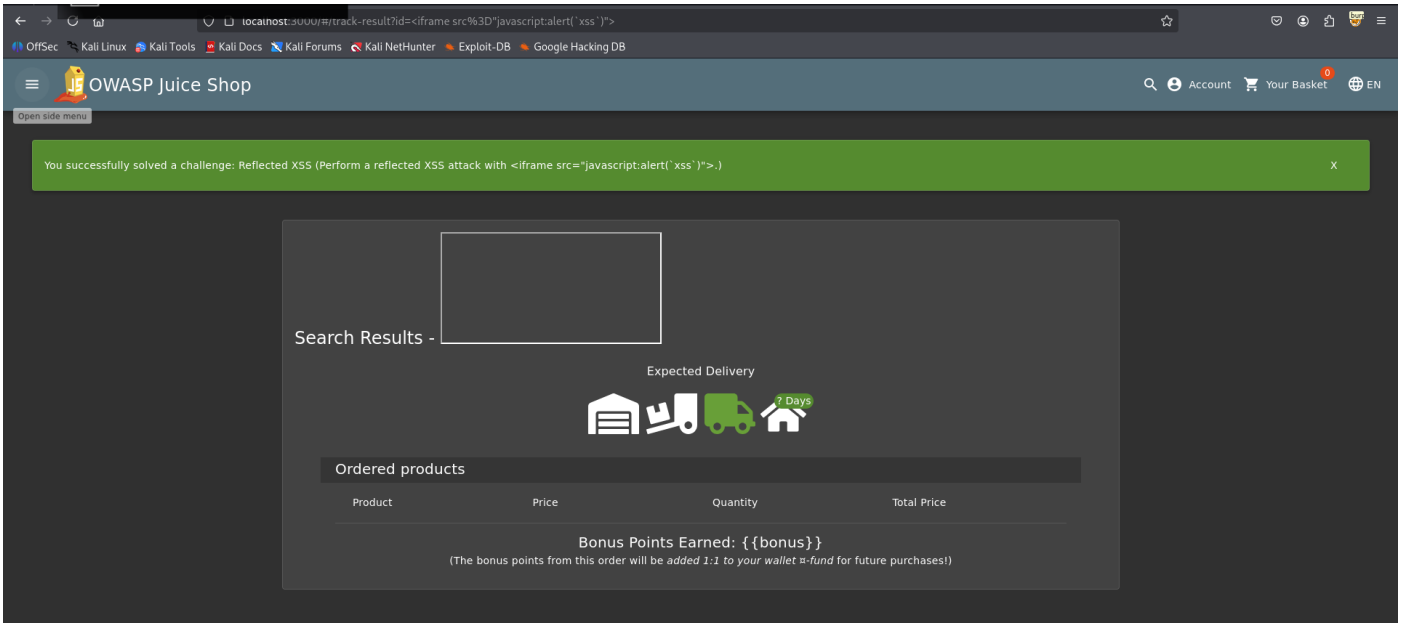
□ □ Reflected XSS

We should find and XSS vulnerability which will be sent to the server and returned to us immediately.

Using admin page, we got access to their deliveries and noticed an option to track them.



Id is used to find specific order, so we can try to insert Id instead of the script there, and it will return alert to us.

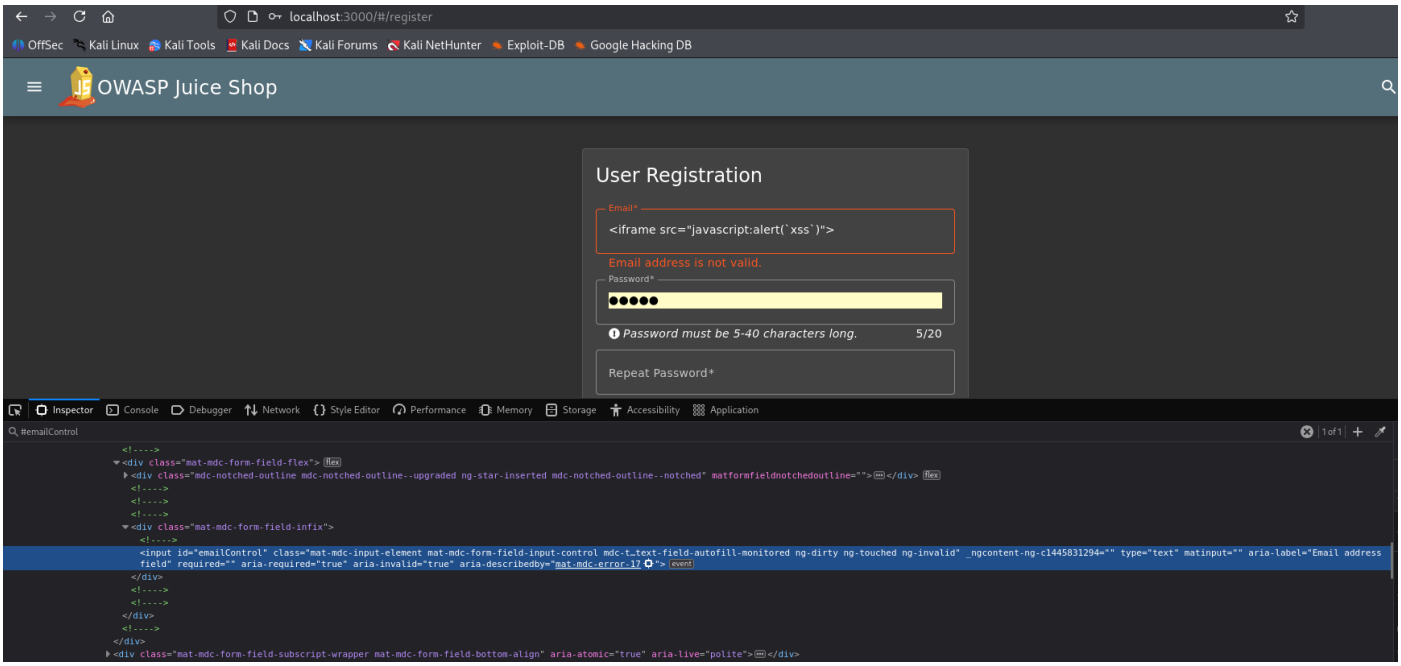


□ □ □ Client-side XSS Protection

Client-side means that the error would be displayed to us before we would submit something on the endpoint.

Also, it must be a stored XSS so that means that our script should be sorted somewhere on the server side.

After searching, email field in the user registration form was chosen as it would show an error about invalid email address even before we would submit it and email would be stored in the DB after submission.



After looking through the page source code we could find the input sanitation rule, to prevent user from entering invalid email addresses.

We would modify the script with \\ so the JSON would parse correctly through our request.

Send Cancel

Request

Pretty Raw Hex

```
Accept-Encoding: gzip, deflate, br
Content-Type: application/json
Content-Length: 268
Origin: http://localhost:3000
Connection: keep-alive
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=
Ln8BRqlWYb5eQvOgkjwOeQhDTxf6SVt2fQrTqVTxm0ZEDzP7layKNpM9X2J4; cookieconsent_status=
dismiss
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0

{
    "email":"<iframe src="javascript:alert(`xss`)">",
    "password":"12345",
    "passwordRepeat":"12345",
    "securityQuestion":{
        "id":4,
        "question":"Father's birth date? (MM/DD/YY)",
        "createdAt":"2026-04-20T14:48:24.017Z",
        "updatedAt":"2026-04-20T14:48:24.017Z"
    },
    "securityAnswer":"123"
}
```

Response

Pretty Raw Hex Render

```
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 1644

{
    "error":{
        "message":
        "Expected ',' or '}' after property value in JSON at position 23 (line 1
        column 24)",
        "stack":
        "SyntaxError: Expected ',' or '}' after property value in JSON at positio
        n 23 (line 1 column 24)\n    at JSON.parse (<anonymous>)\n    at jsonPars
        er (/home/roman/Documents/WebSec Project/juice-shop/build/server.js:319:3
        3)\n    at Layer.handle [as handle_request] (/home/roman/Documents/WebSec
         Project/juice-shop/node_modules/express/lib/router/layer.js:95:5)\n    a
        t trim_prefix (/home/roman/Documents/WebSec Project/juice-shop/node_modul
        es/express/lib/router/index.js:328:13)\n    at /home/roman/Documents/WebS
        ec Project/juice-shop/node_modules/express/lib/router/index.js:286:9\n
         at Function.process_params (/home/roman/Documents/WebSec Project/juice-s
        hop/node_modules/express/lib/router/index.js:346:12)\n    at next (/home/
        roman/Documents/WebSec Project/juice-shop/node_modules/express/lib/router
        /index.js:280:10)\n    at /home/roman/Documents/WebSec Project/juice-shop
        /node_modules/body-parser/lib/read.js:137:5\n    at AsyncResource.runInAs
        yncScope (node:async_hooks:214:14)\n    at invokeCallback (/home/roman/Do
        cuments/WebSec Project/juice-shop/node_modules/raw-body/index.js:238:16)\
        n    at done (/home/roman/Documents/WebSec Project/juice-shop/node_module
```

Request

Pretty Raw Hex

```
Accept-Encoding: gzip, deflate, br
Content-Type: application/json
Content-Length: 270
Origin: http://localhost:3000
Connection: keep-alive
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=
Ln8BRqlWYb5eQvOgkjwOeQhDTxf6SVt2fQrTqVTxm0ZEDzP7layKNpM9X2J4; cookieconsent_status=
dismiss
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0

{
    "email":"<iframe src=\"javascript:alert(`xss`)\">",
    "password":"12345",
    "passwordRepeat":"12345",
    "securityQuestion":{
        "id":4,
        "question":"Father's birth date? (MM/DD/YY)",
        "createdAt":"2026-04-20T14:48:24.017Z",
        "updatedAt":"2026-04-20T14:48:24.017Z"
    },
    "securityAnswer":"123"
}
```

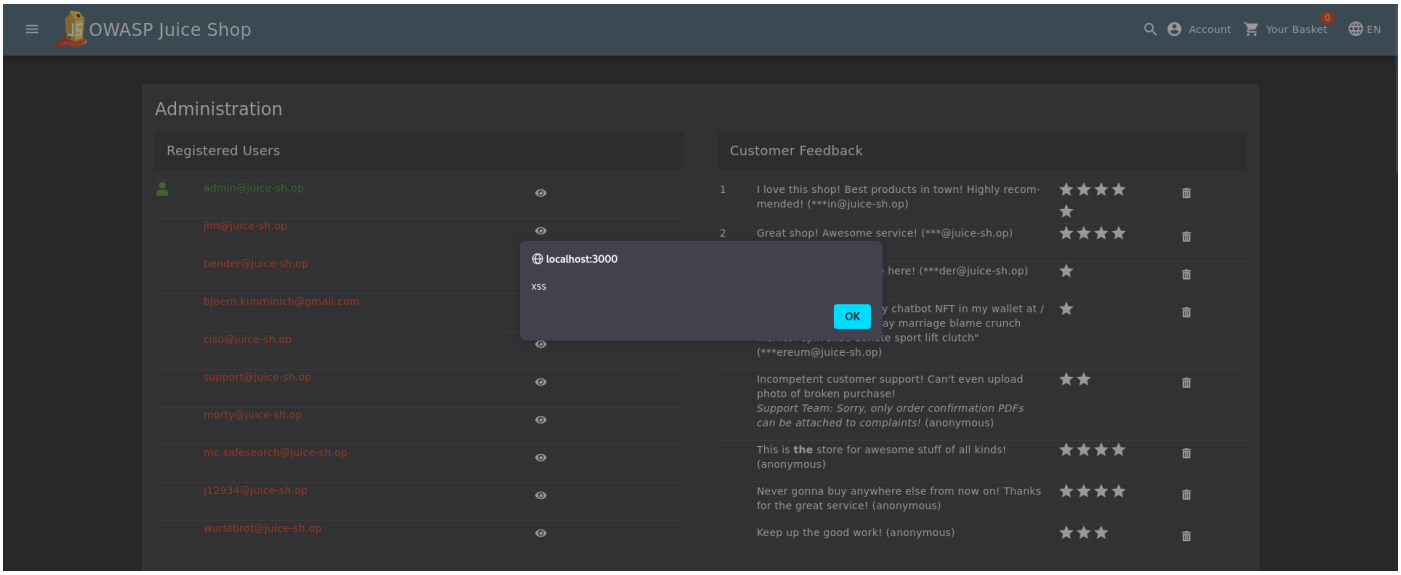
Response

Pretty Raw Hex Render

```
X-Recruiting: /#/jobs
Location: /api/Users/26
Content-Type: application/json; charset=utf-8
Content-Length: 331
ETag: W/"14b-IvbuwSMnWyFc+AyKMikdTYqBo3Y"
Vary: Accept-Encoding
Date: Mon, 20 Apr 2026 17:01:45 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
    "status":"success",
    "data":{
        "username":"",
        "role":"customer",
        "deluxeToken":"",
        "lastLoginIp":"0.0.0.0",
        "profileImage":"/assets/public/images/uploads/default.svg",
        "isActive":true,
        "id":26,
        "email":"<iframe src=\"javascript:alert(`xss`)\">",
        "updatedAt":"2026-04-20T17:01:45.532Z",
        "createdAt":"2026-04-20T17:01:45.532Z",
        "deletedAt":null
    }
}
```

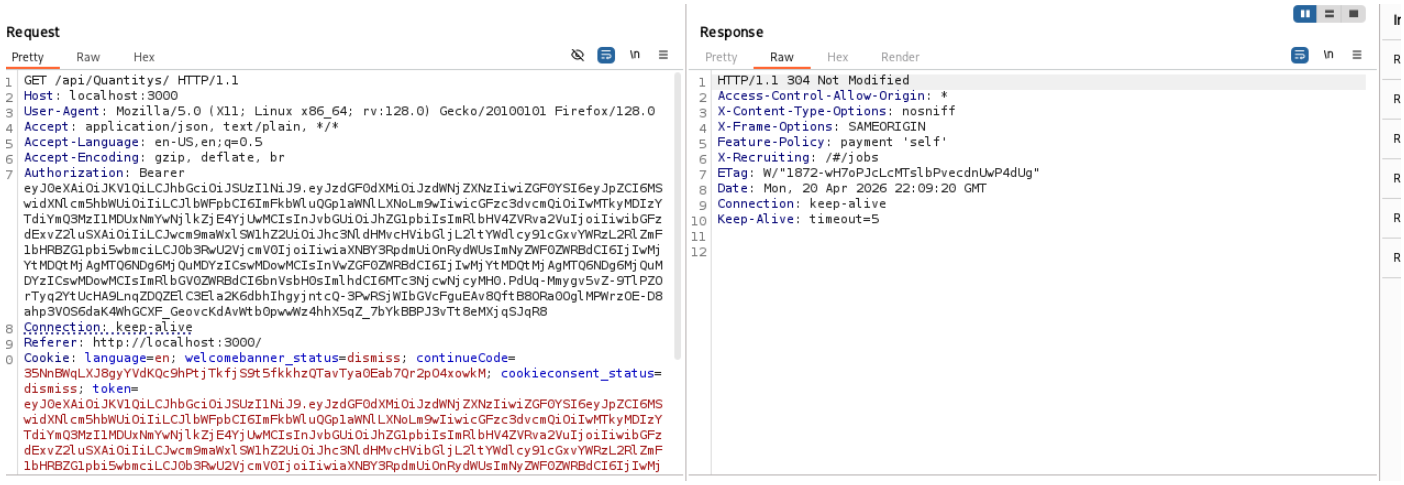
Now our script is successfully stored on the server. We could verify it by logging in as admin and visit the administration page.



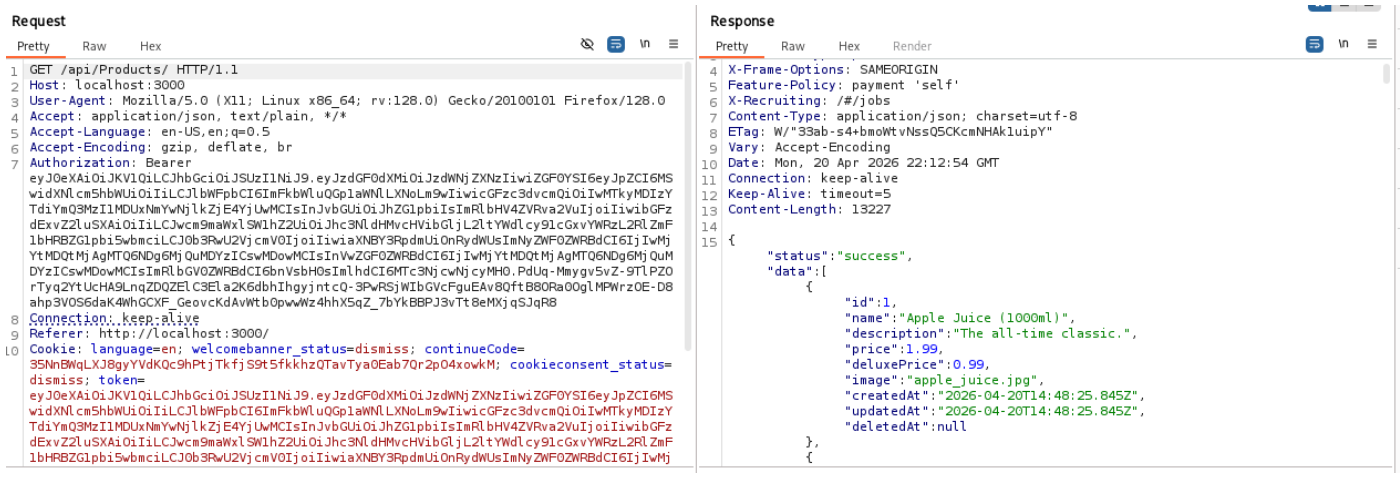
□ □ □ API-only XSS

To complete the challenge, we need to insert the given script through an API call to the server.

First, we check for api calls in HTTP history of our proxy and find /api/Quanitys call.

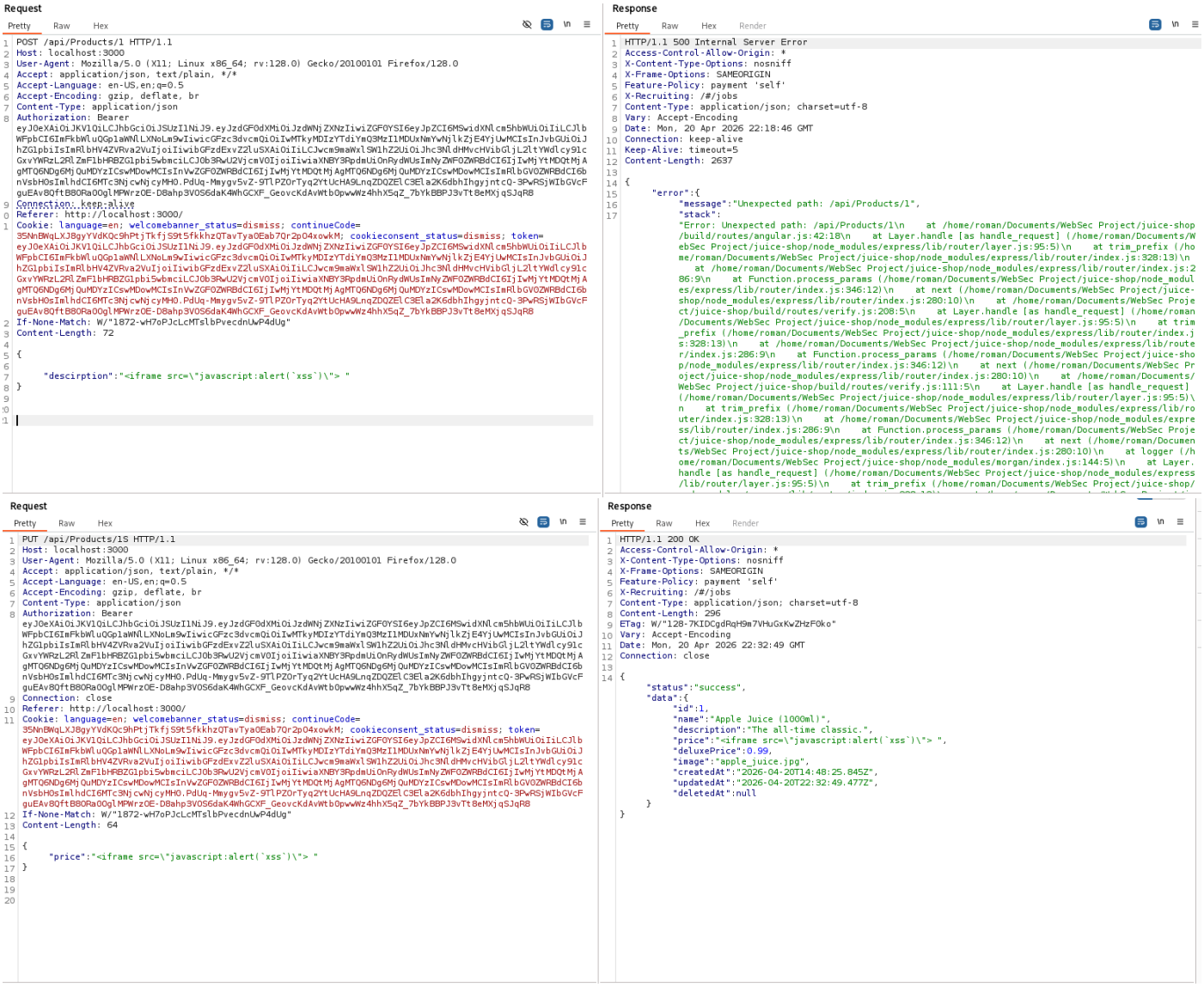


Attempts to modify it were unsuccessful. Next step was to try other API requests, request to the Products page was successful so we could test further with it.



Now we can try to modify it and insert our script inside the products data.

Post request would not work as we try to update the existing product and not create a new one, so we will Use PUT method instead.



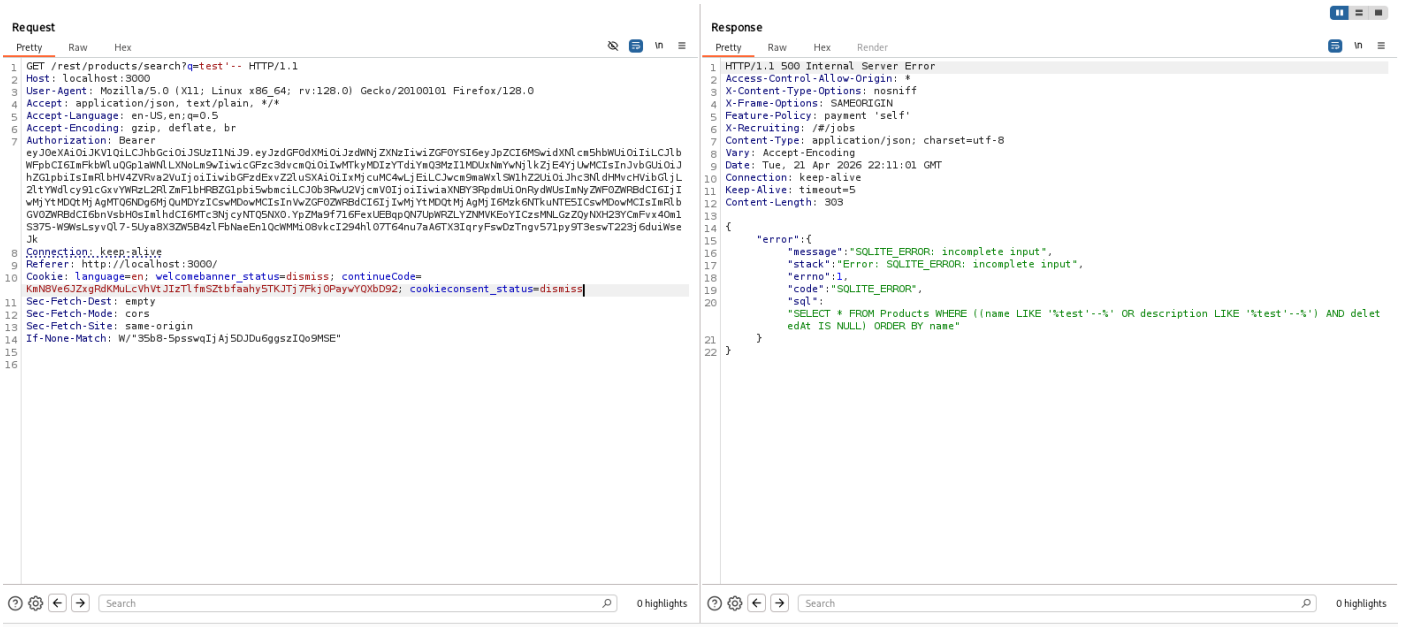
As we are using PUT method, we need to change the request by adding Content-Type: application/json. This way the server will parse the body of our request as a json, otherwise it will drop a 404 error, treating it as a plaintext.

Injections

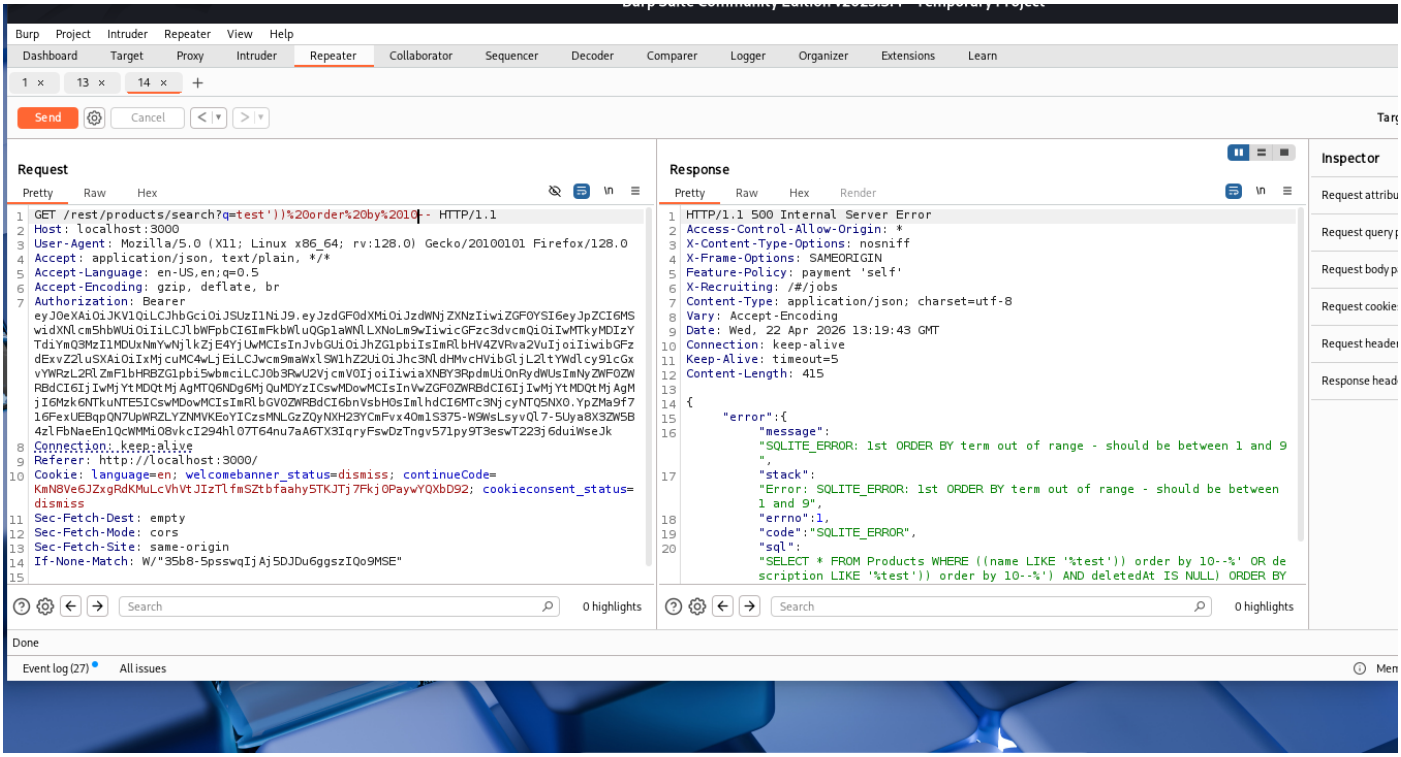
□ □ □ Database Schema

The most logical endpoint for this challenge would be product search, as it actively calls for the Database for products.

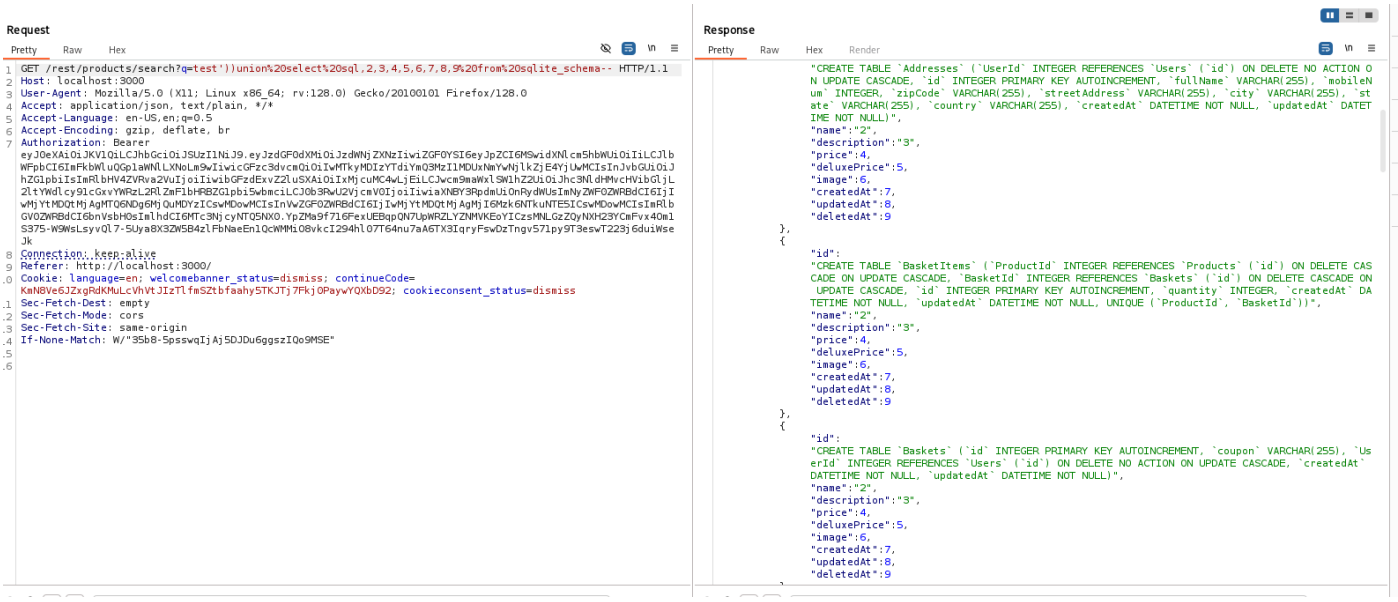
Firstly, we test if search is vulnerable to SQL injections with simple ones. We receive errors where we can see that Sqlite is used on the server. We can also see how our queries look like, so it would be easier to create future injections.



Now we can use UNION SELECT query to retrieve Database schema, but to do that we need to get the number of columns returned from the original query. The most basic way to do this is by using 'ORDER BY' until we receive an error.



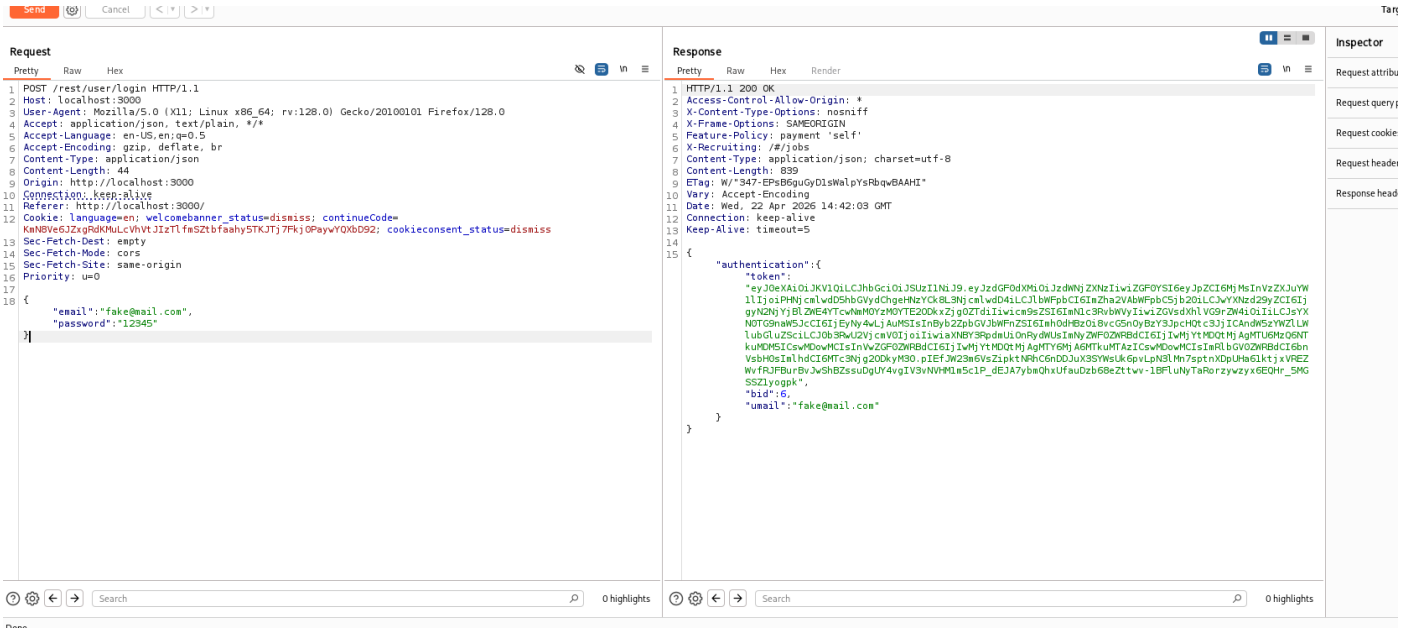
Original query returns 9 columns, also '%20' is used instead of spaces in the query to make it work. We only need to replace one of the columns with 'sql', this is the column with text that describes the object.



□ □ □ □ Ephemeral Accountant

For this challenge, we need to log in with a non-existent account.

First, we will intercept the usual login attempt.



If credentials are valid, authentication token is returned. So that is what we are expected to get with our ephemeral account.

From the previous challenge we got Database schema, there is a user record information which we can use to mimic a registered user with union select query.

```
},
{
    "id":
    "CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR(255) DEFA
    ULT '', `email` VARCHAR(255) UNIQUE, `password` VARCHAR(255), `role` VARCHAR(255) DEFAULT '
    customer', `deluxeToken` VARCHAR(255) DEFAULT '', `lastLoginIp` VARCHAR(255) DEFAULT '0.0.0
    .0', `profileImage` VARCHAR(255) DEFAULT '/assets/public/images/uploads/default.svg', `totp
    Secret` VARCHAR(255) DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT
    NULL, `updatedAt` DATETIME NOT NULL, `deletedAt` DATETIME)",
    "name":"2",
    "description":"3",
    "price":4,
    "deluxePrice":5,
    "image":6,
    "createdAt":7,
    "updatedAt":8,
    "deletedAt":9
},
{
    "id":
```


Using information from the schema, query is created to mimic the real user with target email 'WHERE '1'='1'' is added at the end so our query would run successfully.

Vulnerability Analysis

XSS

- XSS is a type of injection attacks categorized as A05 in OWASP Top 10 2025 [19]. Malicious scripts are injected into trusted web application and are then sent to normal users. It may access sensitive information stored by browser or change the content of a webpage [15].
- XSS can be Server side or Client side. Server side XSS is included in HTTP response from the server and can be stored or reflected. Client side XSS occurs when untrusted user supplied data is used to update the DOM and can be stored or reflected as well.
- OWASP Juice Shop is vulnerable for Server side XSS injections both reflected – like Reflected XSS challenge, and stored XSS – Client-side XSS protection challenge where we inserted it into the users list.
- To mitigate this type of vulnerability, output encoding is strongly advised for various contexts in the web application. It will prevent web application from treating any output as code, and only as text.
- Another mitigation technique is HTML Sanitization; it strips dangerous HTML from the variable and returns a safe string [16].

SQL injections

- SQL injections are another type of injections attack categorized as A05 in OWASP Top 10 2025. In such attack SQL query is injected into the input data from the client to application [17].
- Successful SQL injection may give access to sensitive information from the database, and ability to modify it according to attackers' goal.
- In OWASP Juice Shop web application it can be seen how the one vulnerability led to another – after getting access to Database Schema, we malicious actor understood the structure of USER table and could create a new one in login form. Potentially, he could modify the Database or Delete it commpletely.
- Primary mitigation technique is Prepared Statements with Parameterized Queries. This will allow database to distinguish between code and data, and attacker won't be able change the intended query, as all his input will be passed as a parameter inside of the original query [18].

SAST

Tool and methodology

To perform SAST on the OWASP Juice Shop web application, Semgrep community edition was used.

Semgrep uses pattern-based rules to detect insecure coding practices, data-flow issues, and known vulnerability patterns. Its rules can also express semantic patterns such as “user input flows into a SQL query” or “unescaped data is rendered into HTML,” which makes it effective in identifying real vulnerabilities in web applications. In other words, Semgrep is capable of tracking the flow of data through the code, and identify vulnerabilities based on how it is handled later by the system.

For example, tool flags sequelize.query()because user input is used directly in the query string, and not because of the function itself.

Other reason to use Semgrep was that it scans source code directly, without using any supporting instruments. This makes it faster than other options, such as npm audit that requires creation of lock-file before scan.

Each finding includes:

- The rule name (e.g., `express-sequelize-injection`)
- A clear description of the vulnerability
- A link to further explanation
- The exact code snippet causing the issue

SQL injection in Search and login.

Those vulnerabilities are located in search.ts and login.ts files.

Found with Semgrep rule: `express-sequelize-injection.`

Application uses the following code:

```
models.sequelize.query(
    `SELECT * FROM Users WHERE email = '${req.body.email || ''}'
     AND password = '${security.hash(req.body.password || '')}'
     AND deletedAt IS NULL`
 )
```

and

```
models.sequelize.query(
    `SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
    description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`
 )
```

It can be seen how user input is sent directly to the database without any processing or filtering. Login form hashes security password, but it does not prevent potential SQL injections and with right execution password would not be required anyway.

Those vulnerabilities enable such injections as “`' OR 1=1—`” to allow authentication bypass, or “`' UNION SELECT ...—`” which may lead to potential exfiltration of sensitive data from the database.

Cross-Site Scripting vulnerabilities

Vulnerabilities are found in files views/promotionVideo.pug, routes/chatbot.ts, data/static/codefixes/restfulXssChallenge_2.ts

Pug file is used to dynamically create pages; it provides template and inserts the dynamic data like surnames in there.

In views/promotionVideo.pug there is a detected usage of '`!=` or `!{...}`' in a Pug template. It means that the XSS protection in Pug that filters out scripting characters is not working, so any values there will be treated by browser as HTML or JavaScript. That opens path to injecting the page with malicious scripts.

In routes/chatbot.ts file vulnerability found in the following code:

```
body: bot.training.state
 ? bot.greet(`${user.id}`)
 : `${config.get<string>('application.chatBot.name')} isn't ready...`
```

User id value there is used as a part of the response from chat bot. It means that if the user id will be changed to malicious script, it will still be executed by the browser.

In data/static/codefixes/restfulXssChallenge_2.ts file there is an attempt from the developer to filter out characters used in XSS by replacing them:

```
49¦ tableData[i].description = tableData[i].description.replaceAll('<',
'&lt;').replaceAll('>', '&gt;')
```

This was flagged as a vulnerability because it can be easily omitted by using other payloads or encoding them beforehand, so the '>' and '<' characters will not appear in XSS script.

Path traversal

In the following files there is a similar pattern of path resolving, which allows users to traverse through the file system.

- `routes/fileServer.ts`
- `routes/keyServer.ts`
- `routes/logfileServer.ts`
- `routes/quarantineServer.ts`

Pattern looks the following way:

```
res.sendFile(path.resolve('encryptionkeys/', file))
```

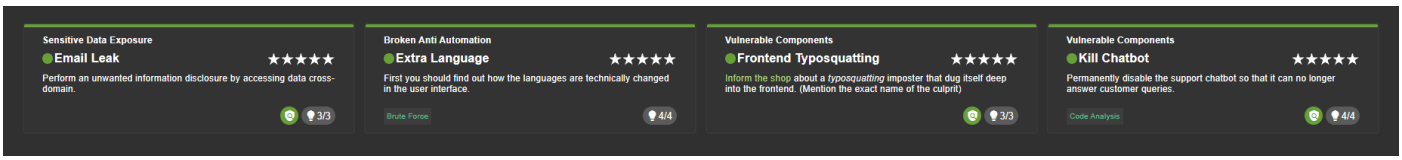
'file' value is taken from the user. So potentially malicious actor can enter something like:

```
../../../../etc/passwd
```

It will allow attackers to gain access to files outside of the intended directory. Such vulnerability may allow attackers to gain access to user credentials, or other sensitive files.

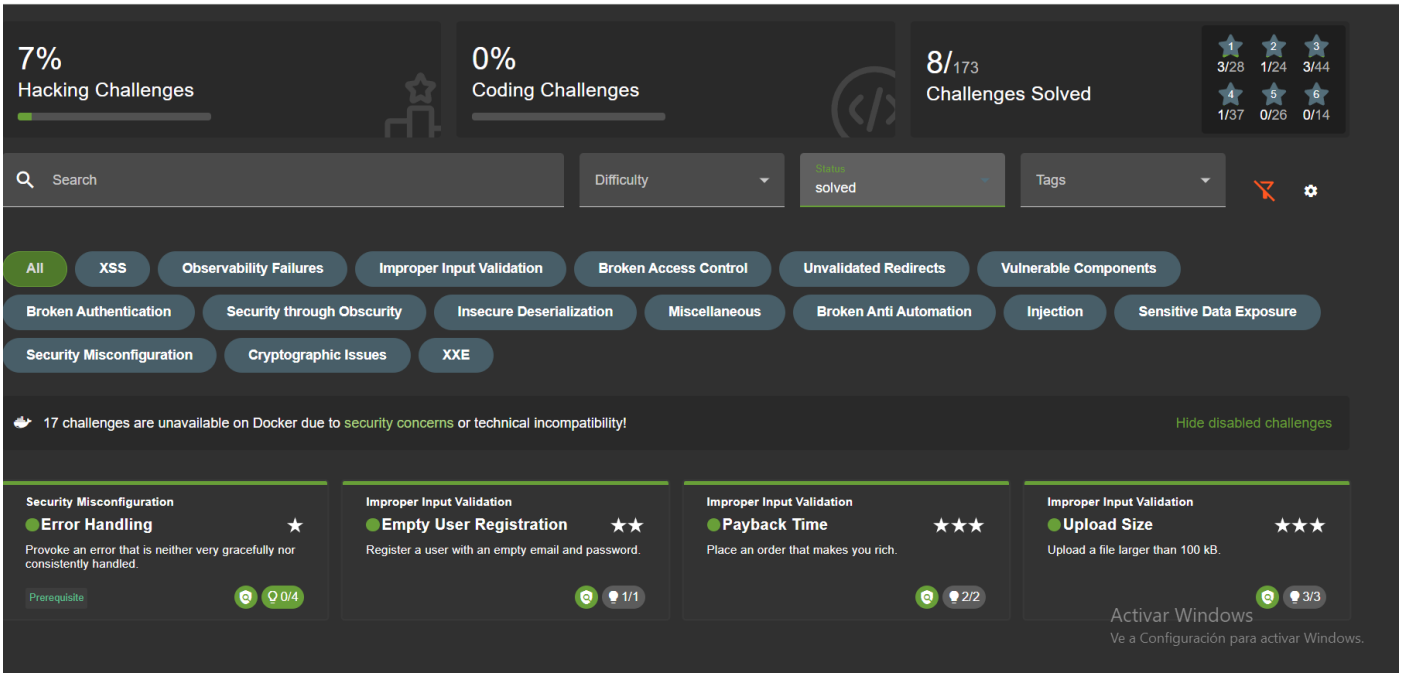
Juice Shop solved Challenges

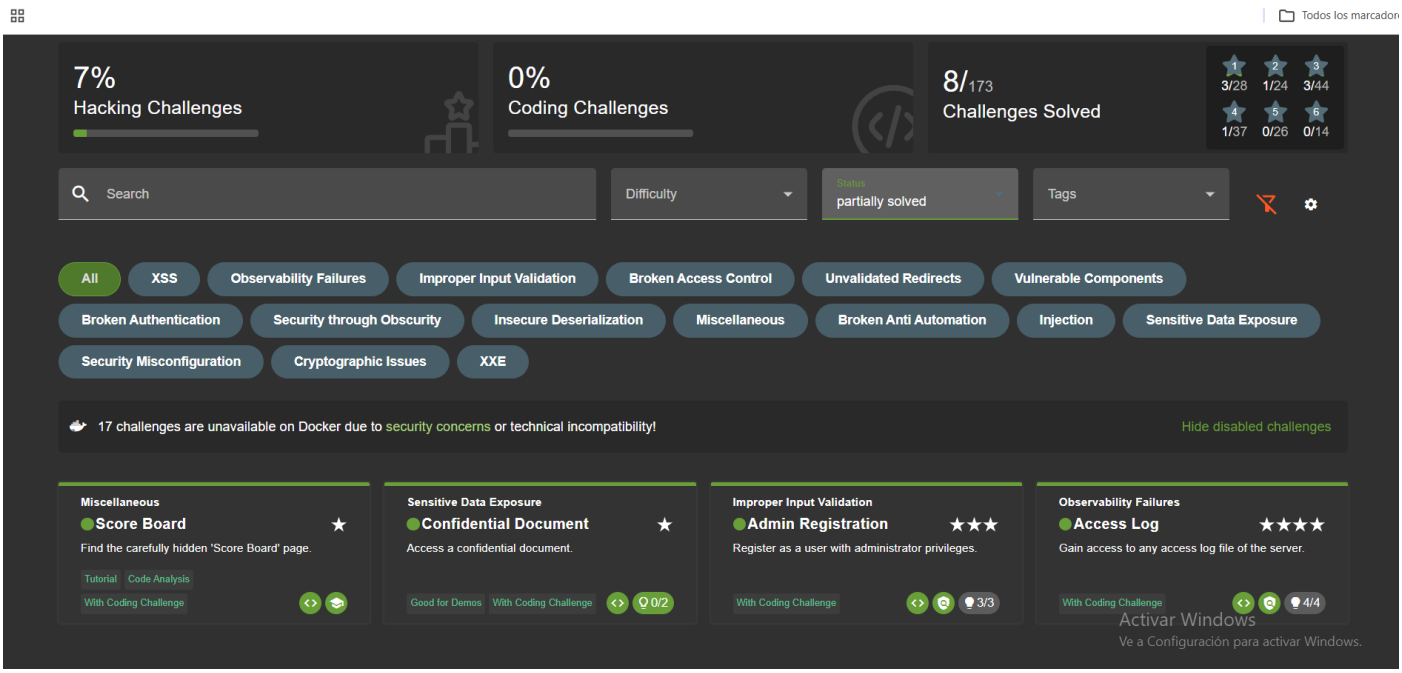
Change Bender's Password was completed, but for some reason it wasn't counted in the scoreboard.

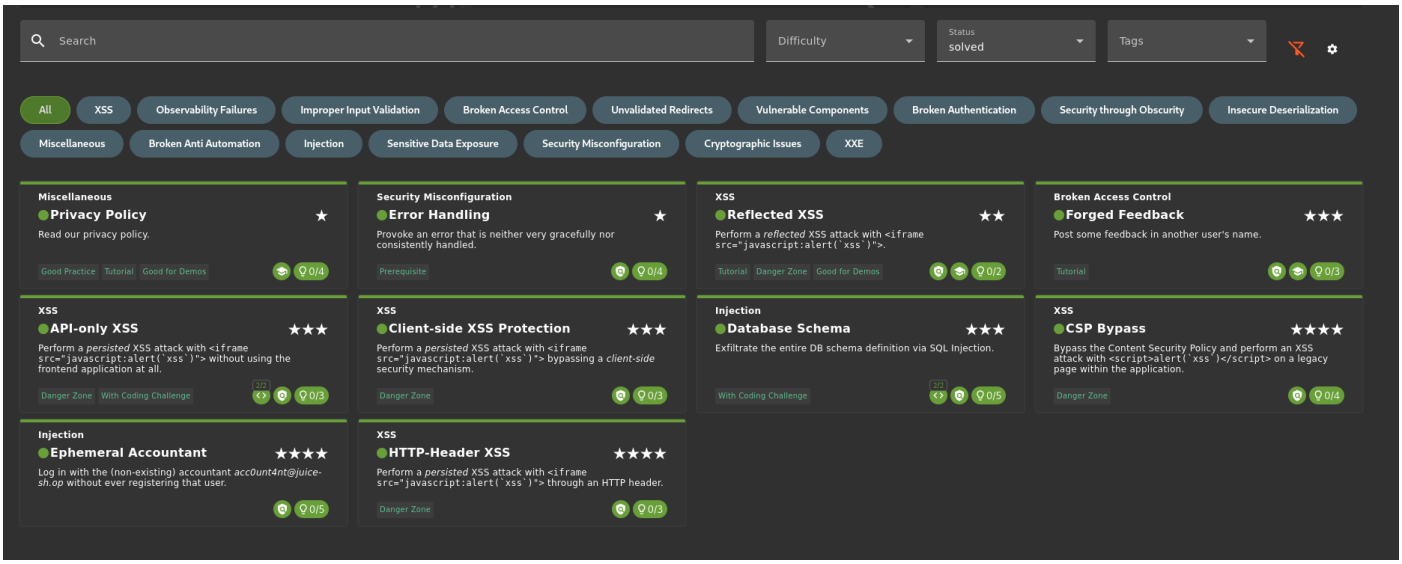


Cross-site Imaging was complete but didn't count on the site for unknown reason.

Admin Registration and Access Log challenges were not in the solved tab for some reason.







DAST

Introduction

The main goal of this exercise was to apply automated dynamic scanning in practice, analyse the vulnerabilities detected by the tool, and manually verify at least one of the findings to better understand its real impact. Since Juice Shop is intentionally insecure, the purpose was not to “discover unknown flaws” but to demonstrate how common web vulnerabilities appear and how they can be identified.

Testing Methodology

OWASP ZAP was chosen because it is free, widely used, and suitable for testing web applications. It supports both passive scanning (just observing traffic) and active scanning (sending malicious inputs to find vulnerabilities).

For this assessment, ZAP was configured with default settings and the following parameters:

- Target: http://juiceshop.local:3000
- Scope: full application
- Risk levels: all (High, Medium, Low, Informational)
- Confidence levels: all included
- False positives excluded
- Traffic captured via Google Chrome (proxy setup)

ZAP acted as a proxy and intercepted all browser requests while interacting with the application.

Scan Process

The scan started with spidering (crawling), which discovered 135 endpoints. After that, an active scan was launched to test those endpoints using predefined attack patterns.
In total, 13 alerts were identified across 9 different categories. ZAP also reported a 32% network failure rate, meaning some requests failed or endpoints did not respond properly. This is expected for Juice Shop, as it is intentionally unstable in some areas.
The scan mostly used GET requests (around 98%), with very few POST requests. Memory usage reached about 85%, which suggests the scan was moderately heavy but still manageable in a local environment.

Manual Validation

After the automated scan, one key vulnerability (SQL Injection) was manually tested. This step is important because automated tools can sometimes report false positives. By reproducing the issue manually and analysing responses, it is possible to confirm whether the vulnerability is real.

Findings Overview

The scan produced 13 total alerts. Most of them were Medium severity (5 findings), with one High severity issue (SQL Injection). There were also several Low and Informational findings.

Even though not all issues are critical on their own, together they show that the application has weak security configuration overall.

Detailed Findings

SQL Injection

The most serious issue found was a SQL Injection vulnerability in the login functionality. ZAP detected it by sending a single quote (') in the email field of a login request.

The server responded with a 500 Internal Server Error, and the response included a full SQLite error message with the SQL query:

SELECT * FROM Users WHERE email = ''' AND password = '...' AND deletedAt IS NULL

This clearly shows that user input is directly inserted into the SQL query without proper sanitisation. Because of this, an attacker can manipulate the query logic.

For example, a payload like:

' OR '1'='1

could bypass authentication completely, since the condition would always be true.

Another issue here is that the application exposes detailed database errors to the user. This gives attackers useful information about the database structure.

Fix:

Use parameterised queries (prepared statements)

Do not display database errors in responses

Validate user input properly

This vulnerability was manually confirmed, so it is definitely real and exploitable.

Missing Content Security Policy

The application does not include a Content-Security-Policy (CSP) header in its responses. CSP is used to control which resources (scripts, styles, etc.) can be loaded by the browser.

Without CSP, the application is more vulnerable to attacks like Cross-Site Scripting (XSS), because there are no restrictions on where scripts can come from.

Although some other headers (like X-Frame-Options) are present, CSP is still missing, which leaves a gap in protection.

Fix:

Add a CSP header that restricts allowed sources. It is recommended to start in “report-only” mode first.

CORS Misconfiguration

The application allows requests from any origin using:

Access-Control-Allow-Origin: *

This means that any website can send requests and read responses from this application (for public endpoints).

While this is less dangerous for authenticated data, it still exposes unnecessary information and could be abused in certain scenarios.

Fix:

Restrict CORS to trusted domains instead of allowing all origins.

No HTTPS

All traffic is sent over HTTP instead of HTTPS. This means data is not encrypted and can be intercepted.

This includes login credentials, session tokens, and other sensitive data.

In this case, it is expected because the app runs locally, but in a real system this would be a serious issue.

Fix:

Enable HTTPS, Redirect HTTP to HTTPS Use HSTS

Missing Anti-Clickjacking Protection

Some responses do not include X-Frame-Options or CSP frame restrictions. This makes the application potentially vulnerable to clickjacking attacks, where a user is tricked into clicking hidden elements inside an iframe.

Even though the risk is not very high for all endpoints, it is still a bad practice to leave this unconfigured.

Fix:

Add X-Frame-Options: DENY or SAMEORIGIN, or use CSP frame-ancestors.

Session ID in URL

Session identifiers are included in URLs as query parameters, for example:

...&sid=oSwa6yLINOmTUTEQAAPi

This is risky because URLs can be logged, stored in browser history, or leaked via headers.

If an attacker gets access to these URLs, they may hijack user sessions.

Fix:

Use secure cookies instead of putting session IDs in URLs.

Manual SQL Injection Validation

To confirm the SQL Injection issue, a manual request was sent:

{ "email": "'", "password": "12345678" }

The server returned a 500 error with a full SQL query in the response. This confirms that input is not properly handled.

This proves that the vulnerability is real. An attacker could continue testing more payloads to bypass login or extract data.

Manual validation increases confidence in the result, since it shows actual behaviour rather than just automated detection.

Conclusion

This DAST assessment successfully identified multiple vulnerabilities in the OWASP Juice Shop application. The most critical issue was the SQL Injection vulnerability in the login endpoint, which was confirmed manually and can directly affect authentication and database security.

Other findings, such as missing HTTPS, CSP, and CORS misconfiguration, show that the overall security setup is weak. Even though some of these issues are not immediately exploitable, together they increase the attack surface significantly.

The exercise also shows why combining automated tools with manual testing is important. ZAP helped quickly find potential problems, while manual validation helped confirm them and understand their impact.

Since Juice Shop is intentionally vulnerable, these results are expected. However, they reflect real-world issues that can appear in poorly secured applications. The recommended fixes (such as input validation, proper headers, and secure communication) are standard practices that should be applied in any production system.

References

[1]

OWASP, "A3:2017-Sensitive Data Exposure | OWASP," *owasp.org*, 2017. https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure

[2]

"A02 Cryptographic Failures - OWASP Top 10:2021," *Owasp.org*, 2021. https://owasp.org/Top10/2021/A02_2021-Cryptographic_Failures/

[3]

mitre, "CWE - CWE-200: Exposure of Sensitive Information to an Unauthorized Actor (4.1)," *cwe.mitre.org*, Jul. 19, 2006. https://cwe.mitre.org/data/definitions/200.html

[4]

"Sensitive Data Exposure: How To Protect Against Data Leaks – PreEmptive," *PreEmptive In-App Protection*, Dec. 04, 2025. https://www.preemptive.com/blog/sensitive-data-exposure/ (accessed Apr. 21, 2026).

[5]

"A05 Security Misconfiguration - OWASP Top 10:2021," *Owasp.org*, 2021. https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/

[6]

sbow, "Security Misconfiguration (A02:2025): How Incorrect Settings Expose Your Entire Infrastructure," *IntelligenceX Blog, a Cyber Security Blog and Hacking News*, Jan. 16, 2026. https://blog.intelligencex.org/security-misconfiguration-owasp-a02-2025-complete-guide (accessed Apr. 21, 2026).

[7]

GeeksforGeeks, “OWASP Top 10 Vulnerabilities,” *GeeksforGeeks*, Dec. 10, 2020. https://www.geeksforgeeks.org/ethical-hacking/owasp-top-10-vulnerabilities-and-preventions/ (accessed Apr. 21, 2026).

[8]

E. Kost, “What Caused the Uber Data Breach in 2022?,” *Upguard.com*, Nov. 18, 2022. https://www.upguard.com/blog/what-caused-the-uber-data-breach (accessed Apr. 21, 2026).

[9]

H. Dwibhashyam, “Mitigating OWASP API Security Top 10 Risks – 2023 using F5 BIG-IP,” *DevCentral - an F5 Community*, Aug. 20, 2025. https://community.f5.com/kb/technicalarticles/mitigating-owasp-api-security-top-10-risks-%E2%80%93-2023-using-f5-big-ip/342988

[10]

OWASP, “API2:2023 Broken Authentication - OWASP API Security Top 10,” *owasp.org*, 2023. https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/

[11]

N. Maric, “Broken Authentication: Impact, Examples, Fixes,” *Bright Security*, Jan. 12, 2024. https://brightsec.com/blog/broken-authentication-impact-examples-and-how-to-fix-it/#data

[12]

Owasp, “A01 Broken Access Control - OWASP Top 10:2021,” *Owasp.org*, 2021. https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/

[13]

PortSwigger, “Access control vulnerabilities and privilege escalation | web security academy,” *portswigger.net*, 2023. https://portswigger.net/web-security/access-control

[14]

N. Marić, “Broken Access Control: Attack Examples and 4 Defensive Measures,” *Bright Security*, Dec. 29, 2023. https://brightsec.com/blog/broken-access-control-attack-examples-and-4-defensive-measures/

[15]

OWASP, “Cross Site Scripting (XSS) | OWASP,” *Owasp.org*, 2020. https://owasp.org/www-community/attacks/xss/

[16]

OWASP, “Cross Site Scripting Prevention - OWASP Cheat Sheet Series,” *cheatsheetseries.owasp.org*, 2020. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html#html-sanitization

[17]

OWASP, “SQL Injection,” *OWASP*, 2024. https://owasp.org/www-community/attacks/SQL_Injection

[18]

OWASP, “Query Parameterization · OWASP Cheat Sheet Series,” *Owasp.org*, 2019. https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

[19]

OWASP, “A05 Injection - OWASP Top 10:2025 RC1,” *Owasp.org*, 2025. https://owasp.org/Top10/2025/A05_2025-Injection/

[20]

“A09 Security Logging and Monitoring Failures - OWASP Top 10:2021,” *Owasp.org*, 2021. https://owasp.org/Top10/2021/A09_2021-Security_Logging_and_Monitoring_Failures/index.html

[21]

J. G. Camaño, “Observabilidad: qué es y por qué es clave para tu infraestructura IT,” *CTL*, Apr. 08, 2026. https://ctl.tech/blog/observabilidad-que-es/ (accessed Apr. 27, 2026).

[22]

“What is Improper Input Validation? | Mayhem,” *www.mayhem.security*. https://www.mayhem.security/blog/what-is-improper-input-validation

[23]

Cymulate, “Input Validation,” *Cymulate*, Apr. 29, 2025. https://cymulate.com/cybersecurity-glossary/input-validation/

[24]

“¿Qué es Robots.txt? | Guía de archivos robots.txt,” *Cloudflare.com*, 2025. https://www.cloudflare.com/es-es/learning/bots/what-is-robots-txt/

[25]

Where are Apache file access logs stored, “Where are Apache file access logs stored?,” *Unix & Linux Stack Exchange*, May 18, 2012. https://unix.stackexchange.com/questions/38978/where-are-apache-file-access-logs-stored